



Islington college
(इस्लिङ्गटन कलेज)

Module Code & Module Title

CS6P05NI Project

Interim Report

25% Individual Coursework

Submission: Final Submission

AY 2025 2026

Academic Semester: Autumn Semester 2025

Credit: 30 Credit Year Long Module

Student Name: Prabesh Sundar Taksari

London Met ID: 23047464

College ID: NP01NT4A230138

Assignment Due Date: November 23, 2025

Assignment Submission Date: Wednesday, December 30, 2025

Internal Supervisor: Prashant Pudasaini

External Supervisor: Suraj Neupane

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Acknowledgement

I would like to appreciate my outstanding supervisors, Mr. Prashant Pudasaini and Mr. Suraj Neupane, who have supported me in every way possible at any stage of this project and helped me out.

The supervision of Mr. Prashant Pudasaini as the Internal Supervisor has played a crucial role in offering the academic guidance of the project and ensuring that the project meets all the required university and academic standards. In the same way, the supervision and expertise of the External Supervisor, Mr. Suraj Neupane, with the valuable experience in the industry and broader perspective he gave me, have assisted me considerably in the development of the project concept and through the implementation of the project concept.

I would like to express my deep gratitude to both supervisors who support and encourage me to discover new directions of research and provide regular and informative feedback.

Lastly, I would like to say that I appreciate Islington College a lot, which gave me a chance to explore, learn and work in the sphere that I am interested in.

Abstract

The Intelligence Recon System (IRS) is an AI-based cyber security reconnaissance platform that will automate and centralize the vulnerability assessment processes. It combines several command-line security tools into one, web-based interface, which allows asynchronous, parallel execution to speed up and more efficient scan. An AI decision node is a smart tool that recommends and orders tools depending on target context and user intent and minimizes the influence of human bias in the triage process and configuration errors. The system structures raw outputs and aggregates them to produce easy to read and understand data, which facilitates the analysis of results and simplifies the work of security teams. A live dashboard is used to show the progress of the scans, assets found and vulnerabilities discovered and persistent storage stores the entire historical data to be analyzed and audited. Standardized, shareable reports are also created by the IRS to assist in complying, recording incidents, and communicating with stakeholders. Replacing the manual recon with an automated intelligent system, the platform enhances the organizational security posture and enables advanced reconnaissance to be available even to smaller teams.

Table of Contents

1. INTRODUCTION.....	1
1.1. Introduction to Topic.....	1
1.2. Problem Statement	3
1.3. Project as Solution	3
1.4. Aim and Objectives	4
1.4.1. Aim	4
1.4.2. Objectives.....	4
1.5. Structure of the Report.....	5
1.5.1. Background	5
1.5.2. Development	5
1.5.3. Analysis of Progress	5
1.5.4. Further Work.....	5
2. BACKGROUND	6
2.1. Target Client.....	6
2.2. Understanding the Project.....	6
2.2.1. Project Description	6
2.2.2. Project Deliverables.....	7
2.3. Similar Project Review	7
2.3.1. Project 1: reNgine.....	7
2.3.2. Project 2: Pentest-Tools	8
2.3.3. Project 3: SOCRadar	9
2.4. Comparison Table.....	10
2.5. Analysis and Conclusion of Comparison.....	11
3. DEVELOPMENT	12
3.1. Considered Methodology	12
3.1.1. Waterfall methodology	12
3.1.2. V-Model Methodology.....	12
3.2. Selected Methodology.....	12
3.2.1. Agile Framework.....	12
3.3. Work breakdown structure.	14
3.3.1. Previous WBS	14
3.3.2. Updated WBS.....	14
3.4. Gantt chart	15
3.4.1. Previous Gantt Chart	15

3.4.2.	Updated Gantt Chart	17
3.5.	Milestone.....	21
3.5.1.	Progress of Milestones	22
4.	ANALYSIS OF PROGRESS	23
4.1.	Progress Table.....	23
4.2.	Progress Review	25
4.2.1.	Current Scenario of progress.....	25
4.2.2.	Progress Timeline.....	25
4.2.3.	Action Plan	26
5.	FURTHER WORK.....	26
5.1.	Phases to complete.....	26
5.1.1.	Remaining Development	27
5.1.2.	Integration and Testing	27
5.1.3.	Control and Optimization	27
5.1.4.	Final Optimization.....	27
6.	REFERECES	28
7.	APPENDIX.....	31
7.1.	Appendix 1: Waterfall Methodology.....	31
7.2.	Appendix 2: V-Model Methodology	32
7.3.	Appendix 3: Agile Methodology.....	34
7.4.	Appendix 4: Scrum Methodology	35
7.4.1.	Role of scrum	35
7.4.2.	Justification for choosing Methodology.....	37
7.5.	Appendix 5: Previous WBS	38
7.6.	Appendix 6: Updated WBS	39
7.7.	Appendix 7: Milestone.....	40
7.8.	Appendix 8: Findings of pre-survey	43
7.8.1.	Findings of the pre-survey	43
7.9.	Appendix 9: Software Requirements Specifications	52
7.9.1.	Software Requirements Specifications	52
7.10.	Appendix 10: Product Backlog.....	57
8.1.	Appendix 11: Diagrams.....	59
8.1.1.	Flow Chart	59
8.1.2.	System architecture diagram.....	60
8.1.3.	Sequence Diagram.....	61

8.1.4. Use Case Diagram	63
8.2. Development Progress.....	65
8.3. Appendix Project risks, threats, and contingency plans	87
8.4. Expected Outcomes and Deliverables	88
8.5. Deliverables	88
8.6. Resource Requirements	89
8.6.1. Hardware Requirement.....	89
8.6.2. Software Requirement.....	89

Table of Figures

Figure 1 Vulnerability Landscape (Alexander Kolesnikov, 2024).	1
Figure 2 Manual Reconnaissance vs Intelligence Recon System	2
Figure 3 Workflow of reNgin	7
Figure 4 System Architecture of Pentest Tools	8
Figure 5 System Architecture of SOCRadar.....	9
Figure 6 Scrum Methodology Process (Donetonic, 2025).	12
Figure 7 Previous WBS	14
Figure 8 Updated WBS	14
Figure 9 Project Planning	15
Figure 10 Requirement Analysis	15
Figure 11 Sprint 1.....	15
Figure 12 Sprint 2.....	15
Figure 13 Sprint 3.....	16
Figure 14 Sprint 4.....	16
Figure 15 Sprint 5.....	16
Figure 16 Sprint 6.....	16
Figure 17 Control.....	16
Figure 18 Closure!.....	16
Figure 19 Updated Gantt Chart	17
Figure 20 Updated Gantt Chart	17
Figure 21 Updated Gantt Chart	18
Figure 22 Updated Gantt Chart in Detail	19
Figure 23 Updated Gantt Chart in detail.....	20
Figure 24 Milestone	21
Figure 25 Progress Table	23
Figure 26 Progress Table	24
Figure 27 Phases to Complete	26
Figure 28 Waterfall Methodology (Welch, 2025).	31
Figure 29 V-Model Methodology (Simran, 2025).....	32
Figure 30 Agile Methodology (Kamis, 2019).	34
Figure 31 Previous WBS	38
Figure 32 Updated WBS	39

Figure 33 Milestone.....	40
Figure 34 Pre-Survey Finding 1	44
Figure 35 Pre-Survey Finding 2	45
Figure 36 Pre-Survey Finding 3	45
Figure 37 Pre-Survey Finding 4	46
Figure 38 Pre-Survey Finding 5	46
Figure 39 Pre-Survey Finding 6	47
Figure 40 Pre-Survey Finding 7	47
Figure 41 Pre-Survey Finding 8	48
Figure 42 Pre-Survey Finding 9	48
Figure 43 Pre-Survey Finding 10	49
Figure 44 Pre-Survey Finding 11	49
Figure 45 Pre-Survey Finding 12	50
Figure 46 Pre-Survey Finding 13	50
Figure 47 Pre-Survey Finding 14	51
Figure 48 Flowchart Diagram	59
Figure 49 System Architecture Diagram.....	60
Figure 50 Sequence Diagram of Intelligence Recon System	61
Figure 51 Use Case Diagram in a scenario of Security Analyst	63
Figure 52 Use Case Diagram in a scenario of Security Administrator	64
Figure 53 Activating the project environment and checking the Python version.....	65
Figure 54 IRS Python Dependencies	65
Figure 55 Installing Python dependencies.....	66
Figure 56 Starting the IRS FAST API with Uvicorn	66
Figure 57 Fast API Swagger page showing the root endpoint	67
Figure 58 Installing Nabbu tool with its go modules	67
Figure 59 Verify Successful Installation.....	68
Figure 60 Installation of nuclei scanner and go	68
Figure 61 Verified installation of nuclei.....	69
Figure 62 Install httpx and verified	69
Figure 63 Install Subfinder	70
Figure 64 Verified installation of subfinder	70
Figure 65 Install Assetfinder with go and fetching dependencies.....	71
Figure 66 Verified installation of assetfinder.....	71

Figure 67 Install Amass.....	71
Figure 68 Cloning Sublist3r, installing requirements and verified installation	72
Figure 69 Install CTL	72
Figure 70 Verified installation of ctl	73
Figure 71 Install GAU	73
Figure 72 Verified installation of GAU	74
Figure 73 Install Katana	74
Figure 74 Verified installation of katana	75
Figure 75 Install Gospider	75
Figure 76 Verified installation of gospider	76
Figure 77 Install and Verified ffuf tool	76
Figure 78 Install and verified cewl	77
Figure 79 Install and Verified dnsx	77
Figure 80 Install and Verified Shuffledns.....	78
Figure 81 Asynchronous subprocess execution in the backend.....	78
Figure 82 IRS environment configuration (.env) for Gemini and tool settings	79
Figure 83 Fast API and CORS setup	80
Figure 84 Scan creation endpoint with tool validation in the IRS Fast Api backend .	80
Figure 85 Scan delete and retry endpoint for managing scans in IRS API.....	81
Figure 86 Scan status, detail, and list endpoints querying IRS scan results	81
Figure 87 Scan status enum and scan model	82
Figure 88 Tool run status enum and Tool Run model	82
Figure 89 Finding enums and Finding model	83
Figure 90 Scan Service create_scan.....	83
Figure 91 Gather initial clues with Naabu and Httpx	84
Figure 92 Execute recon tools and save findings.....	85
Figure 93 AI decision class and Gemini API call	86
Figure 94 Execute a single tool with timeout and output file.....	86

Table of Tables

Table 1 Comparison Projects Table	10
Table 2 Project Milestone	22
Table 3 Product Backlog	58
Table 4 Table of Software Requirements	89

Table of Abbreviations

Abbreviations	Full Form
IRS	Intelligence Recon System
CLI	Command Line Interface
SRS	Software Requirements Specification
OWASP	Open Web Application Security Project

1. INTRODUCTION

Cyber security reconnaissance is an essential step in the cyber security sector, where systematically collecting information about target systems, networks and organizations takes place (Cymulate, 2025). Reconnaissance helps threat actors to develop detailed profiles of targets to strategize effective attack methods, and defenders to use such techniques on the offensive to determine and strengthen security positions (Arora, 2025).

1.1. Introduction to Topic

Web applications and online services have become part of modern organizations, increasing their attack surface exponentially, and revealing repeated vulnerabilities to misconfigurations, missing access controls, and injections, so automated reconnaissance based on OWASP delineations is required in addition to manual checks (Sumanth, 2023).

Standard web applications commonly have various vulnerabilities simultaneously in access control, information disclosure, authentication and input validation, so intersecting OWASP style vulnerabilities are difficult to comprehend with manual testing ad hoc.

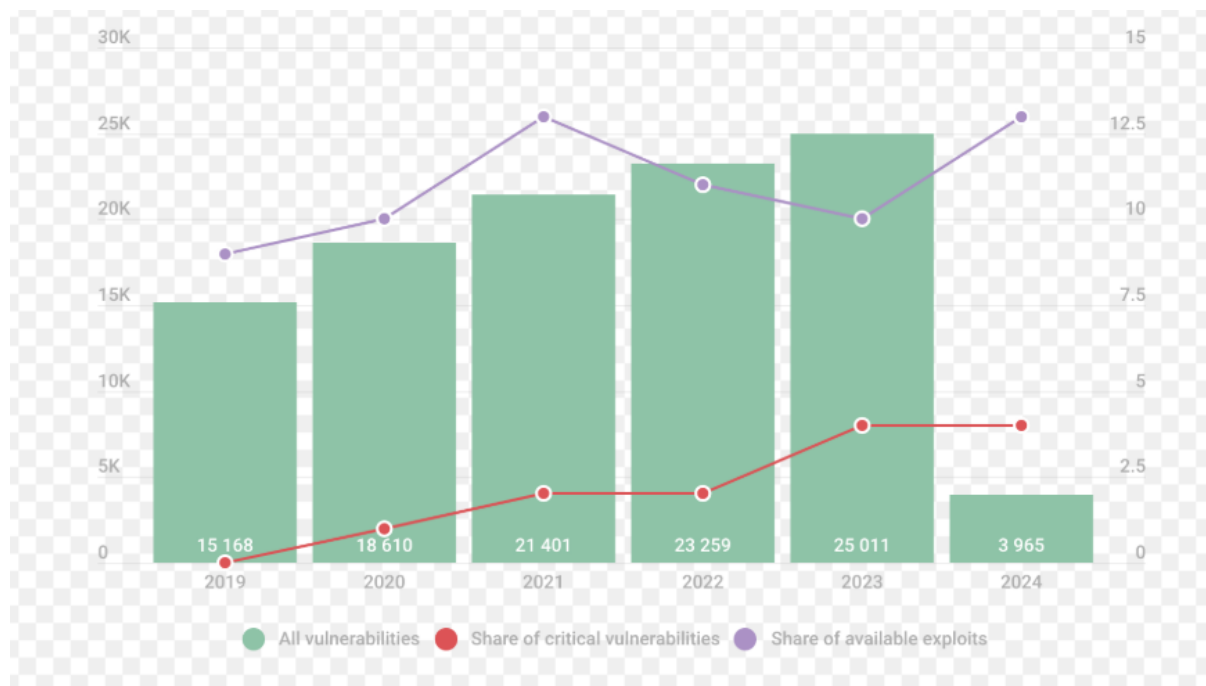


Figure 1 Vulnerability Landscape (Alexander Kolesnikov, 2024).

IRS is an artificial intelligence-powered web application that brings together various CLI tools and only carries out context-specific scans based on early warning factors and OWASP typologies, providing students, teams, and analysts with an efficient workflow and repeatability.

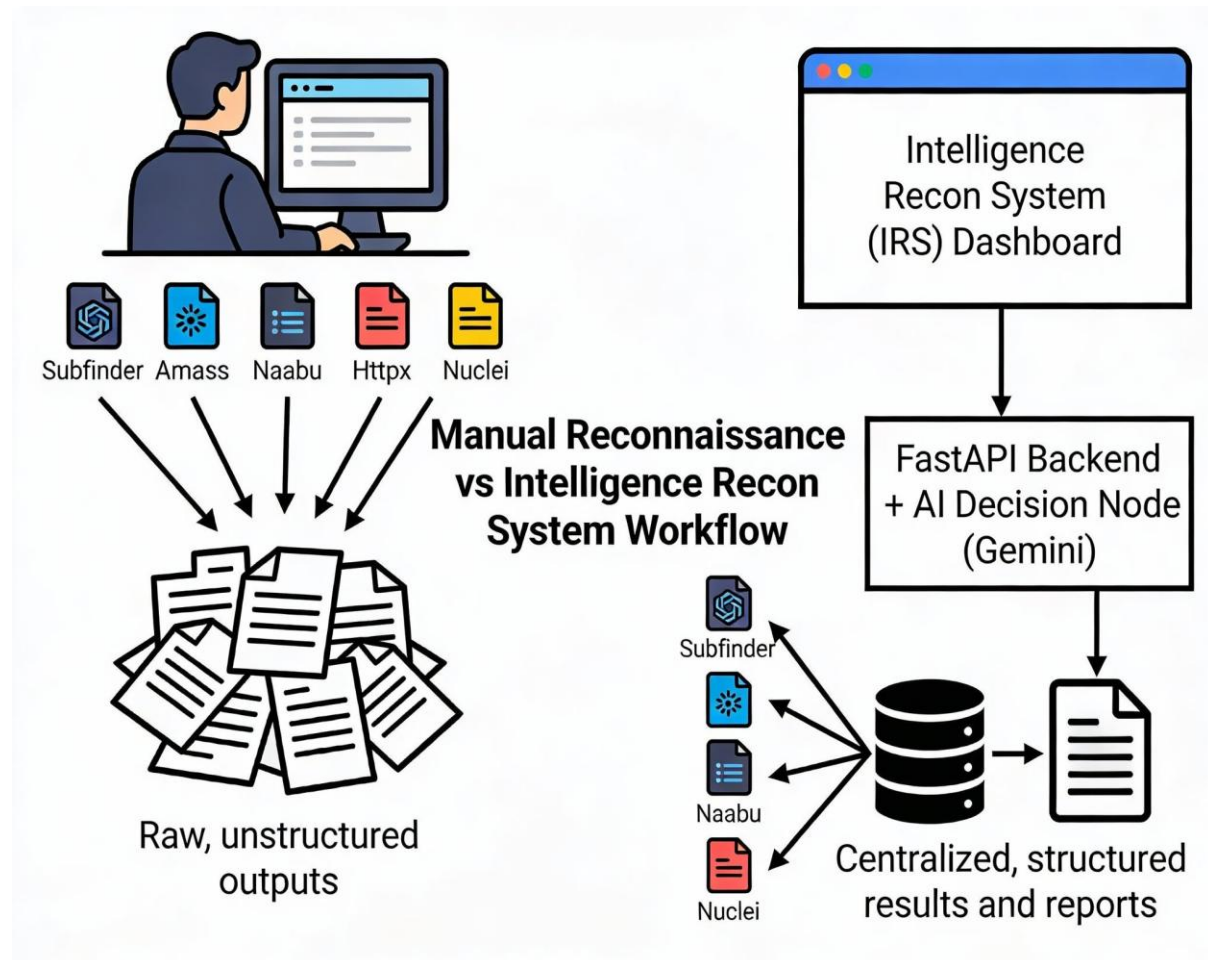


Figure 2 Manual Reconnaissance vs Intelligence Recon System

1.2. Problem Statement

- **Manual and Time Consuming:** Manual recon is time-consuming, failing to keep up with changing cyber threats and demand of timely security opinions. (Sumanth, 2023).
- **Human Error and Bias:** Even professional ethical hackers tend to make errors, such as failing to interpret information or not detecting a vulnerability. False alarms or missing threats can be caused by unconscious bias.
- **Manual and Fragmented Execution:** Recon involves chaining numerous manual CLI tools. This fragmentation wastes time in complicated syntax and leads to inefficiency and inconsistency in the outcomes (Ariganello, 2022).
- **Absence of Centralized Management and Professional Reporting:** Scan results are distributed locally thus making it difficult to retrieve the data and track its history. Report generation is slow and manual, and prone to errors.

1.3. Project as Solution

- **Removes Manual Time Waste:** The system automates the whole working system allowing the security analysis to take place in a fast and timely manner, even in extensive settings.
- **Reduces Minimizes Error and Bias:** The inclusion of AI Decision Node makes the tool selection objective and optimized, which reduces human misinterpretation or bias when making triage decisions.
- **Automated and Parallel execution:** This system incorporates the specialty CLI tools into one scalable framework, permitting automated and parallel chaining to conquer the disjointed and manual execution.
- **Centralized Management and Reporting:** Data is gathered, normalized, and stored continuously, to give consistent and structured outputs to real-time analysis and the creation of professional reports to the stakeholders with a single click.

1.4. Aim and Objectives

1.4.1. Aim

The Aim of the IRS is to create, design, and thoroughly test an integrated, unified, and smartly coordinated security platform that automates the multi-step, multi-stage reconnaissance process of professional CLI programs, and thus replaces manual and fragmented vulnerability testing with automated, centralized, and report-enabled security testing based on the OWASP Top 10 threats.

1.4.2. Objectives

To achieve the project goal, the following objectives that need to be achieved:

- Created a strong, 3-tier (Frontend, Fast Api Backend, and Tool Execution) web and Python-based platform.
- Develop and implement one easy-to-use web dashboard to minimize inputs, provide real time response, and enable OWASP-oriented target and tool selection.
- Have asynchronous execution with dynamic chaining of different CLI tools with a reliable data flow and a responsive non-blocking user interface.
- Add an AI Decision Node that is used to optimize the tool execution plan contextually by smartly utilizing the possibility of excluding irrelevant or redundant scanning checks.
- Create an effective data parsing component to process all raw and inconsistent tool output and provide this in a centralized, consistent, and structured format.
- Apply persistent storage to provide all historical scan results enabling users to monitor the history of asset discovery and vulnerability.
- Offer a real-time web dashboard with scans and the capacity to create and download professional, standardized PDF/HTML reports.
- Study and contrast the performance of the Intelligence Recon System to conventional manual CLI reconnaissance processes regarding scan time, checks redundancy, and coverage of OWASP consistent findings.
- Prepare and assess the system as a structured technical reporting with background research, reviewing of similar projects, architectural design, development progress and testing results to show the feasibility and learning effectiveness of the IRS platform.

1.5. Structure of the Report

1.5.1. Background

This subsection introduces the Intelligence Recon System and explains why traditional, manual CLI-based reconnaissance is inefficient and fragmented. It motivates integrating multiple tools into a single OWASP-aligned platform. The target users are students, small security teams, and bugbounty hunters are briefly characterized..

1.5.2. Development

This subsection explains the three-tiered architecture system frontend, Fast API backend and tool execution layer. It describes the way Python subprocesses and asynchronous calls coordinate approximately 15 built-in CLI tools. The features of the AI Decision Node workflow, unified dashboard, organized outputs, and report generation are also described.

1.5.3. Analysis of Progress

In this subsection, the degree to which the initial objectives and milestones have been reached is assessed so far. It puts emphasis on accomplished architecture configuration, major tool integration, and fundamental dashboard capabilities. It also records delay of schedules, issues, architectural modification and states in passing about existing stability and partially completed features..

1.5.4. Further Work

In this sub section, I have summarized the remaining work required to achieve final deliverables. It involves the schemes of enhancing AI-based execution validation, introducing or adjusting tools, and optimizing the user interface. Further testing, performance improvement, documentation, and potential extensions of it to multi-user functionality and wider threat coverage are listed as priorities.

2. BACKGROUND

2.1. Target Client

The primary target clients for IRS are designed for three main types of users:

- The students and learners of cybersecurity who need to train on the actual reconnaissance and OWASP-based testing but have access to the help of a simple web dashboard and AI rather than complicated terminal routines.
- Small security teams in small companies, colleges or labs that do not have big commercial platforms though they require automated and consistent recon with centralized results and easy reports.
- Single security analysts/bug-bounty hunters that need to accelerate multi-tool recon, lessen manual and human error, and store history of their scans in a single location.

2.2. Understanding the Project

2.2.1. Project Description

The IRS is a web-based reconnaissance cybersecurity tool that substitutes manual command-line operations with one AI-supported dashboard. It combines 15 open-source tools including Nuclei, Naabu, Httpx, Subfinder, Amass, FFuf, GAU, DNSx, Katana, Gospider, CeWL, ShuffleDNS, Assetfinder, Sublist3r, and CTL that automate subdomain discovery, port scanning, HTTP probing, endpoint enumeration and OWASP Top 10 centric vulnerability checks.

IRS is a three-tier application having an HTML/CSS/JavaScript frontend, Fast API backend and an execution layer that invokes all CLI tools asynchronously using Python subprocess. Users specify a target domain and make an OWASP Top 10 focus and then an AI Decision Node driven by Gemini narrows down the scanned tools to only context-relevant scans to minimize redundancy and human error.

The results of scans are normalized and stored and displayed in a single dashboard with running progress and downloadable HTML/PDF reports. Using automated multi-tool reconstruction, AI-based tool selection, and central reporting, IRS allows small security teams and students to conduct an effective and professionally-quality reconnaissance, rather than lumbering through CLI-based reconnaissance.

2.2.2. Project Deliverables

- **IRS:** The complete system can execute hyper-focused, AI-optimized security assessments via one web screen.
- **Unified Interface and Monitoring Dashboard:** This allows analysts to manage all security scans, track real-time progress, and review aggregated findings from a single central hub.
- **Structured Output and Persistent Results:** It transforms raw data so that the user can easily analyze the standardized results and securely store all the historical data for long-term review.
- **Security Tool Execution Layer:** This layer runs the 15 in-built tools, enabling the system to execute a complete and powerful suite of security checks instructed by the AI.
- **Downloadable Standardized Report:** Users can generate professional, ready-to-share HTML reports for audits, documentation, or incident review.

2.3. Similar Project Review

2.3.1. Project 1: reNgin

Author: Yogesh Ojha

URL: <https://github.com/yogeshojha/rengine>

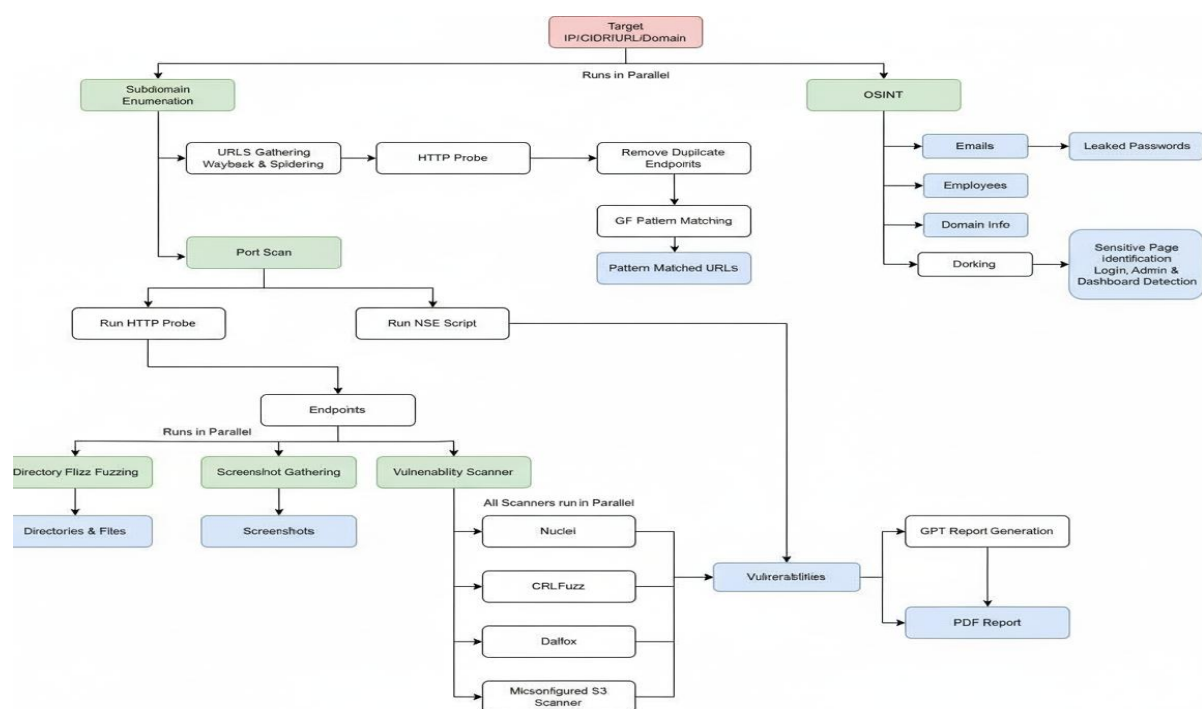


Figure 3 Workflow of reNgin

Description: reNgin is an open-source web-based framework that will automate reconnaissance of bug hunters and penetration testers attending bug bounties. It allows them to specify recon profiles and targets via a web UI and executes a large set of popular tools in the background. The platform will carry out subdomain discovery, DNS resolution, port scanning, HTTP probing, screen shots and vulnerability scanning within one workflow. The data obtained is all mined, standardized and stored in the central database that is easily analyzed. Each target has dashboards displaying subdomains, open ports, technologies, and screenshots and identified vulnerabilities. This assists security professionals in saving on time and ensuring uniform recons in numerous engagements (Ojha, 2025).

2.3.2. Project 2: Pentest-Tools

Author: Pentest-Tools Team

URL: <https://pentest-tools.com/>

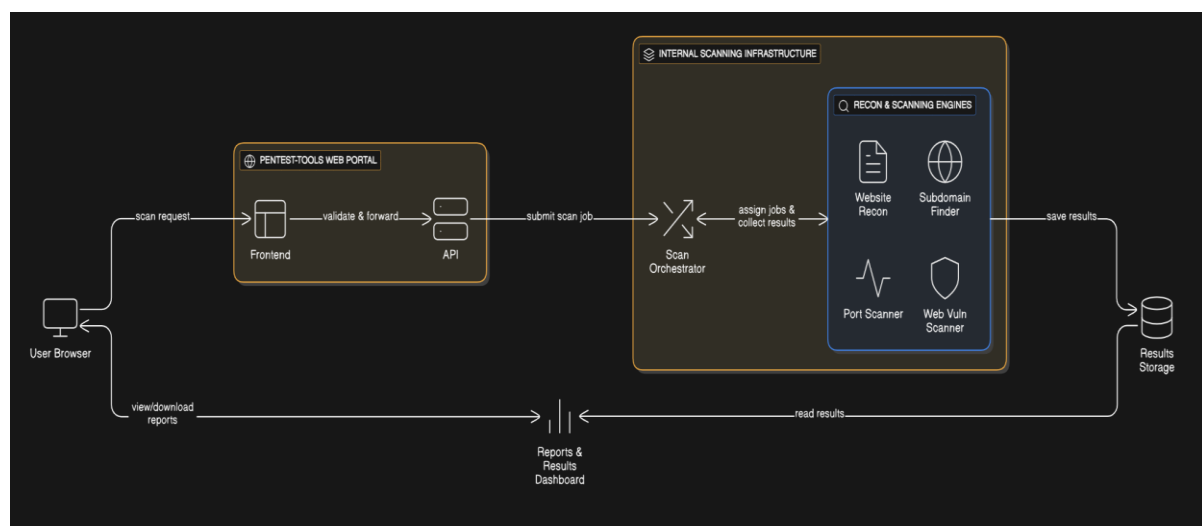


Figure 4 System Architecture of Pentest Tools

Description: Pentest -Tools.com is a commercial web-based service that provides a set of reconnaissance and vulnerability-scanning tools that are available via browser. Its modules are web reconnaissance, subdomain finder, port scanner, technology fingerprinting as well as web vulnerability scanners. One just needs to enter a target URL or domain and choose one or a few tools; the scans are performed on their own infrastructure, and the results are displayed in a single report. The tool of recon of the site, e.g., determines technologies, HTTP headers, files of interest, and

misconfigurations. This will reflect your IRS idea of having many checks running on one web dashboard rather than per tool of the CLI (Team, 2025).

2.3.3. Project 3: SOCRadar

Author: SOCRadar Cyber Intelligence Inc.

URL: <https://socradar.io/labs>

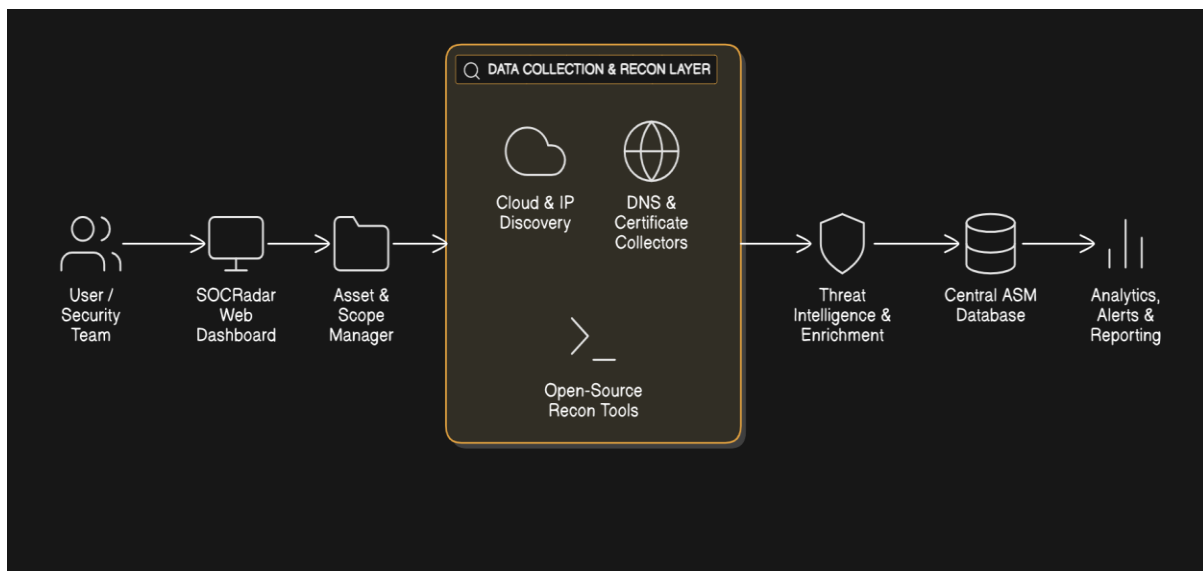


Figure 5 System Architecture of SOCRadar

Description: SOCRadar is a business tool, which integrates External Attack Surface Management (EASM), digital risk protection, and threat intelligence. It is constantly identifying and scanning internet-facing resources of the organization's domains, IPs, cloud computing, exposed services and matching them with threat-intelligence feeds. The system applies automated reconnaissance, open-source tools as well as its own data sources and identifies newly exposed services, misconfigurations, and vulnerabilities. There is a central dashboard with assets, findings, risk scores, and alerts to assist security teams in prioritizing remediation (SOCRadar, 2025).

2.4. Comparison Table

Features	P1(3.1.1.)	P2(3.1.2.)	P3(3.1.3.)	IRS
Web-based dashboard	Yes	Yes	Yes	Yes
Automated multi-tool scanning	Yes	Partly (per tool)	Yes	Yes
Recon focused on web targets and domains	Yes	Yes	No	Yes
Visibility of all external organization assets	No	No	Yes	No
AI-generated summary and findings report	No	No	No	Yes
Ability to keep and review past scan results	Yes	Yes	Yes	Yes
Assessment guided by OWASP Top-10 risks	No	No	No	Yes
Requires installation on your own server	Yes	No	No	Yes
Easy to learn and use	Medium (setup is harder)	Yes	Yes	Yes
Continuous / repeat scans	Yes	Yes	Yes	Yes

Table 1 Comparison Projects Table

2.5. Analysis and Conclusion of Comparison

Intelligence Recon System (IRS) features are also affected by the three reference tools. Whereas reNginer is about the ability to highly configure reconnaissance pipelines to technical users, IRS is about making automated multi-tool recon simpler by a guided web interface and a tool selection system with AI to help keep small teams undertaking professional-grade reconnaissance with less manual effort. Similarly to the case when Pentest-tools.com has a cloud dashboard to run a scan of a web-site and view clear vulnerability-reports, IRS takes the concept of a single web-based dashboard with multi-target scans, live progress, and standardized HTML/PDF report, which is implemented using open-source elements that may be used in a lab and student setup.

The principle of SOCRadar to centralize the discovered assets and exposures over time also influences IRS, which maintains the results of recon and assets in persistent storage to allow users to revisit targets and review the past results, albeit at a smaller scale than the enterprise attack-surface-management systems. Unlike these tools, IRS has an AI decision node to minimize human bias when selecting a tool and a parsing layer which transforms various command-line outputs into a single standard format to enhance analysis and reporting.

These systems have several features common to IRS but also encompass several sophisticated features like internet-wide asset discovery, rich external threat-intelligence feeds, and highly automated remediation workflows not yet available in IRS but potentially available in future editions. Therefore, IRS is a hybrid between reNginer, Pentest-Tools.com and SOCRadar powered by AI-driven organization, formatted outputs, and a minimal web structure, thus being a convenient reconnaissance tool to use among students and small security teams.

3. DEVELOPMENT

3.1. Considered Methodology

In IRS, the risk mitigation required the proper choice of SDLC which has led to the option of both Waterfall and Agile approaches.

3.1.1. [Waterfall methodology](#)

3.1.2. [V-Model Methodology](#)

3.2. Selected Methodology

3.2.1. Agile Framework

Agile project management is chosen to be used in the Intelligence Recon System as it allows iterative development, constant feedback and allowing flexibility in case of change in requirements or technical details. [Learn more](#)

3.2.1.1. Scrum Methodology

Scrum Methodology is officially chosen to be the project management tool used in this IRS project. This option has been informed by the fact that it requires a very flexible strategy in dealing with challenging regulatory requirements and a fast-changing specification. [Learn more](#)

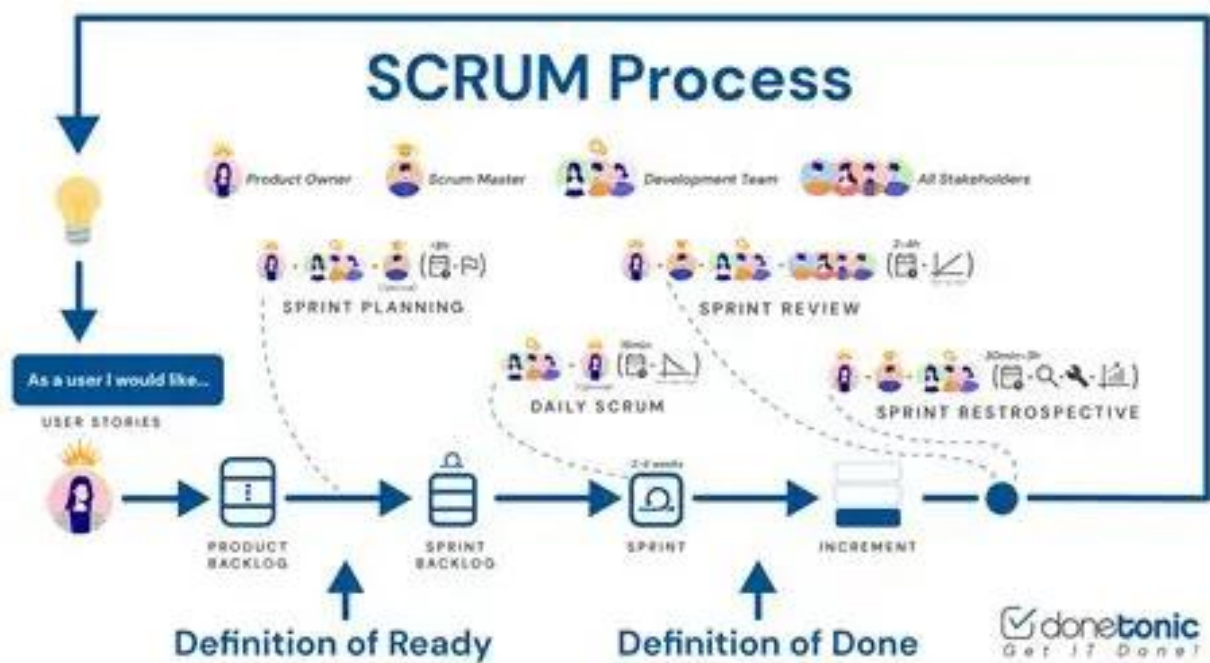


Figure 6 Scrum Methodology Process (Donetonic, 2025).

In the case of IRS, Scrum divides the development process into the following phases:

- **Initiation phase:** Establish project objectives, scope, key features, and resources needed (backend, tools, AI, and storage).
- **Planning and estimate phase:** Divide the project into manageable sprints, develop a product backlog, and create a story point assignment on tasks to determine effort and priority.
- **Implementation phase:** Within every sprint, finish on tasks that have been selected and provide a working bit of IRS. Progress of the project is ongoing.
- **Review and retrospective phase:** During a review with the supervisor, after every sprint, assess the progress made, collect feedback, and discuss problems to make improvements in future sprints.
- **Release:** Each set of sprints should be followed by a stable increment (e.g., a complete recon workflow with report) and ensure that it is to expectations.

Reason to choose Agile Scrum as methodology for the project

The final decision was to use Agile Scrum since it is based on three fundamental pillars which are highly applicable in the project:

- Transparency implies that the important work and information is visible and clear to everyone within the team and stakeholders. Backlog, sprint goals, progress and Definition of Done are openly shared to eliminate concealed tasks and surprises.
- Inspection is the frequent examination of the product and the process to identify issues at an early stage. Using such events as daily stand ups, sprint reviews and backlog refinement, the team can review progress and quality thus identifying problems before they escalate.
- Adaptation is a modification of plans or methods of doing things depending on what is being inspected. Reviews and retrospective Most of the team will adjust priorities, processes, or designs so that the project will be on track and constantly improving, iteration by iteration.

3.3. Work breakdown structure.

The Work Breakdown Structure illustrated below separates the project into distinct phases and tasks, facilitating the organization, scheduling, and management of all work required for successful completion (Manager, 2025).

3.3.1. Previous WBS



Figure 7 Previous WBS

For More Clear Vision [click here](#)

3.3.2. Updated WBS

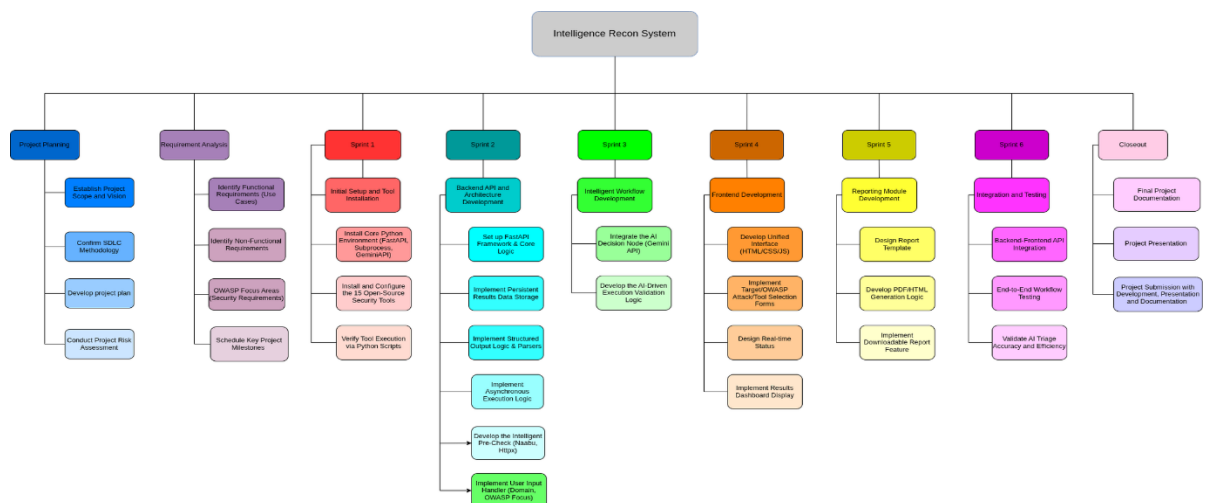


Figure 8 Updated WBS

For More Clear Vision [click here](#)

The deviations of the updated WBS are primarily because Sprint 2 was accomplished before schedule, which meant that some of the Sprint 3 activities were executed earlier. This has caused some of the activities that were initially projected to occur later in the structure to be brought forward.

3.4. Gantt chart

A Gantt chart is a horizontal bar chart that is utilized to represent a project on a timeline. Every task is presented as a bar where the beginning of work is displayed, the duration of the work is indicated, the due date is shown, and the percentage of fulfillment is indicated (LaPrad, 2025). The following Gantt Chart shows the activities of IRS project with their respective timeline.

3.4.1. Previous Gantt Chart

1. Project Planning

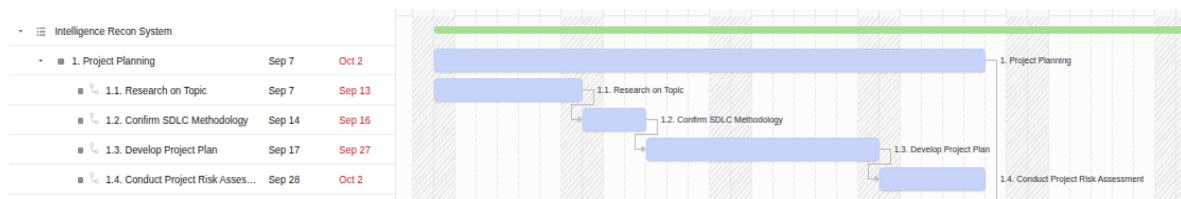


Figure 9 Project Planning

2. Requirement Analysis

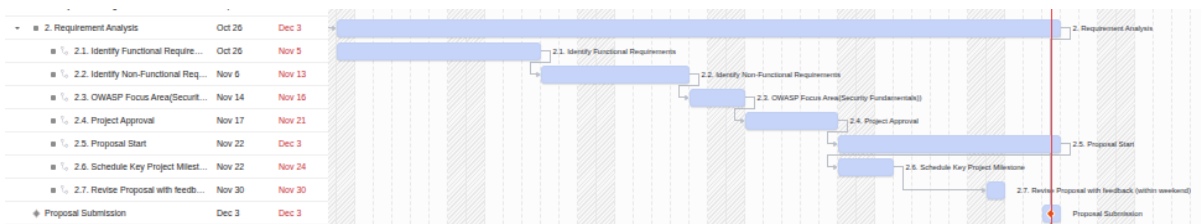


Figure 10 Requirement Analysis

3. Sprint 1

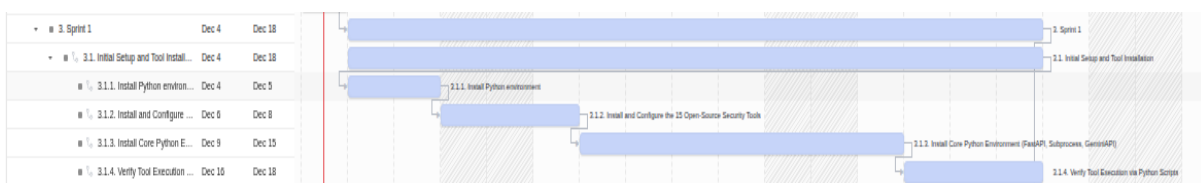


Figure 11 Sprint 1

4. Sprint 2

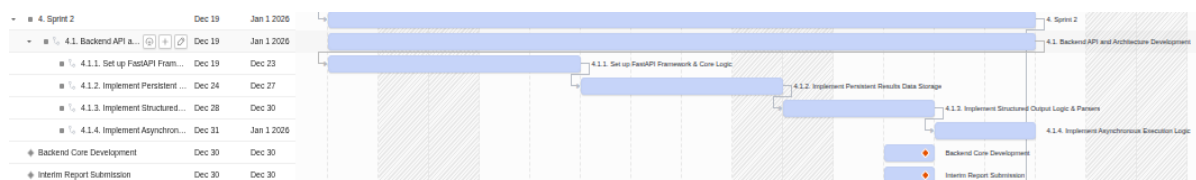


Figure 12 Sprint 2

5. Sprint 3

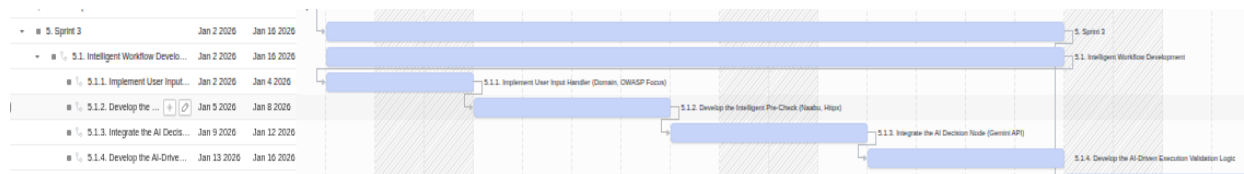


Figure 13 Sprint 3

6. Sprint 4

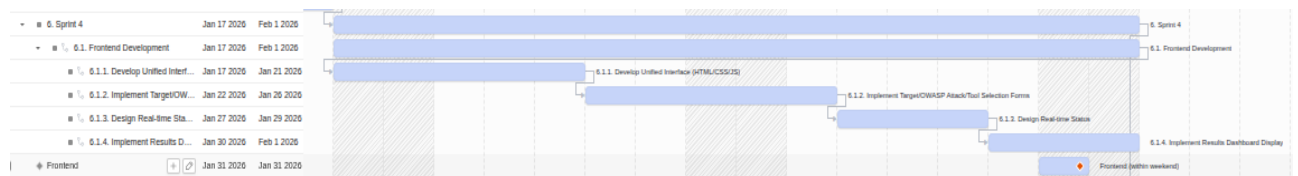


Figure 14 Sprint 4

7. Sprint 5

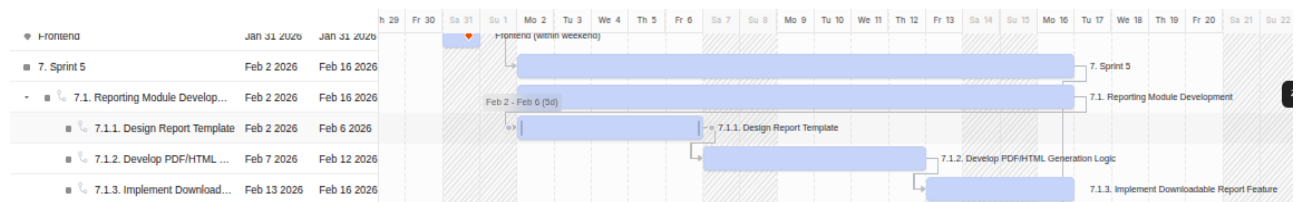


Figure 15 Sprint 5

8. Sprint 6

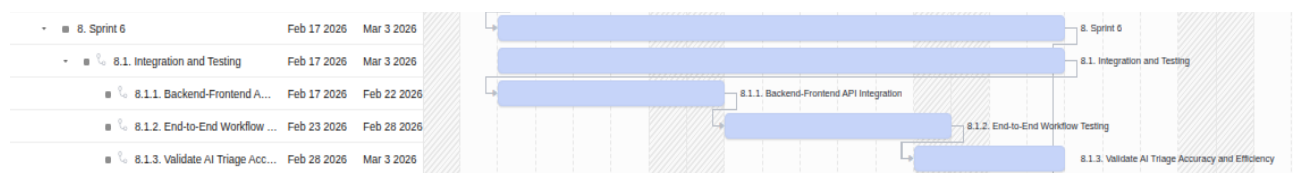


Figure 16 Sprint 6

9. Control

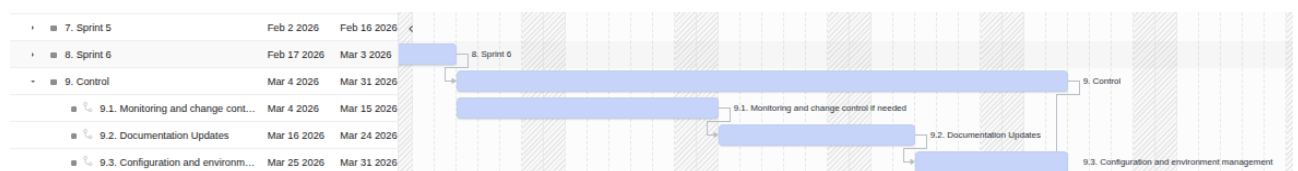


Figure 17 Control.

10. Closure

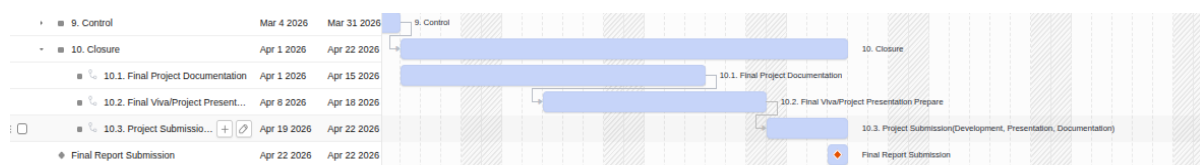


Figure 18 Closure!

3.4.2. Updated Gantt Chart

There is a deviation in the updated Gantt because Sprint 2 was done earlier than expected. This led to the early initiation of some Sprint 3 activities. These changes show speed up progress and ensure that the remaining sprints and final deliveries are well organized in the new schedule.

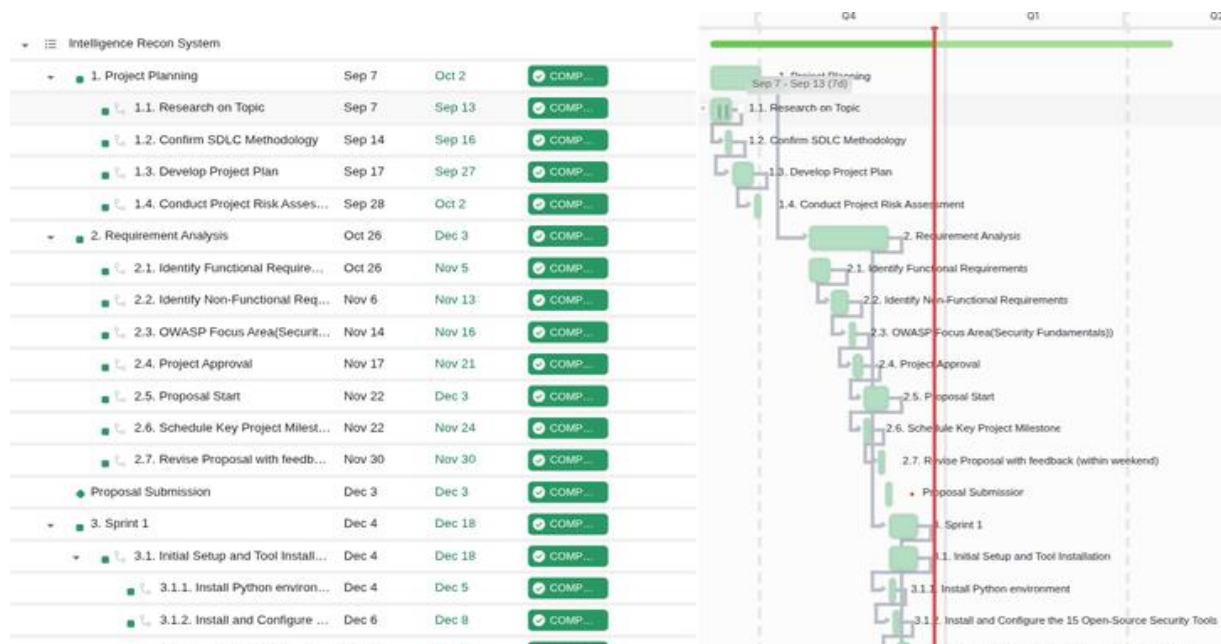


Figure 19 Updated Gantt Chart

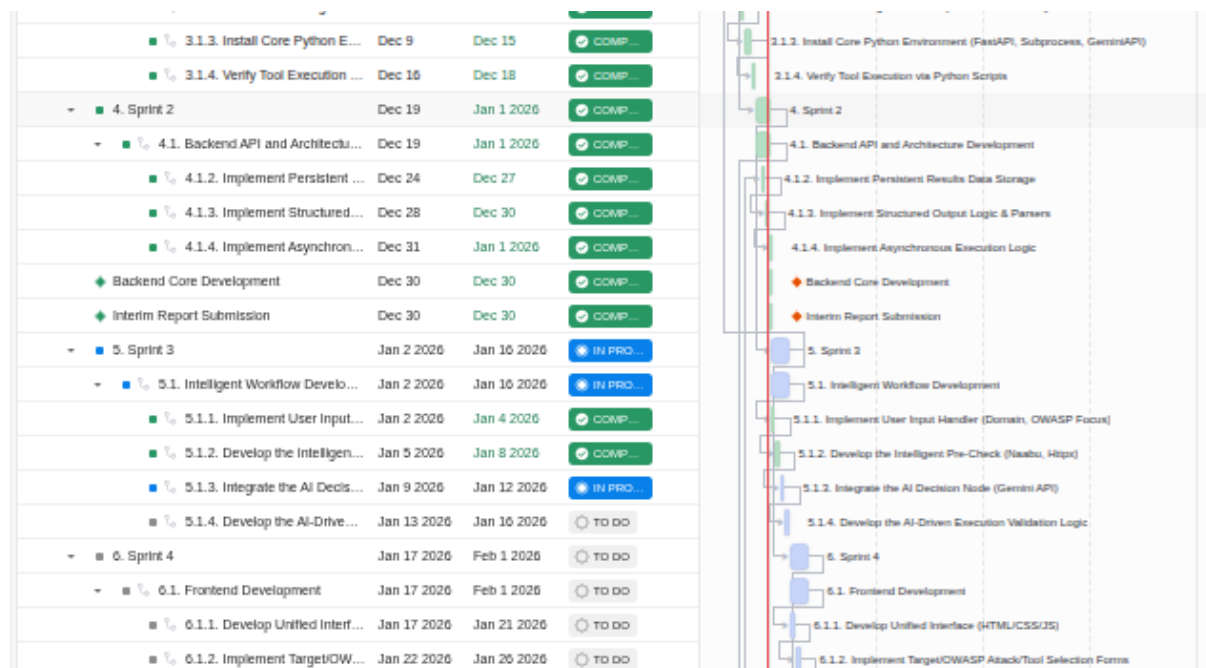


Figure 20 Updated Gantt Chart

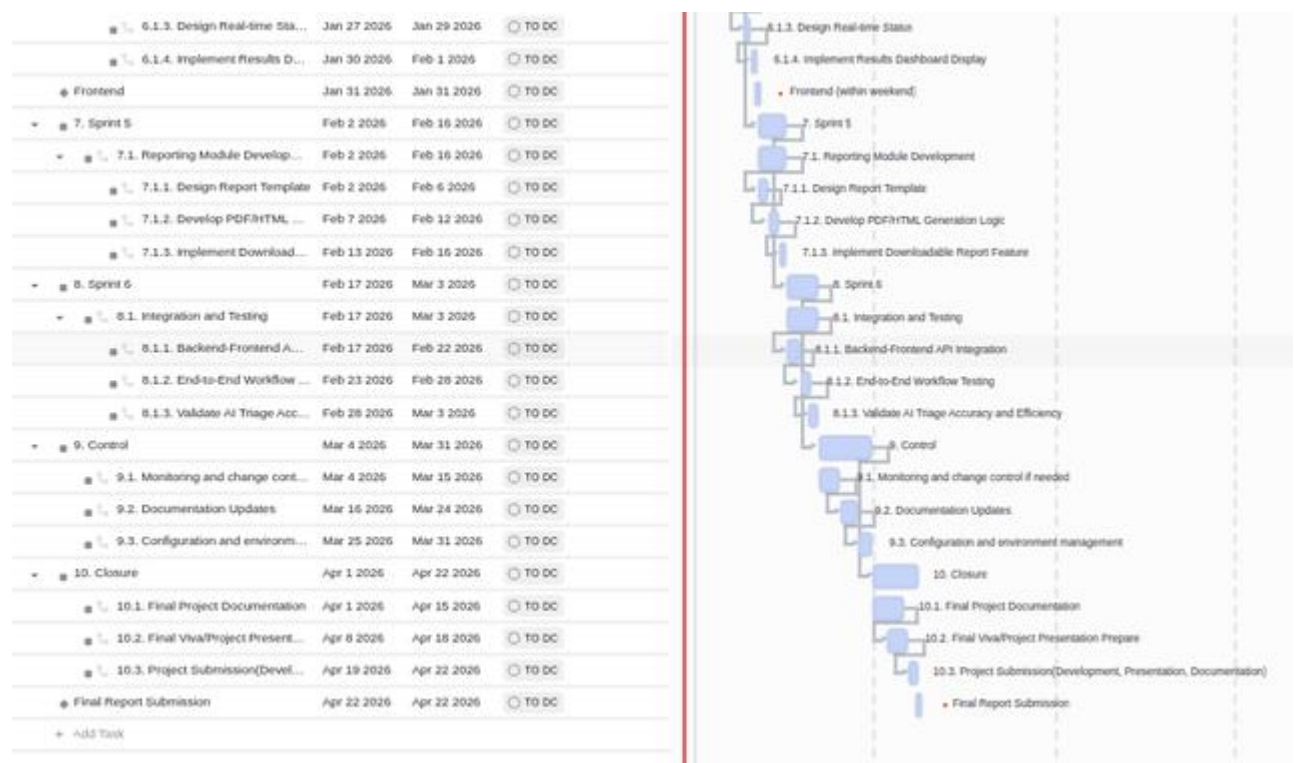


Figure 21 Updated Gantt Chart

Tasks	Start Date	End Date	Status
Project Planning	9/7/2025	10/2/2025	Completed
Research on Topic	9/7/2025	9/13/2025	Completed
Confirm SDLC Methodology	9/14/2025	9/26/2025	Completed
Develop Project Plan	9/17/2025	9/27/2025	Completed
Conduct Project Risk Assessment	9/28/2025	10/2/2025	Completed
Requirement Analysis	10/26/2025	12/3/2025	Completed
Identify Functional Requirements	10/26/2025	11/5/2025	Completed
Identify Non-Functional Requirements	11/6/2025	11/13/2025	Completed
OWASP Focus Area (Security Fundamentals)	11/14/2025	11/16/2025	Completed
Project Approval	11/17/2025	11/21/2025	Completed
Proposal Start	11/22/2025	12/3/2025	Completed
Schedule Key Project Milestone	11/22/2025	11/24/2025	Completed
Revise Proposal with feedback	11/30/2025	11/30/2025	Completed
Proposal Submission	12/3/2025	12/3/2025	Completed
Sprint 1	12/4/2025	12/18/2025	Completed
Initial Setup and Tool Installation	12/4/2025	12/4/2025	Completed
Install Python environment	12/4/2025	12/5/2025	Completed
Install and configure the 15 Open-Source Security Tools	12/6/2025	12/8/2025	Completed
Install Core Python Environment (Fast API, Subprocess, Gemini API)	12/9/2025	12/15/2025	Completed
Verify Tool Execution via Python Scripts	12/16/2025	12/18/2025	Completed
Sprint 2	12/19/2025	1/1/2026	Completed
Backend API and Architecture Development	12/19/2025	1/1/2026	Completed
Set up Fast API Framework & Core Logic	12/19/2025	12/23/2025	Completed
Implement Persistent Results Data Storage	12/24/2025	12/27/2025	Completed
Implement Structured Output Logic & Parsers	12/28/2025	12/30/2025	Completed
Implement Asynchronous Execution Logic	12/31/2025	1/1/2026	Completed
Backend Core Development	12/30/2025	12/30/2025	Completed
Interim Report Submission	12/30/2025	12/30/2025	Completed
Sprint 3	1/2/2026	1/16/2026	In progress
Intelligent Workflow Development	1/2/2026	1/16/2026	In progress
Implement User Input Handler (Domain, OWASP Focus)	1/2/2026	1/4/2026	Completed
Develop the Intelligent Pre-Check (Naabu, Httpx)	1/5/2026	1/8/2026	Completed
Integrate the AI Decision Node (Gemini API)	1/9/2026	1/12/2026	In progress
Develop the AI-Driven Execution Validation Logic	1/13/2026	1/16/2026	To Do
Sprint 4	1/17/2026	2/1/2026	To Do
Frontend Development	1/17/2026	2/1/2026	To Do
Develop Unified Interface (HTML/CSS/JS)	1/17/2026	1/21/2026	To Do
Implement Target/OWASP Attack/Tool Selection Forms	1/22/2026	1/26/2026	To Do
Design Real-time Status	1/27/2026	1/29/2026	To Do
Implement Results Dashboard Display	1/30/2026	2/1/2026	To Do
Frontend	1/31/2026	1/31/2026	To Do

Figure 22 Updated Gantt Chart in Detail

Sprint 5	2/2/2026	2/16/2026	To Do
Reporting Module Development	2/2/2026	2/16/2026	To Do
Design Report Template	2/2/2026	2/6/2026	To Do
Develop PDF/HTML Generation Logic	2/7/2026	2/12/2026	To Do
Implement Downloadable Report Feature	2/13/2026	2/16/2026	To Do
Sprint 6	2/17/2026	3/3/2026	To Do
Integration and Testing	2/17/2026	3/3/2026	To Do
Backend-Frontend API Integration	2/17/2026	2/22/2026	To Do
End-to-End Workflow Testing	2/23/2026	2/28/2026	To Do
Validate AI Triage Accuracy and Efficiency	2/28/2026	3/3/2026	To Do
Control	3/4/2026	3/31/2026	To Do
Monitoring and change control if needed	3/4/2026	3/15/2026	To Do
Documentation Updates	3/16/2026	3/24/2026	To Do
Configuration and environment management	3/25/2026	3/31/2026	To Do
Closure	4/1/2026	4/22/2026	To Do
Final Project Documentation	4/1/2026	4/15/2026	To Do
Final Viva/Project Presentation Prepare	4/8/2026	4/18/2026	To Do
Project Submission (Development, Presentation, Documentation)	4/19/2026	4/22/2026	To Do
Final Report Submission	4/22/2026	4/22/2026	To Do

Figure 23 Updated Gantt Chart in detail

3.5. Milestone

The project milestone is a reference or benchmark to showcase a significant goal, event or a task in a project lifecycle (Staff, 2025). The following figure is the milestone of IRS project which gives clear directions of the project in the long run.

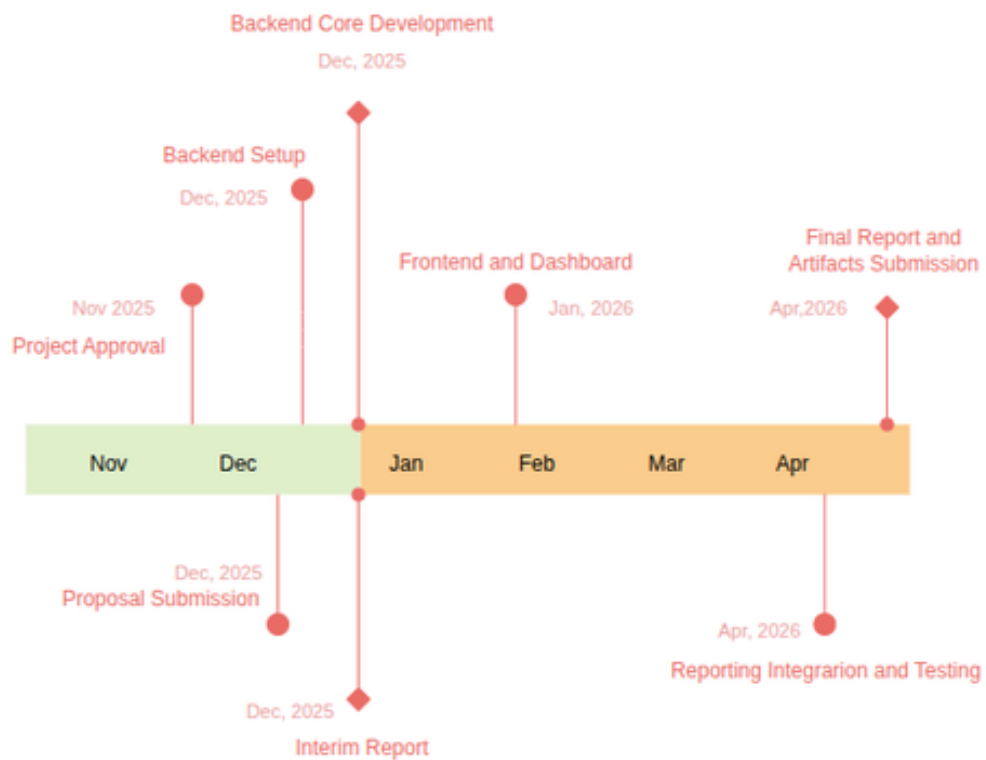


Figure 24 Milestone

For clear vision [click here](#)

3.5.1. Progress of Milestones

IRS project is moving on well. The first milestones like Project Approval, Proposal Submission and Backend Setup are completed and it indicates the progress of the project. Moreover, there is Interim Report Submission and Backend Core Development which is in progress. Frontend and Dashboard development, Reporting Integration and Testing, Final Report and Artifacts Submission are the other activities that remain incomplete but will be delivered on time. As soon as they are completed it will result in the development phase being complete and then extensive testing phase. Once all the stages have been done, documentation process will be embarked upon to achieve a successful project implementation and a comprehensive knowledge of each stage.

Completion Date	Milestone	Status
November 21, 2025	Project Approval	Completed
December-3, 2025	Proposal Submission	Completed
December 15, 2025	Backend Setup	Completed
December 30, 2025	Interim Report Submission	Completed
December 30, 2025	Backend Core Development	In progress
January-31, 2026	Frontend and Dashboard	To Do
April 8, 2026	Reporting Integration and Testing	To Do
April 22, 2026	Final Report and Artifacts Submission	To Do

Table 2 Project Milestone

For more information [click here](#)

4. ANALYSIS OF PROGRESS

4.1. Progress Table

Tasks	Start Date	End Date	Status
Project Planning	9/7/2025	10/2/2025	Completed
Research on Topic	9/7/2025	9/13/2025	Completed
Confirm SDLC Methodology	9/14/2025	9/26/2025	Completed
Develop Project Plan	9/17/2025	9/27/2025	Completed
Conduct Project Risk Assessment	9/28/2025	10/2/2025	Completed
Requirement Analysis	10/26/2025	12/3/2025	Completed
Identify Functional Requirements	10/26/2025	11/5/2025	Completed
Identify Non-Functional Requirements	11/6/2025	11/13/2025	Completed
OWASP Focus Area (Security Fundamentals)	11/14/2025	11/16/2025	Completed
Project Approval	11/17/2025	11/21/2025	Completed
Proposal Start	11/22/2025	12/3/2025	Completed
Schedule Key Project Milestone	11/22/2025	11/24/2025	Completed
Revise Proposal with feedback	11/30/2025	11/30/2025	Completed
Proposal Submission	12/3/2025	12/3/2025	Completed
Sprint 1	12/4/2025	12/18/2025	Completed
Initial Setup and Tool Installation	12/4/2025	12/4/2025	Completed
Install Python environment	12/4/2025	12/5/2025	Completed
Install and configure the 15 Open-Source Security Tools	12/6/2025	12/8/2025	Completed
Install Core Python Environment (Fast API, Subprocess, Gemini API)	12/9/2025	12/15/2025	Completed
Verify Tool Execution via Python Scripts	12/16/2025	12/18/2025	Completed
Sprint 2	12/19/2025	1/1/2026	Completed
Backend API and Architecture Development	12/19/2025	1/1/2026	Completed
Set up Fast API Framework & Core Logic	12/19/2025	12/23/2025	Completed
Implement Persistent Results Data Storage	12/24/2025	12/27/2025	Completed
Implement Structured Output Logic & Parsers	12/28/2025	12/30/2025	Completed
Implement Asynchronous Execution Logic	12/31/2025	1/1/2026	Completed
Backend Core Development	12/30/2025	12/30/2025	Completed
Interim Report Submission	12/30/2025	12/30/2025	Completed
Sprint 3	1/2/2026	1/16/2026	In progress
Intelligent Workflow Development	1/2/2026	1/16/2026	In progress
Implement User Input Handler (Domain, OWASP Focus)	1/2/2026	1/4/2026	Completed
Develop the Intelligent Pre-Check (Naabu, Httpx)	1/5/2026	1/8/2026	Completed
Integrate the AI Decision Node (Gemini API)	1/9/2026	1/12/2026	In progress
Develop the AI-Driven Execution Validation Logic	1/13/2026	1/16/2026	To Do
Sprint 4	1/17/2026	2/1/2026	To Do
Frontend Development	1/17/2026	2/1/2026	To Do
Develop Unified Interface (HTML/CSS/JS)	1/17/2026	1/21/2026	To Do
Implement Target/OWASP Attack/Tool Selection Forms	1/22/2026	1/26/2026	To Do
Design Real-time Status	1/27/2026	1/29/2026	To Do
Implement Results Dashboard Display	1/30/2026	2/1/2026	To Do
Frontend	1/31/2026	1/31/2026	To Do

Figure 25 Progress Table

Sprint 5	2/2/2026	2/16/2026	To Do
Reporting Module Development	2/2/2026	2/16/2026	To Do
Design Report Template	2/2/2026	2/6/2026	To Do
Develop PDF/HTML Generation Logic	2/7/2026	2/12/2026	To Do
Implement Downloadable Report Feature	2/13/2026	2/16/2026	To Do
Sprint 6	2/17/2026	3/3/2026	To Do
Integration and Testing	2/17/2026	3/3/2026	To Do
Backend-Frontend API Integration	2/17/2026	2/22/2026	To Do
End-to-End Workflow Testing	2/23/2026	2/28/2026	To Do
Validate AI Triage Accuracy and Efficiency	2/28/2026	3/3/2026	To Do
Control	3/4/2026	3/31/2026	To Do
Monitoring and change control if needed	3/4/2026	3/15/2026	To Do
Documentation Updates	3/16/2026	3/24/2026	To Do
Configuration and environment management	3/25/2026	3/31/2026	To Do
Closure	4/1/2026	4/22/2026	To Do
Final Project Documentation	4/1/2026	4/15/2026	To Do
Final Viva/Project Presentation Prepare	4/8/2026	4/18/2026	To Do
Project Submission (Development, Presentation, Documentation)	4/19/2026	4/22/2026	To Do
Final Report Submission	4/22/2026	4/22/2026	To Do

Figure 26 Progress Table

4.2. Progress Review

4.2.1. Current Scenario of progress

The Project planning and Sprint 1 of IRS are achieved. Sprint 2 has also been finished and it provided the core and architecture foundation of the backend a bit earlier than expected. Since these two sprints had been completed earlier than expected work under Sprint 3 was initiated earlier. Simultaneously, a pre-survey was carried out among the students of the educational area to obtain their views on automated reconnaissance systems. This survey feedback is being applied to improve IRS features, workflow and usability (see [Appendix 7.8](#)).

To address the project in an iterative manner, IRS product backlog has been developed to rank the tasks within the sprints. The estimated amount of effort of each backlog item (tool integration, dashboard components, AI decision logic and reporting modules, etc., and in the case of the Sprint 3 item, which are currently underway) are estimated using story points (see [Appendix 10](#)). Several design and analysis diagrams have been created too including system architecture, flowchart, UML sequence diagram (see [Diagrams](#)).

The IRS SRS is finished and outlines both functional and non-functional requirements of the system, which forms a strong basis on further development (see [Appendix 9](#)).

Altogether, IRS has good progress as Sprints 1 and 2 are finished and Sprint 3 activities are already in progress ([Development Progress](#)).

4.2.2. Progress Timeline

The IRS is an iterative growth process that includes feedback during each sprint. As the work continued, the initial plan was revised as well as the Gantt chart and WBS were updated to the real order of activities. Initiation, planning, Sprint 1 and Sprint 2 were finished on or ahead of schedule thus permitting some of the Sprint 3 to be initiated early before planned. Even though Sprint 3 is technically a future work, its early starting has assisted the project in keeping overall progress on track.

4.2.3. Action Plan

The remaining sprint activities of the Intelligence Recon System (IRS) will be done as per the revised Gantt chart to ensure that the project follows the schedule. The development process will be parallel to testing, optimization and documentation so that everything remains stable and working. Every sprint will be undertaken in a systematic manner, addressing challenges in the process of completing the project to the latter.

5. FURTHER WORK

5.1. Phases to complete

Interim Report Submission	12/30/2025	12/30/2025	Completed
Sprint 3: Intelligent Workflow Development	1/2/2026	1/16/2026	In progress
Implement User Input Handler (Domain, OWASP Focus)	1/2/2026	1/4/2026	Completed
Develop the Intelligent Pre-Check (Naabu, Httpx)	1/5/2026	1/8/2026	Completed
Integrate the AI Decision Node (Gemini API)	1/9/2026	1/12/2026	In progress
Develop the AI-Driven Execution Validation Logic	1/13/2026	1/16/2026	To Do
Sprint 4: Frontend Development	1/17/2026	2/1/2026	To Do
Develop Unified Interface (HTML/CSS/JS)	1/17/2026	1/21/2026	To Do
Implement Target/OWASP Attack/Tool Selection Forms	1/22/2026	1/26/2026	To Do
Design Real-time Status	1/27/2026	1/29/2026	To Do
Implement Results Dashboard Display	1/30/2026	2/1/2026	To Do
Frontend	1/31/2026	1/31/2026	To Do
Sprint 5: Reporting Module Development	2/2/2026	2/16/2026	To Do
Design Report Template	2/2/2026	2/6/2026	To Do
Develop PDF/HTML Generation Logic	2/7/2026	2/12/2026	To Do
Implement Downloadable Report Feature	2/13/2026	2/16/2026	To Do
Sprint 6: Integration and Testing	2/17/2026	3/3/2026	To Do
Backend-Frontend API Integration	2/17/2026	2/22/2026	To Do
End-to-End Workflow Testing	2/23/2026	2/28/2026	To Do
Validate AI Triage Accuracy and Efficiency	2/28/2026	3/3/2026	To Do
Control	3/4/2026	3/31/2026	To Do
Monitoring and change control if needed	3/4/2026	3/15/2026	To Do
Documentation Updates	3/16/2026	3/24/2026	To Do
Configuration and environment management	3/25/2026	3/31/2026	To Do
Closure	4/1/2026	4/22/2026	To Do
Final Project Documentation	4/1/2026	4/15/2026	To Do
Final Viva/Project Presentation Prepare	4/8/2026	4/18/2026	To Do
Project Submission (Development, Presentation, Documentation)	4/19/2026	4/22/2026	To Do
Final Report Submission	4/22/2026	4/22/2026	To Do

Figure 27 Phases to Complete

5.1.1. Remaining Development

Remaining development tasks include completing Sprint 4 frontend work, finalizing the unified web interface, and implementing any pending AI-driven triage and tool-selection logic. The interface must fully support configuring targets, selecting OWASP categories, launching scans, viewing consolidated results, and triggering report generation, while the backend AI layer is refined using real scan outcomes, so decisions are consistent and reliable.

5.1.2. Integration and Testing

After development, the full workflow must be executed end-to-end, from user input through asynchronous tool orchestration, AI decisions, data storage, and reporting. Comprehensive tests on representative domains will be used to validate backend–frontend API integration, verify AI prioritization accuracy, and identify issues such as timeouts, parsing errors, or broken UI flows so all modules operate smoothly together.

5.1.3. Control and Optimization

Once core correctness is validated, work shifts toward stability, performance, and operational readiness. Performance will be tuned for concurrent scans, error handling and logging strengthened for easier troubleshooting, and basic monitoring plus change-control practices defined to manage configuration updates and environment changes in lab or team settings.

5.1.4. Final Optimization

The final phase focuses on targeted UI and configuration refinements and completion of project documentation. Adjustments will ensure all components align with the intended system features, while concise documentation is updated in parallel with testing to capture workflows, AI behavior, and best practices so the system is clearly understood and ready for real-time deployment as an effective cybersecurity solution.

6. REFERECES

- Alexander Kolesnikov, V. M., 2024. *Secure List*. [Online]
Available at: <https://securelist.com/vulnerability-report-q1-2024/112554/>
[Accessed 29 December 2025].
- Ariganello, J., 2022. *Anomali*. [Online]
Available at: <https://www.anomali.com/blog/more-is-less-the-challenge-of-utilizing-multiple-security-tools>
[Accessed 23 November 2025].
- Cobrief, 2025. *Cobrief*. [Online]
Available at: <https://www.cobrief.app/resources/legal-glossary/methodology-overview-definition-and-example/>
[Accessed 30 November 2025].
- Day, B., 2025. *Forbes*. [Online]
Available at: <https://www.forbes.com/advisor/business/what-is-waterfall-methodology/#what-is-the-waterfall-methodology->
[Accessed 27 November 2025].
- Donetonic, 2025. *Donetonic*. [Online]
Available at: <https://donetonic.com/steps-to-configure-your-scrum-workflow/>
[Accessed 20 December 2025].
- Doshi, H., 2016. *Scrum.org*. [Online]
Available at: <https://www.scrum.org/resources/blog/three-pillars-empiricism-scrum>
[Accessed 30 Noember 2025].
- draw.io, 2023. *Draw.io*. [Online]
Available at: <https://www.drawio.com/blog/types-of-technical-diagrams>
[Accessed 3 December 2025].
- Eastgate, 2025. *Eastgate*. [Online]
Available at: <https://eastgate-software.com/7-stages-of-software-development-life-cycle-sdlc-you-need-to-know/>
[Accessed 26 November 2025].
- geeksforgeeks.org, 2025. *geeksforgeeks.org*. [Online]
Available at: <https://www.geeksforgeeks.org/system-design/use-case-diagram/>
[Accessed 29 December 2025].
- GeeksForGeeks, 2025. *GeeksForGeeks*. [Online]
Available at: <https://www.geeksforgeeks.org/software-engineering/software-development-life-cycle-sdlc/>
[Accessed 26 November 2025].
- GeeksForGeeks, 2025. *GeeksForGeeks*. [Online]
Available at: <https://www.geeksforgeeks.org/software-engineering/software-engineering-sdlc-v-model/>
[Accessed 27 November 2025].

- InterviewBit, 2023. *InterviewBit*. [Online]
Available at: <https://www.interviewbit.com/blog/system-architecture/>
[Accessed 24 December 2025].
- Kamis, 2019. *materimateripti*. [Online]
Available at: <https://materimateripti.blogspot.com/2019/11/rekayasa-perangkat-lunak-rekayasa.html>
[Accessed 27 November 2025].
- LaPrad, L., 2025. *TeamGantt*. [Online]
Available at: <https://www.teamgantt.com/what-is-a-gantt-chart>
[Accessed 25 December 2025].
- Manager, P., 2025. *Project Manager*. [Online]
Available at: <https://www.projectmanager.com/guides/work-breakdown-structure>
[Accessed 29 November 2025].
- Ojha, Y., 2025. *Github*. [Online]
Available at: <https://github.com/yogeshojha/engine>
[Accessed 17 December 2025].
- Peek, S., 2024. *Business News Daily*. [Online]
Available at: <https://www.businessnewsdaily.com/4987-what-is-agile-scrum-methodology.html>
[Accessed 27 November 2025].
- Sasovets, I., 2025. *Techmagic*. [Online]
Available at: <http://techmagic.co/blog/automated-security-testing>
[Accessed 26 November 2025].
- Simran, S., 2025. *bdtask*. [Online]
Available at: <https://www.bdtask.com/blog/software-development-life-cycle>
[Accessed 27 November 2025].
- SOCRadar, 2025. *SOCRadar*. [Online]
Available at: <https://socradar.io/labs>
[Accessed 17 December 2025].
- Staff, C., 2025. *Coursera*. [Online]
Available at: <https://www.coursera.org/articles/what-is-milestone?msocid=2b281f38e3ab640a32060985e2306557>
[Accessed 25 December 2025].
- Sumanth, G., 2023. *APPSECEngineer*. [Online]
Available at: <https://www.appsecengineer.com/blog/manual-vs-automated-reconnaissance>
[Accessed 23 November 2025].
- Team, P., 2025. *Pentest Tools*. [Online]
Available at: [Team, Pentest](#)
[Accessed 17 December 2025].

Vasiliauskas, V., 2025. *teamhood*. [Online]

Available at: <https://teamhood.com/agile/scrum-advantages-disadvantages/>

[Accessed 27 December 2025].

Welch, J., 2025. *Profit.co*. [Online]

Available at: <https://www.profit.co/blog/task-management/the-5-phases-of-waterfall-project-management/>

[Accessed 27 November 2025].

7. APPENDIX

7.1. Appendix 1: Waterfall Methodology

Waterfall methodology refers to the linear project approach that is executed in phases where each phase (requirements, design, implementation, testing, deployment, and maintenance) is accomplished to completion which is followed by the subsequent one. It is based on high level upfront planning and documentation and is less flexible in terms of scope modification after development has begun. This is practical in stable and predictable projects but not practical in dynamic or research driven work (Day, 2025).



Figure 28 Waterfall Methodology (Welch, 2025).

Why Waterfall was considered.

- Simple and easy to follow, having a definite line of stages.
- Good records in every stage, which is suitable for academic reporting and assessment.
- Works with the requirements that may appear clear at the start of the project.

Why Waterfall was not chosen.

- All your project needs (AI behavior, tool chaining, UI) will change probably as you play around.
- Revising the previous stages is difficult, and this is dangerous to an innovative security system.
- You require feedback, testing, and improvement regularly which can be best facilitated by the iterative Agile/Scrum. [Click here](#) to go back.

7.2. Appendix 2: V-Model Methodology

V-Model is a verification/ validation SDLC model in which each development phase on the left (requirements, system design, detailed design, coding) has an equivalent testing phase on the right (unit, integration, system, acceptance testing). The rationale is to design tests in advance and make sure that each requirement is met by a test activity (GeeksForGeeks, 2025).

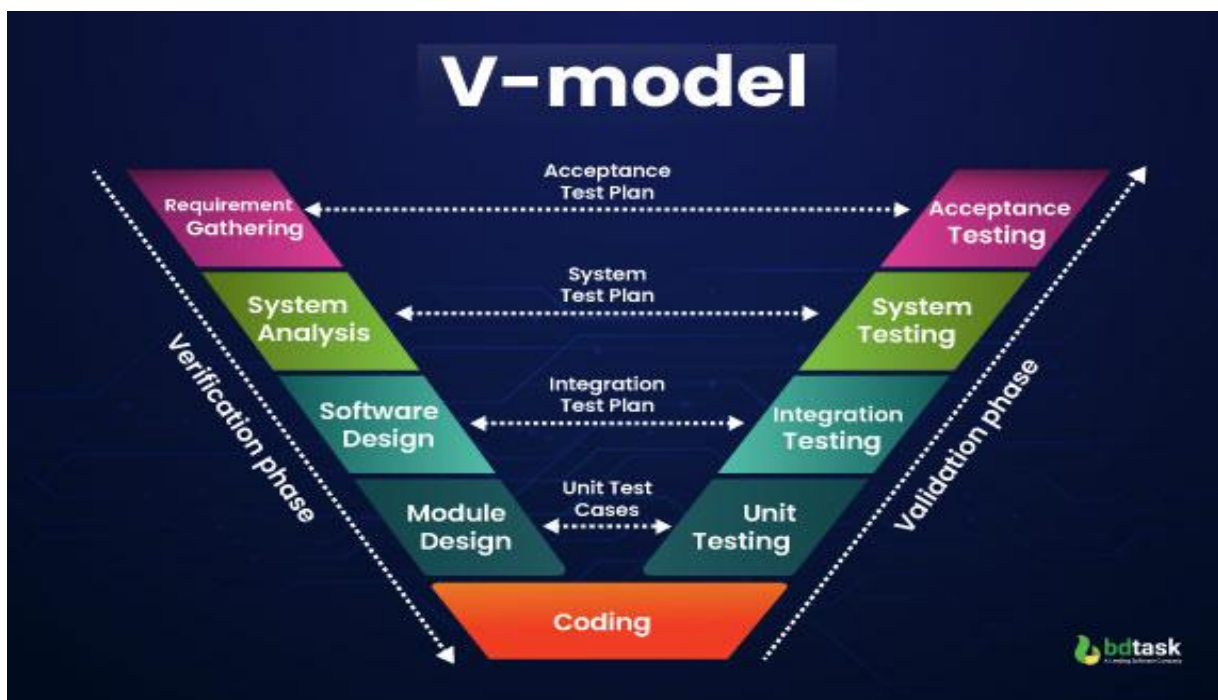


Figure 29 V-Model Methodology (Simran, 2025).

Why was it considered.

V-Model is desirable due to the following reasons:

- The specifications are quite precise: single internet-based platform, OWASP based recon, AI triage, 15 tools, reporting, etc., thus, it is possible to map each requirement to tests.
- It is a small to medium sized project and that is where V-Model is best suited and its well-written documentation and test planning can make a student project look like an academic endeavor.
- Security domain the security domain enjoys good verification: unit tests of parsers, integration tests of tool execution, and system tests of end-to-end scans are all compatible with the systematic testing focus of the V-Model.

Why was it not chosen?

Nevertheless, V-Model is not perfectly appropriate in your case, and Agile/Scrum (sprint-based plan that you are applying) is more suitable:

- The requirements will tend to change, requirements will vary as you experiment, and V-model assumes that requirements will be fixed and stable, whereas the V-model is inflexible when you need to make mid-project changes.
- In pure V-Model, working software is delivered late, whereas this project requires early prototypes (backend first, then intelligent workflow, then frontend) to demonstrate progress at interim milestones and make revisions.
- V-Model is paperwork-intensive and not as flexible as can be an overhead to a one-developer student project where rapid turnaround and quick completion is more significant than phase gates.

[Click here](#) to go back.

7.3. Appendix 3: Agile Methodology

Agile methodology is an adaptable approach to project management through small, iterative units of work and releasing them at an appropriate frequency, followed by constant feedback and interaction amongst the team members to enable quick learning to adapt to the change (Peek, 2024). Agile project management is chosen to be used in the Intelligence Recon System as it allows iterative development, constant feedback and allowing flexibility in case of change in requirements or technical details.



Figure 30 Agile Methodology (Kamis, 2019).

Agile project management has four key values which can be directly applied to the IRS project:

- **People back processes and tools:** Cooperation between the developer and supervisors is encouraged to streamline recon workflows, OWASP-concerned features, and AI prompts, rather than observe a pre-determined process.
- **Coding more than documentation:** With each iteration, a working portion of IRS (e.g., executing a subset of the tools for creating an initial report) must be created, so that operational software is the main gauge of success.
- **Academic cooperation within the limits that are fixed:** Continued cooperation with academic supervisors and the FYP department is made more than keeping the original project scope unaltered.

- **Reactive to change after a plan:** As the behavior of tools, AI models, or constraints in a project evolve, Agile enables a backlog and design decision modification to make the system realistic and maintainable.

[Click here](#) to go back.

7.4. Appendix 4: Scrum Methodology

Scrum is a special agile model that puts this concept into practice by short sprints that are definitely fixed and in which the team takes high priority tasks out of a backlog and delivers a usable increment at the end of each sprint (Peek, 2024). It encourages teamwork, transparency, and constant enhancement, which is in line with the requirements of IRS as a multifaceted system with various tools and AI logic.

- The project implies research, AI integration, various tools, and thus the requirements will be changing during development.
- The supervisors and other stakeholders should provide frequent feedback to improve features such as the dashboard, AI triage, and reporting.
- The team should demonstrate ongoing improvements (prototypes, partial features) rather than till the very end.

7.4.1. Role of scrum

Product Owner (role equivalent): The project supervisor, or stakeholder, who communicates the project vision, assists in prioritization of features.

Scrum Master (role equivalent): The person who oversees the process, by making sure that the Scrum practices (sprint planning, Scrum reviews, Scrum retrospectives) are adhered to as well as barriers prevented and overcome.

Scrum Team: The developer(s) who implements the development parts of the project.

Advantages

- Scrum is very dynamic and flexible, and it is suited to dynamic environments where the requirements are not clear.
- It encourages innovation and creativity by means of close working of ideas.
- It allows acceleration of product delivery or feature, leading to time to market.
- It can reduce the expenses by having less paperwork and formal control as compared to the conventional approach.
- It is more transparent and the ownership is shared, which enhances trust, productivity, and work quality.
- Constant feedback and regular changes tend to make the customer more satisfied.
- In Scrum, individual accountability and ownership enhance motivation and satisfaction of the teams.

Disadvantages

- Scrum needs experienced members of the team that can work hard and provide quality feedback.
- Teams require a long training course which may take months without experience.
- It is complicated and hard to control the process of scaling Scrum to large projects or numerous teams.
- Concentration on little steps does not ensure the delivery of the total project time.
- Scrum may be difficult to blend with more conventional and plan-based models such as waterfall.

These are the key points which show the advantages and disadvantages of scrum methodology (Vasiliauskas, 2025).

7.4.2. Justification for choosing Methodology.

The best methodological choice to the project of the Intelligence Recon System is Agile Scrum due to its ability to support requirements changes, feedback, and continuous delivery of the working part.

- **The changing demands and experimentation.**

The project will be associated with the integration of numerous security tools, tuning AI prompt and changing dashboards and reports based on the testing of the needs, and thus, the requirements cannot be entirely determined initially. The benefits of Scrum in changing scope and priorities between iterations include short sprints and backlog refinement, which do not disrupt the entire plan.

- **Early and incremental working software required.**

The project roadmap is sprint-based (backend setup, intelligent workflow, frontend, reporting, integration) with well-defined interim and final milestones, thus there is an advantage of delivering a working increment at the end of each sprint (for e.g. backend API on Sprint 2, intelligent workflow on Sprint 3, UI on the Sprint 4). Scrum is specifically intended to provide a potentially shippable increment at every sprint, which is in line with the idea of showing progress during reviews and to supervisors.

- **Close cooperation, feedback and quality**

The project demands unending feedback from supervisors and testers regarding security workflow, AI behavior and usability, which Scrum sustains through sprint reviews and retrospectives. Agile principles also adopt the importance of continuous testing and enhancement of each sprint, which also plays an important role in a security-oriented system that reliability and correctness are paramount.

[Click here](#) to go back.

7.5. Appendix 5: Previous WBS



Figure 31 Previous WBS

[Click here](#) to go back.

7.6. Appendix 6: Updated WBS



Figure 32 Updated WBS

[Click here](#) to go back.

7.7. Appendix 7: Milestone

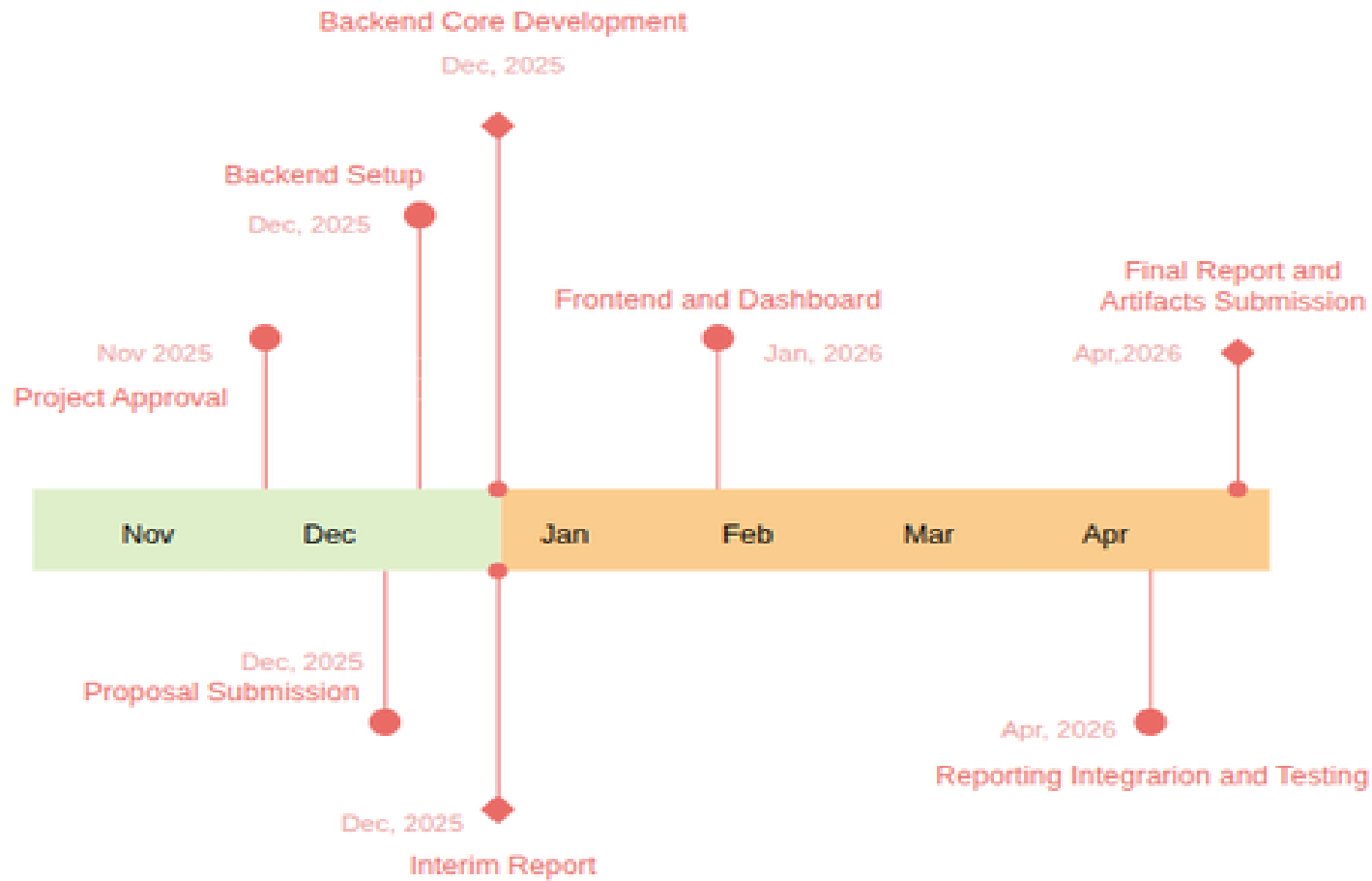


Figure 33 Milestone

Milestone 1: Project Approval

The milestone includes finding the topic of the project, clarifying the project goals, and getting the approval of the supervisor and department. It involves picking the Intelligence Recon System as the project name, preparing the original concept, and definition of the problem, scope and planned results of the research and development part.

Milestone 2: Proposal Submission

This milestone is dedicated to the preparation and submission of the official project proposal. It contains a record of background research, problem analysis, literature review, and system objectives. The proposal identifies the intended methodology, system architecture, tools and technologies to be employed and an approximate timeframe of the development with assessment criteria.

Milestone 3: Backend Setup

This milestone is concerned with defining the initial framework of system development. It involves the installation of the back-end framework with Fast API, the initial database setup, setting up the environment variables, and developing the bare bones of the application in which functionalities and testing will be performed.

Milestone 4: Backend Core Development

The milestone is dedicated to the creation and development of the main logic of the backend, which is the logic behind the functioning of the system. It involves the application of data models, API routing and scan management logic, integration of AI modules, connection to database and coordination of real time processes. The milestone will make sure that the backend will be able to support the process of the tools, data storage, and secure transactions.

Milestone 5: Interim Report Submission

This milestone includes the process of preparing the interim report to show the general progress, analysis, and current development status. It also involves brief reporting of system design, methodology, initial implementation findings, issues facing and modification of the following plan of work as per interim evaluation findings.

Milestone 6: Frontend and Dashboard

This milestone is aimed at creating the interface of the Intelligence Recon System. It involves the design of the dashboard with the help of HTML, CSS, and JavaScript, and the development of the sections to control targets, the choice of the tools, and the input of the OWASP categories. It also includes interface responsiveness, accessibility design, and support to backend APIs to be responsive in real-time.

Milestone 7: Reporting Integration and Testing

This milestone focuses on incorporation of user reporting capability and system level testing. It involves writing structured HTML-based reports, connecting the frontend and backend modules to generate outputs and testing important processes, including scanning, AI-based decision-making, and errors, as well as overall system performance.

Milestone 8: Final Report and Artifacts Submission

This includes final report creation and submission of all final project documentation and deliverables. It comprises the creation of the full system report, the presentation materials and technical documentation such as code, configuration files or user manual.

[Click here](#) to go back.

7.8. Appendix 8: Findings of pre-survey

7.8.1. Findings of the pre-survey

The Intelligence Recon System pre-survey involved 13 people who do actively work in the field of reconnaissance or vulnerability scanning, and they use both CLI and web-based tools. Most of the people interviewed run one or two tools at a time in terminal and said that the most challenging step about recon is getting the results of many tools in one display, then remembering various commands and interpreting raw data. Most of them replied that a single web dashboard, AI-assisted tool selection, centralized scan history, and auto-generated OWASP-based reports will be useful or especially useful, which points to high demand levels on the basic features, planned in IRS.

13 responses

 [Link to Sheets](#)

Summary

Question

Individual

Who has responded?

Email

surajgajalu@gmail.com

gauravdahala@gmail.com

abhinavshrestha83@gmail.com

evaniraut@gmail.com

grvmalla@gmail.com

sameeremailaddress@gmail.com

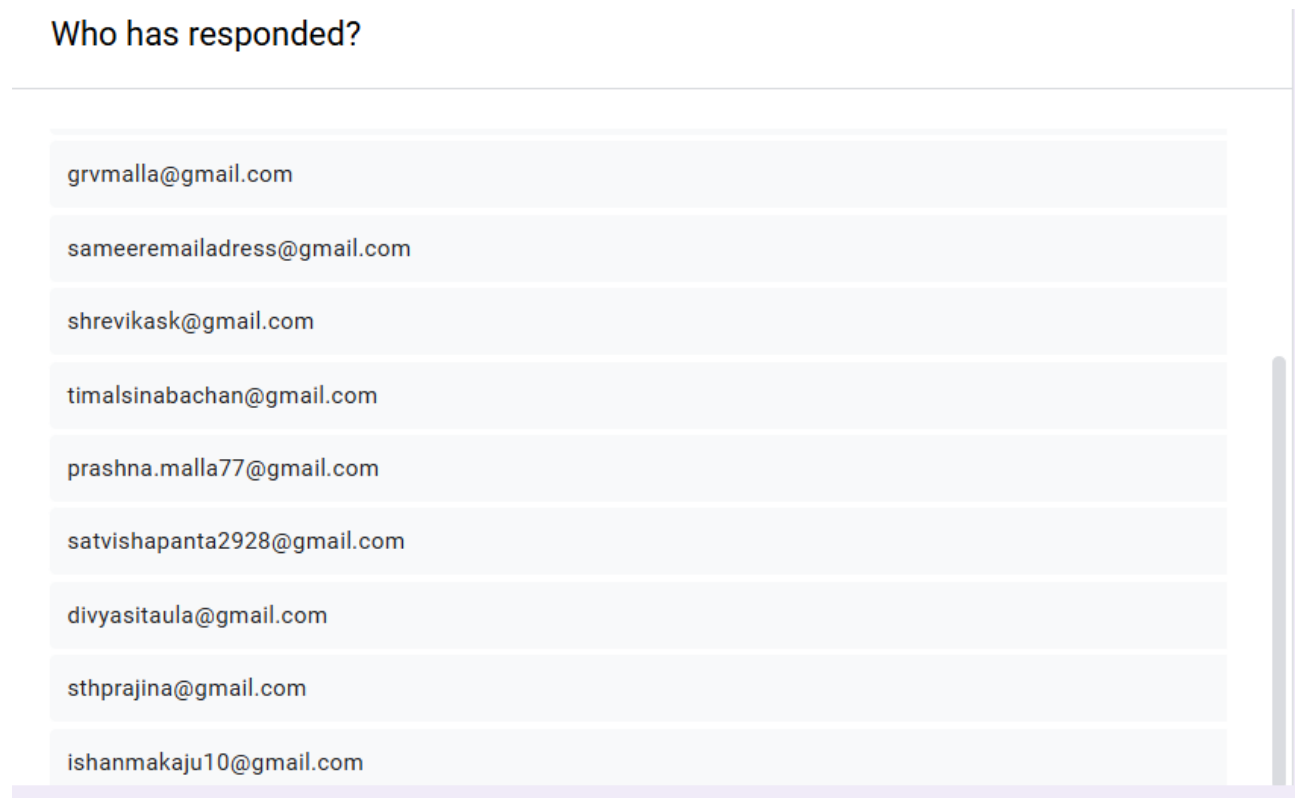
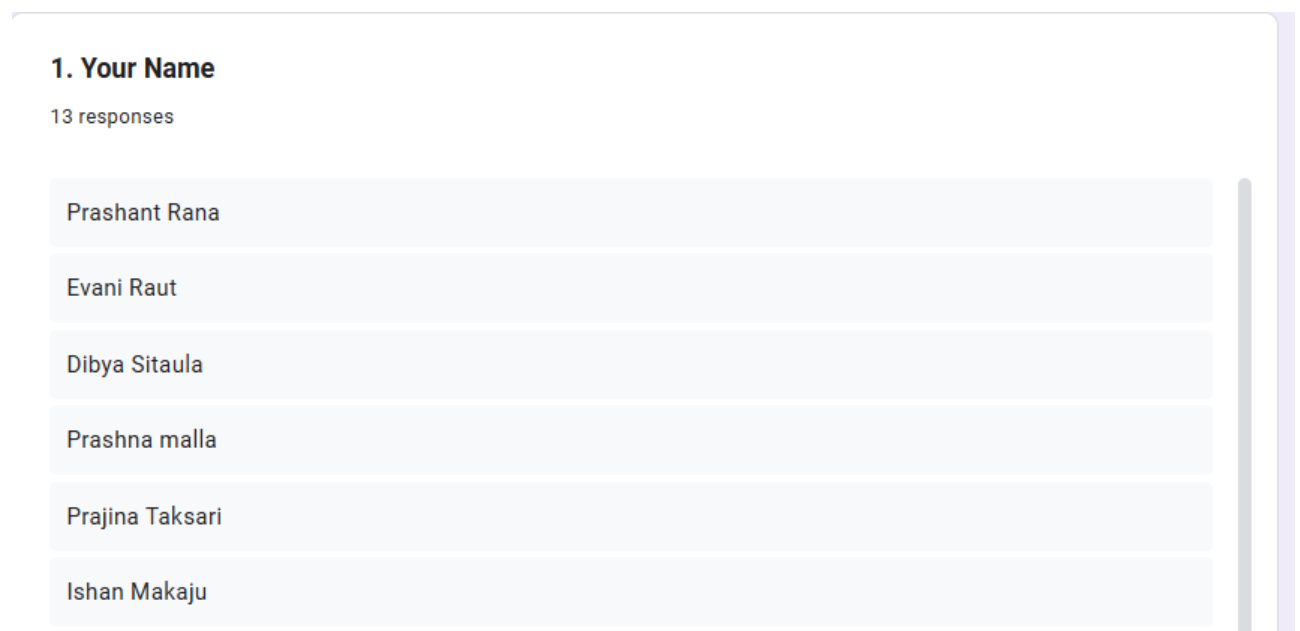


Figure 34 Pre-Survey Finding 1



1. Your Name

13 responses

Prajina Taksari

Ishan Makaju

bachan

Sameer sah

Shrevika Khadka

Satvisha

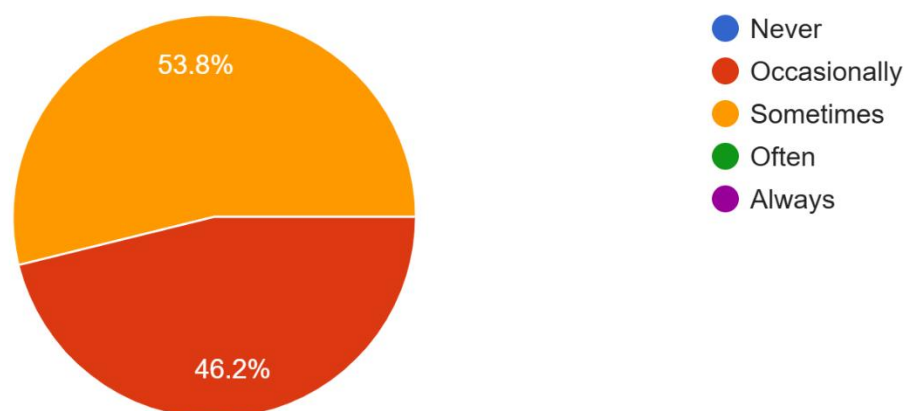
Abhinav shrestha

Gaurav Dahal

Gaurav Malla

*Figure 35 Pre-Survey Finding 2***2. How often do you perform reconnaissance or vulnerability scans?**

13 responses

*Figure 36 Pre-Survey Finding 3*

3. Which tools do you mainly use for recon now?

13 responses

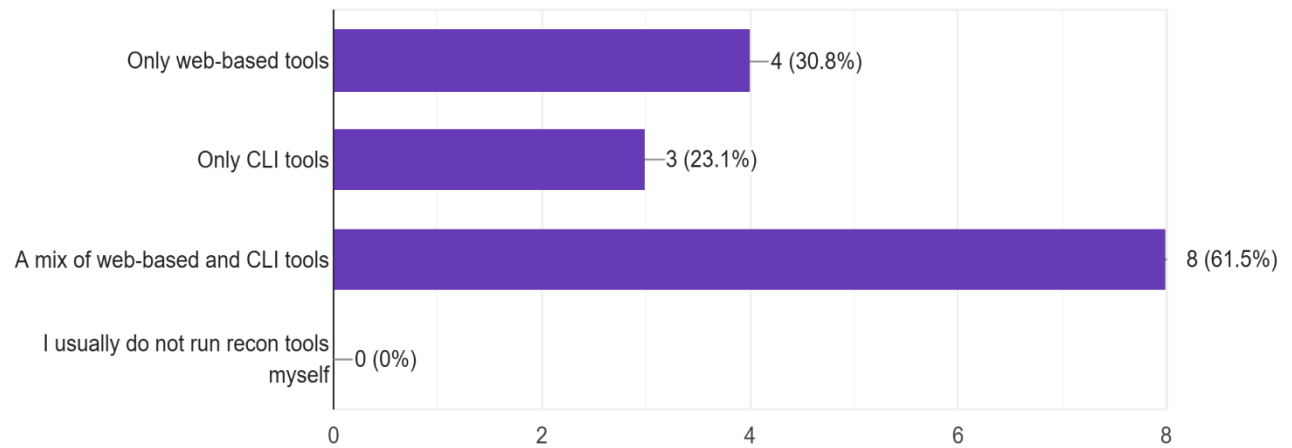


Figure 37 Pre-Survey Finding 4

4. How do you usually run multiple tools for a single target?

13 responses

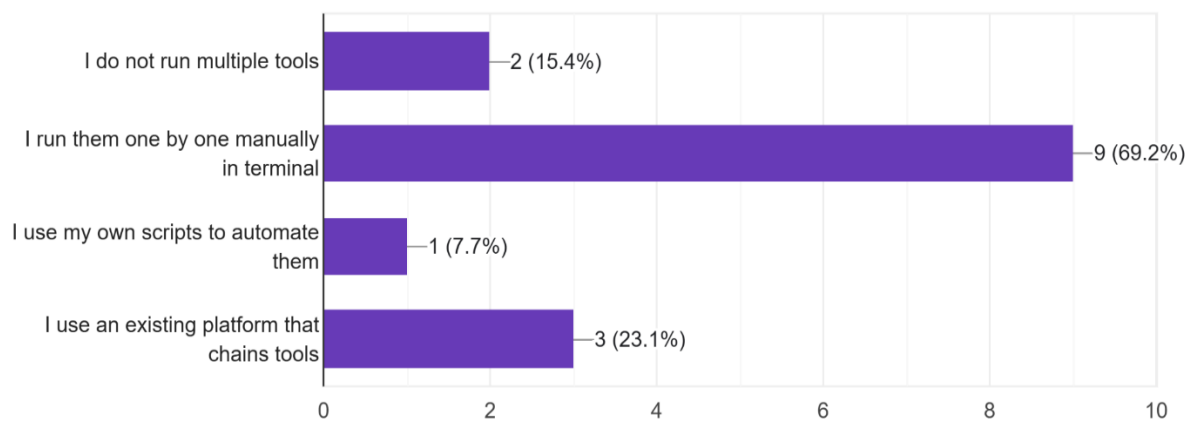


Figure 38 Pre-Survey Finding 5

5. What is the most difficult part of recon for you?

13 responses

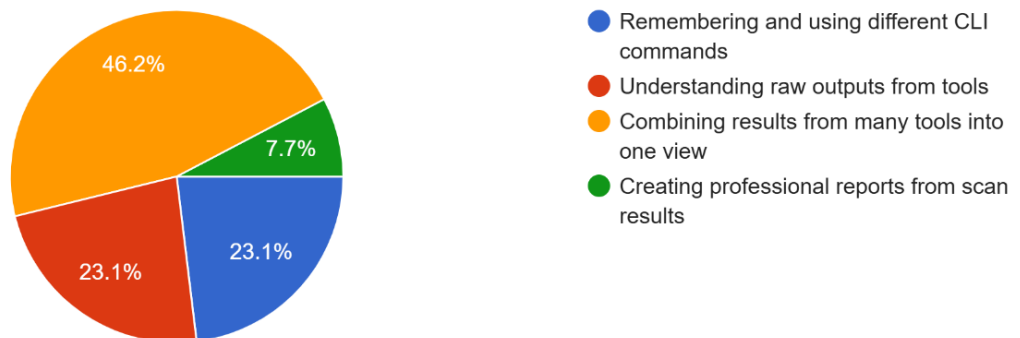


Figure 39 Pre-Survey Finding 6

6. How useful would a single web dashboard that runs multiple recon tools for you be?

13 responses

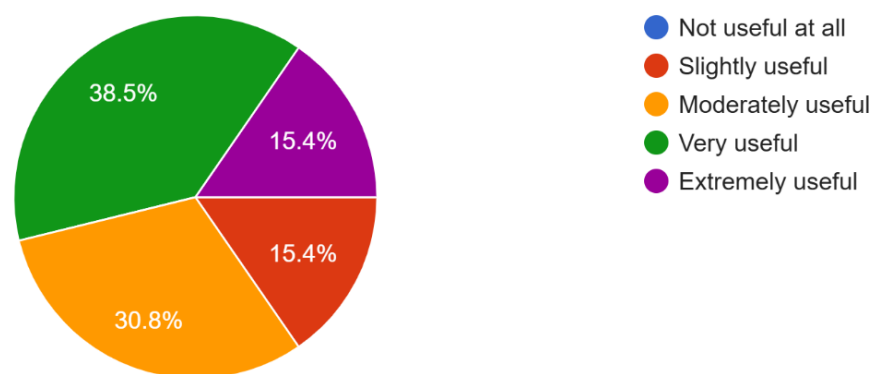


Figure 40 Pre-Survey Finding 7

7. How interested are you in AI-based tool selection (AI decides which tools to run based on target and goal)?

13 responses

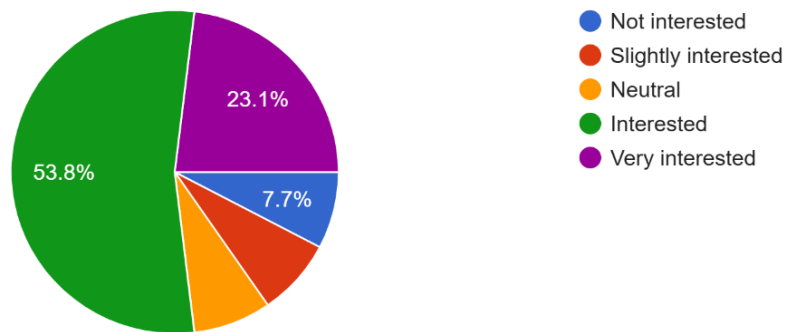


Figure 41 Pre-Survey Finding 8

8. If AI sometimes selects wrong or unnecessary tools, what would you prefer?

13 responses



Figure 42 Pre-Survey Finding 9

9. How important is it to store all scan history and results in one centralized system?

13 responses

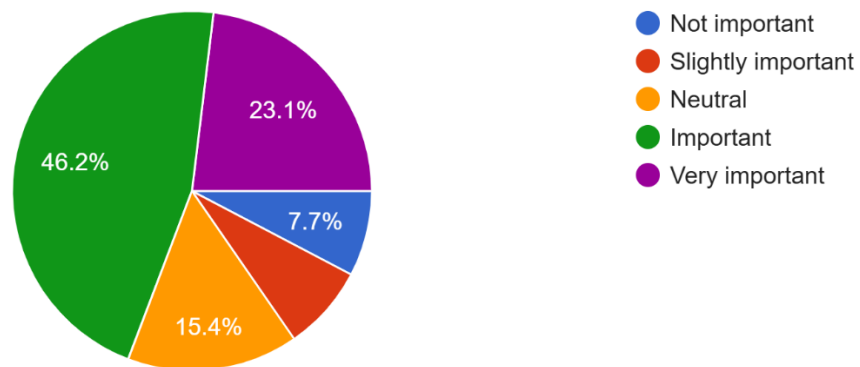


Figure 43 Pre-Survey Finding 10

10. How useful would auto-generated OWASP-based vulnerability reports (HTML/PDF) be for you?

13 responses

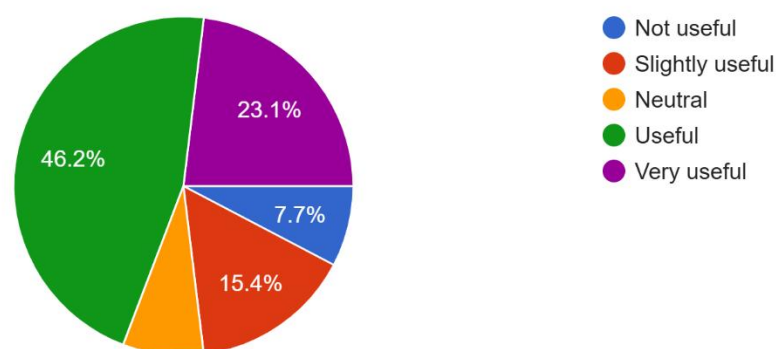


Figure 44 Pre-Survey Finding 11

11. For your own use, what format of report do you prefer?

13 responses

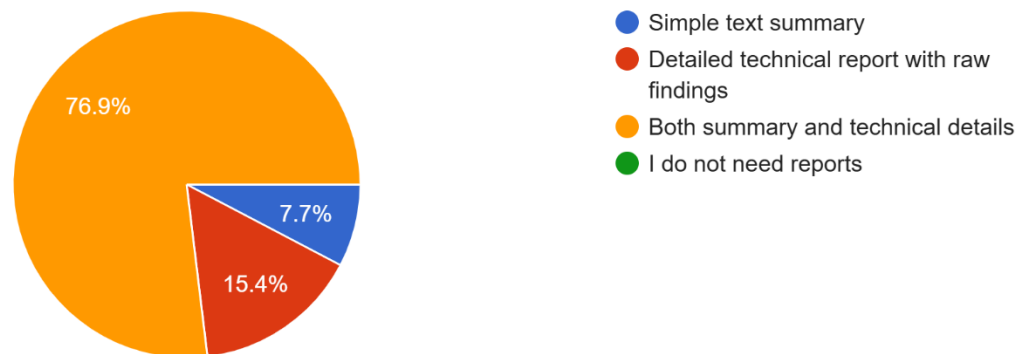


Figure 45 Pre-Survey Finding 12

12. If a system like IRS existed, how likely would you be to use it for your recon tasks?

13 responses

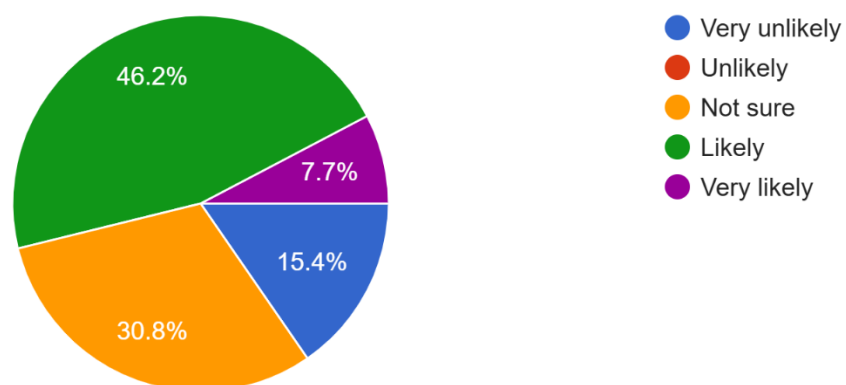


Figure 46 Pre-Survey Finding 13

13. Any other features you would like in an automated recon system?

11 responses

User friendly

No

Automatic mismatch detection

One click export

Vulnerability probing

NOT NECESSARY

Sub domain finder and OSINT

Smart Scan Recommendation Engine

Easy connection with common tools, live scan progress, clear risk levels, simple download options and easy to understand explanations.

Figure 47 Pre-Survey Finding 14[Click here](#) to go back.

7.9. Appendix 9: Software Requirements Specifications

7.9.1. Software Requirements Specifications

An elaborate Software Requirements Specification (SRS) document is critical to the development of software that satisfies business requirements as well as user expectations. Concise, clear, and executable requirements unify project teams, provide the clarity of functionality, and serve as one source of truth during the development. In the case of the Intelligence Recon System (IRS), the SRS is structured in IEEE-like fashion and includes functional and non-functional requirements of the AI-supported reconnaissance platform.

7.9.1.1. Introduction

7.9.1.1.1. Purpose

This SRS outlines the specifications of the Intelligence Recon System (IRS), which is an AI-assisted web platform that coordinates various open-source security tools to execute a specific reconnaissance and vulnerability scan, depending on OWASP categories. The system enables a user to enter a target, retrieve initial clues, have an AI decision node decide the right tools to use, perform scans asynchronously and view results of normalized findings and reports on a dashboard.

The primary intended user is:

Security Analysts: Analysts perform authorized scans, monitor status, analyze results based on severity and OWASP, and report on the results to proceed with the risk analysis or penetration.

Scope: IRS combines and coordinates existing open-source tools (e.g., Naabu) with Httpx, Nuclei, Subfinder, Amass) through a Fast API backend, AI decision node and web frontend. The scope includes the creation of scans, making intelligent choices of tools, running in parallel, normalizing the results, their storage and reporting; exploitation or constant monitoring is not in the scope of the present FYP.

7.9.1.1.2. Definitions, Acronyms, and Abbreviations.

- **IRS:** Intelligence Recon System - AI based reconnaissance and vulnerability scan system.
- **OWASP:** Open Worldwide Application Security Project; offers the OWASP Top 10 types of scans to be focused.

- **AI Decision Node / LLM:** Large language model based on Gemini that decides on which tools to execute given the first clues and OWASP category.
- **Scan:** One test case under a target using a target set of tools and OWASP focus, which is recorded in the database.
- **Tool Run:** Profile of a particular security tool in a scan, status, start and end times and summary.
- **Result:** The normalized vulnerability or reconnaissance outcome of type, severity, location, summary and evidence.

7.9.1.2. Overall Description

7.9.1.2.1. Product Perspective

IRS is an independent web application that integrates reconnaissance processes in such a way that would otherwise be conducted manually through a variety of independent command line tools. It is a browser-based dashboard, Fast API business-logic services on a backend with a business-logic node, AI decision node, and a tool-execution layer, running 15 open-source tools, storing the results in a relational database. REST APIs are made available in such a way that it allows the integration of other systems with IRS in a programmatic way.

7.9.1.2.2. Product Functions

Key functions of IRS include:

- **Target scan creation:** Takes a target (e.g., domain) and OWASP category (optionally permitting the user to select tools) and constructs a new scan record and begins processing.
- **First clue collection:** The automatic selection of Naabu and Httpx to detect open ports, HTTP services, and basic technologies is conducted, and then other tools are not chosen.
- **AI-aided tool selection:** Takes the first indications and OWASP focus and suggests tools to operate or not, with a harmless default option if the AI is not available.
- **Parallel execution:** The tool execution is provided as the selected tools execute concurrently and limited by concurrency and timeouts, and the status of the tools and errors are documented.

- Result normalization and storage Parses heterogeneous tool results into structured results by severity and location and stores all raw and processed data to be reviewed later.
- Reporting and history Reporting and history Gives a dashboard view of scans and findings and creates HTML (and PDF-ready) reports with severity and tool summaries.

7.9.1.2.3. Classes and Characteristics of User

- **Security Analysts**
 - Make and follow scans, find and report results.
 - Need OWASP-views, severity grouping and little need to touch CLI tools.

7.9.1.2.4. Operating Environment

IRS prototype is written on Ubuntu/Linux, and Python 3, Fast Api and SQLite to persist data in the database while developing and testing, although it is not mentioned that the database schema and ORM are designed to be compatible with PostgreSQL when the project becomes production-scale. The frontend is served as static HTML/CSS/JavaScript by Fast API and all interaction between the browser and the server is based on HTTP/JSON APIs.

7.9.1.2.5. Constraints of Design and Implementation

- Only open and stable reconnaissance tools are employed; they must all allow safe invocation based on argument and not shell injection.
- The names of all target input and tools should be verified and sterilized to avoid command injection or abuse.
- The implementation of AI should not go beyond the boundaries of the provider; there should be timeouts and fallbacks when the AI is not fast enough or is not available.
- There should be blocking constraints to prevent the depletion of resources when there are numerous tools or scans running.

7.9.1.3. Specific Requirements

7.9.1.3.1. Functional Requirements (FR)

FR1: Creation and Management of Scans.

IRS will enable security analysts to generate scans by giving them a target and category of OWASP and follow-up the status of each scan through creation to completion.

FR2: Preliminary Evidence Collection (Naabu, Httpx)

IRS will default on Naabu and Httpx when each scan has a new scan to gather open ports and HTTP service data prior to one of the other tools being chosen.

FR3: AI-Driven Tool Selection

IRS will provide the first hints, target specifications and OWASP category to the AI decision node and be provided with a list of tools to execute or abandon, default safe set in case of failure of the AI.

FR4: Asynchronous Execution of Tools.

IRS will run the imposed tools asynchronously with a number of tools per scan and per-tool timeouts being set.

FR5: Findings Normalization and storage.

IRS will extract tool results in form of structured findings (type, severity, location, summary, evidence) and store them in the database together with raw output references.

FR6: Status and Dashboard Views.

IRS will have dashboard pages to create scans, scan status, past scan list and inspect detailed results and tool summary.

FR7: Reporting

IRS will produce an HTML summary of the findings of a chosen scan, by severity and by the tool used, and allow export to PDF, with an external renderer being used when configured.

7.9.1.3.2. Non-Functional Requirements (NFR)

1. Performance

The system will be able to run up to 3 tools in parallel per scan and at the same time maintain the UI responsive.

2. Security

The execution of any command should be done with argument lists and without shell=True, user inputs should be strictly checked, and the keys of AI API and other secrets should be saved in environment variables and should never be logged or copied in any response.

3. Reliability

Breakages of a specific tool or the decision node of the AI should not terminate the entire scan, IRS should indicate the failed Tool Runs suitably but still should keep the successful results and enable generating the report.

4. Usability

The dashboard will employ easy to read three column design with live status update, severity badges and brief messages to make sure that an analyst can get a quick view of the scan progress, and scan results.

5. Scalability and Extensibility Scalability and Extensibility.

It will allow the addition of new tools and replace AI models with minor updates and will allow migrating to PostgreSQL in case of bigger deployments.

7.10. Appendix 10: Product Backlog

8. User Story	Justification	Points
As a security analyst, I would like to provide a target domain and choose a particular OWASP Top 10 category to ensure that the scan is limited to the relevant vulnerabilities.	Maintains clear input-related requirements as well as being in line with OWASP-based assessment objectives.	3
Being a security analyst, I would like to browse and pick among a list of 15 integrated recon and vulnerability tools so that I can manage the execution of these tools.	Enables the configuration of scans to be transparent and controllable and yet still able to validate AI.	5
As a security analyst, I would like the AI decision node to test and optimize my choice of the tool using initial hints (ports, technologies) to ensure that superfluous tools are not executed.	Elaborates scans of pertinent tools, which minimize noise and execution time.	8
As a system administrator, I would like scans to run in the background and asynchronously with obvious status reporting to ensure that the UI is responsive when scans run.	Scales become better and become more user-friendly when performing a multitude of scans or long scans.	5
As a security analyst, I would like all the tool results to be normalized, parsed, and kept in a structured format to allow me to review and correlate results easily.	Facilitates the ability to compare the output of heterogeneous tools as well as facilitate subsequent reporting.	8
As a security analyst, I would like to have a consolidated view of the findings based on the severity and OWASP category to enable me to remediate the findings easily.	Convert raw scan data into prioritized, action to be taken insight.	5

As a user, I would like to create and save an HTML/ PDF report to any completed scan to be able to share assessment results with the stakeholders.	Gives formal support of audits, presentations, and project evaluation.	5
As a system administrator, I would like to have simple monitoring, logging and error handling around scan execution to diagnose and correct failures in an efficient way.	Improves maintainability and helps with stable real time usage.	5
As a technical writer, I wish to write user and developer documentation with architecture and API usage diagrams and step-by-step scan processes to allow others to deploy and extend the system.	Make the system readable by the system assessors, operators and future developers.	5

Table 3 Product Backlog

[Click here](#) to go back

8.1. Appendix 11: Diagrams

8.1.1. Flow Chart

A flow chart of this project is a step-by-step diagram, which displays the sequence of tasks or processes the system undertakes in a recon scan, such as common symbols as a rectangle (process), a diamond (decision), and a control flow arrow.

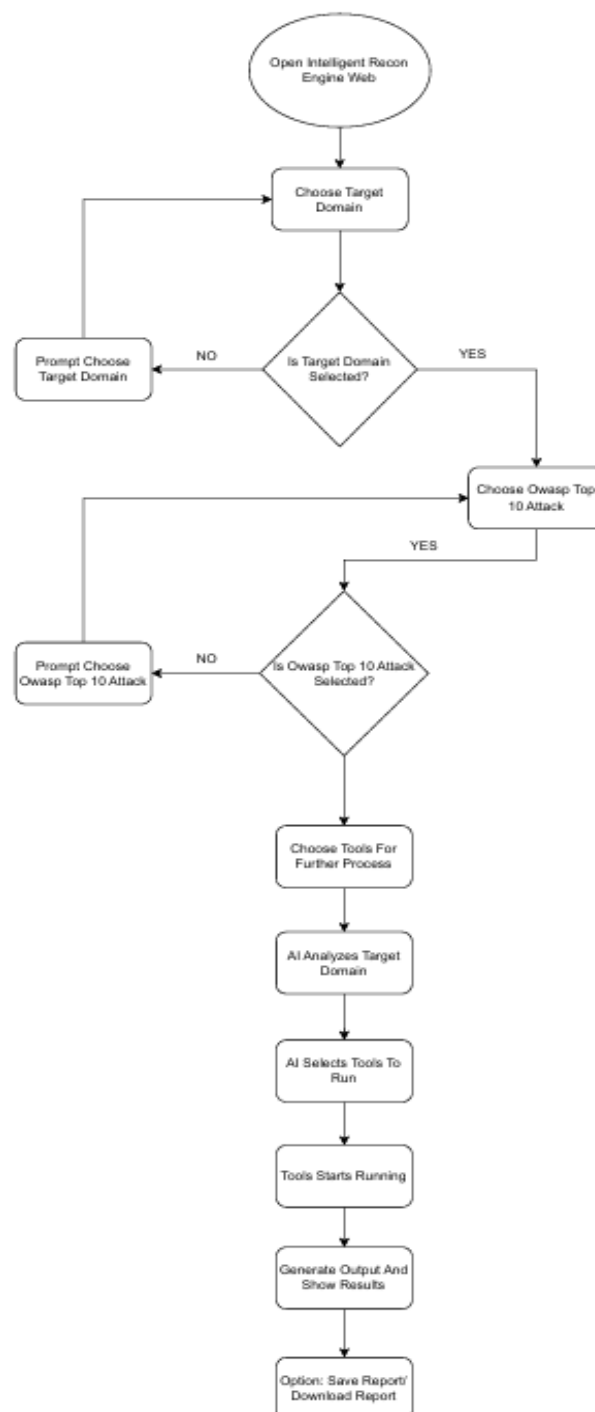


Figure 48 Flowchart Diagram

8.1.2. System architecture diagram

The system architecture diagram represents an abstract diagram of the system component architecture. It gives a brief definition of the component architecture of the system so that it may help to establish component-component relationships as well as the working of the system (InterviewBit, 2023). System architecture diagram displays HTML/CSS/JS, Fast API, Python, and Gemini API as the frontend, API, and the 15 security tools, as well as the flow of the data between them.

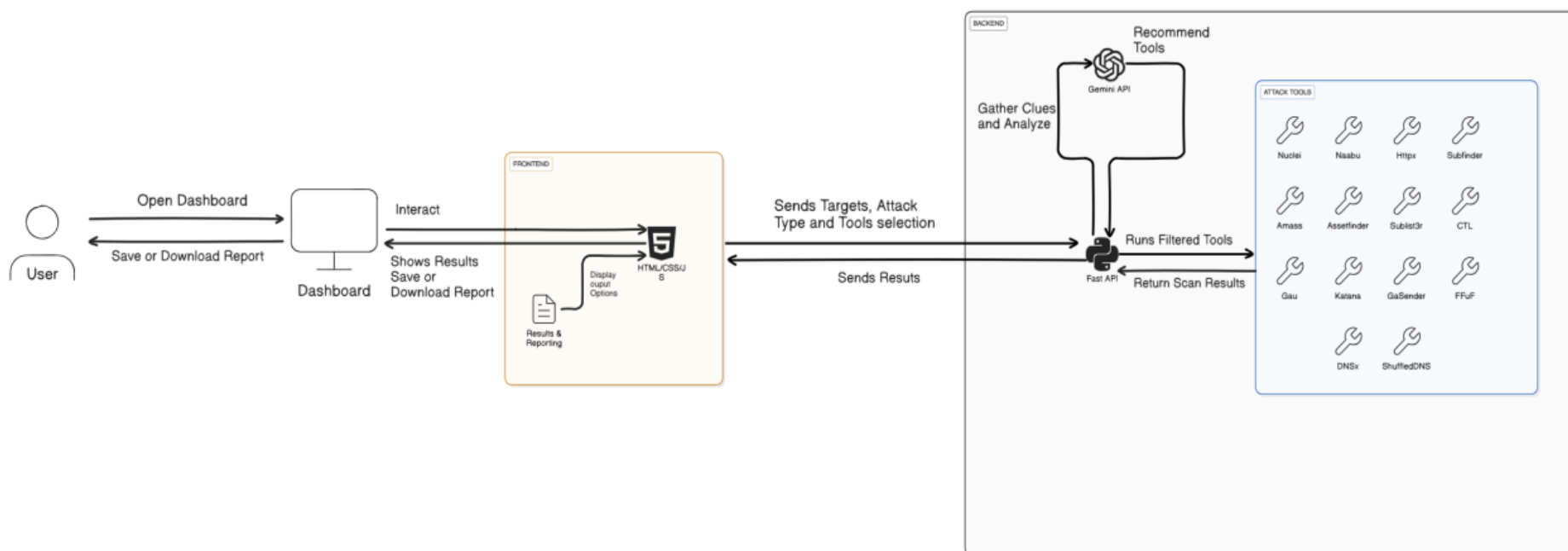


Figure 49 System Architecture Diagram

8.1.3. Sequence Diagram

A Sequence Diagram is a major element of Unified Modeling Language (UML) that is applied to represent the interaction between objects in the sequence. It is concerned with the communication of things with one another overtime and thus is a critical tool of modeling dynamic behavior in a system.

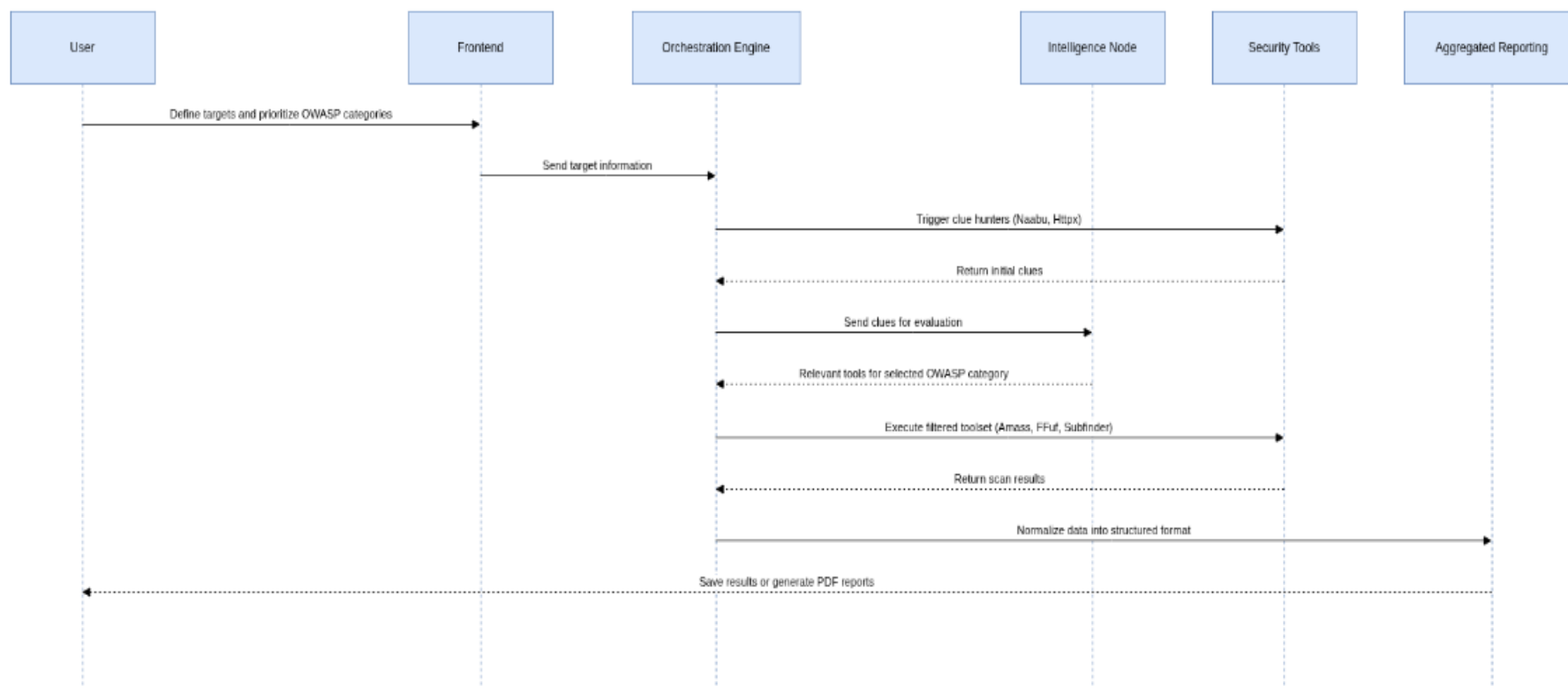


Figure 50 Sequence Diagram of Intelligence Recon System

This is a sequence diagram, which illustrates the way various factors that make up the Intelligence Recon System will interrelate with each other over time to process a request of a security scan. The user enters targets and OWASP categories via the Frontend that forwards these details to the Orchestration Engine that makes calls to the Intelligence Node and the Security Tools to conduct appropriate scans and provide results. Lastly, Aggregated Reporting component transforms the obtained results into a normalized format and transmits them to the Frontend to allow the user to see them or produce a PDF document.

[Click here](#) to go back.

8.1.4. Use Case Diagram

The Unified Modeling Language (UML) is a visual representation known as a Use Case Diagram which shows the interactions between users (actors) and a system (geeksforgeeks.org, 2025). The diagram below illustrates how the system will be used by the security analyst.

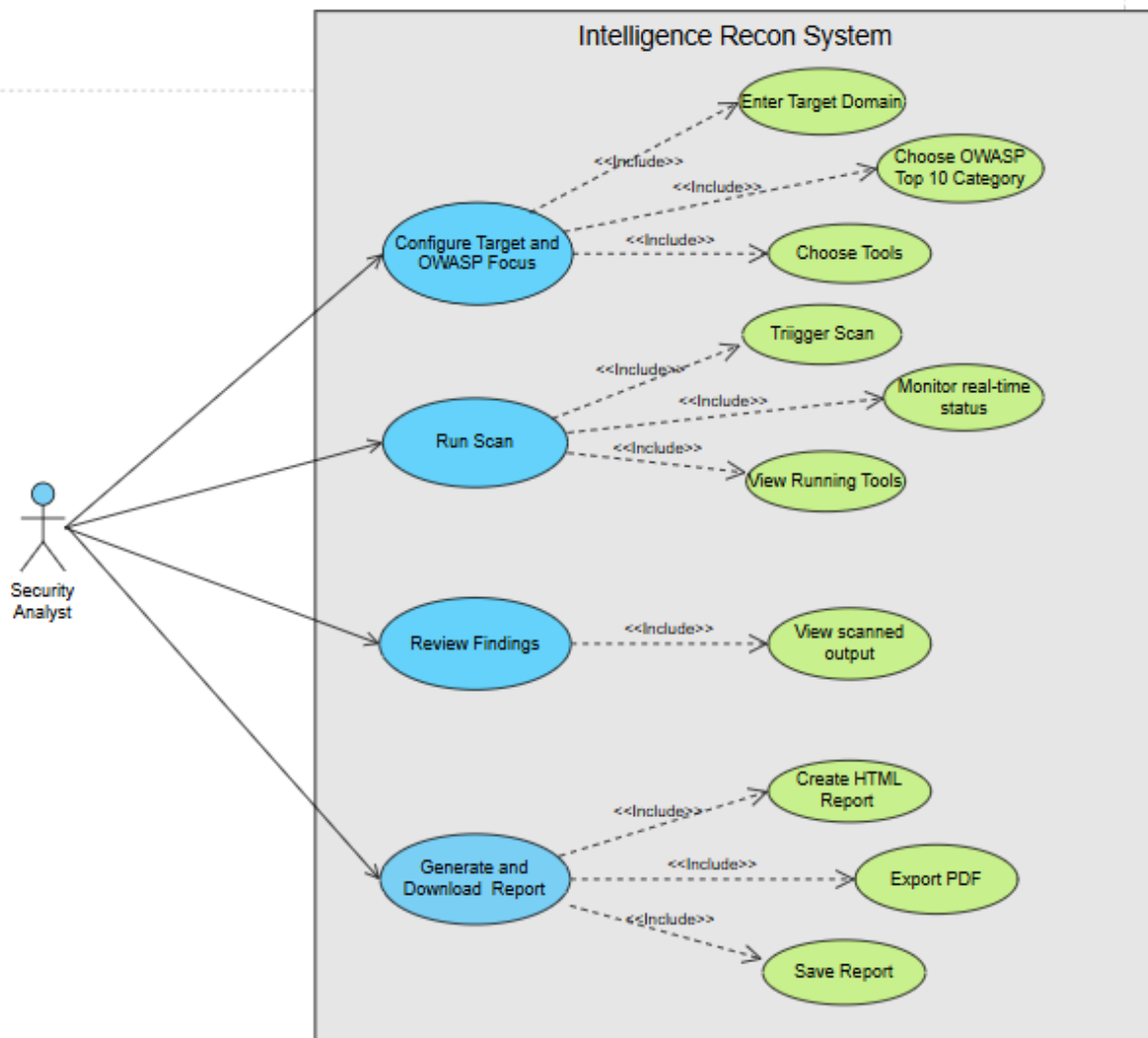


Figure 51 Use Case Diagram in a scenario of Security Analyst

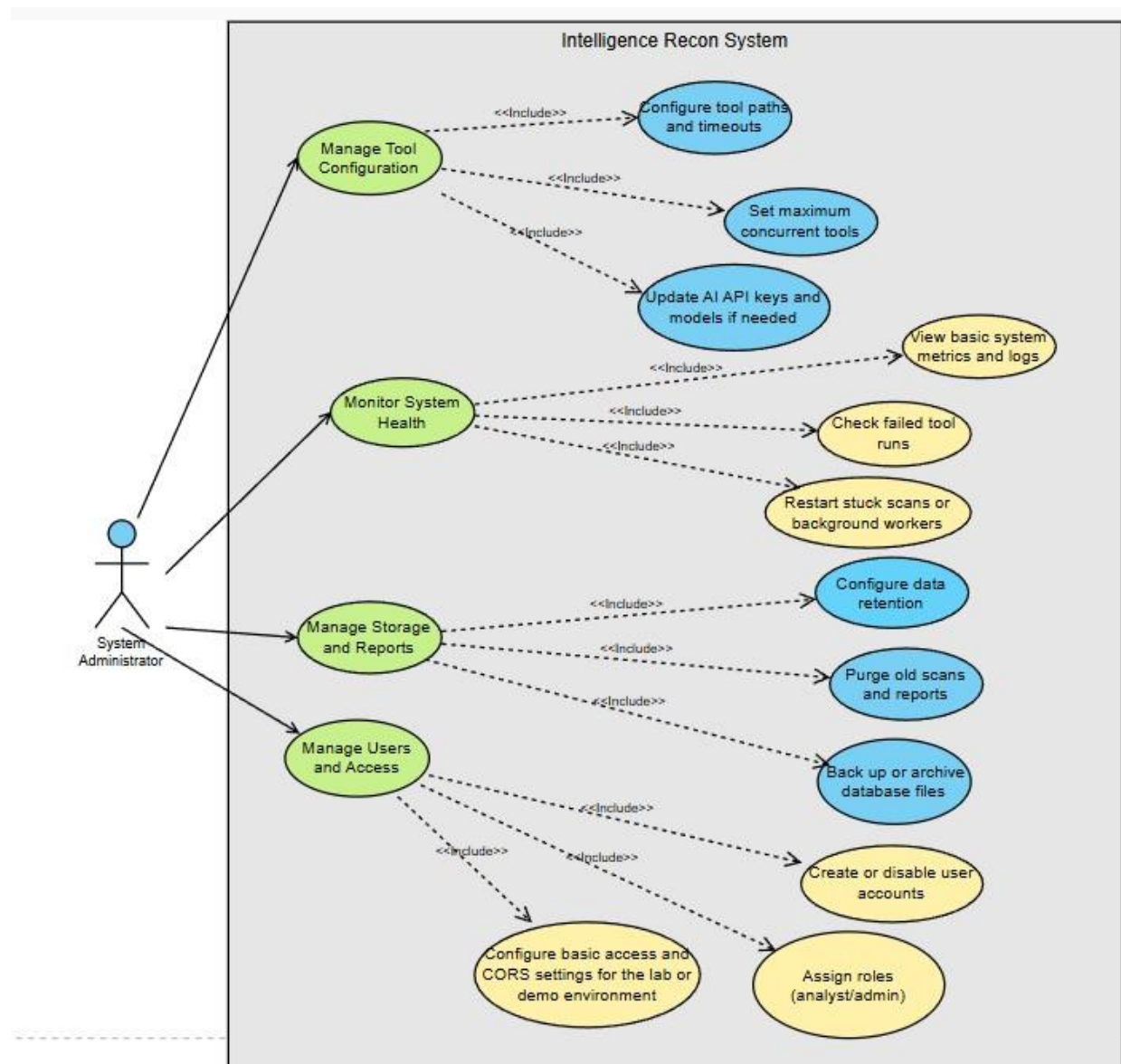
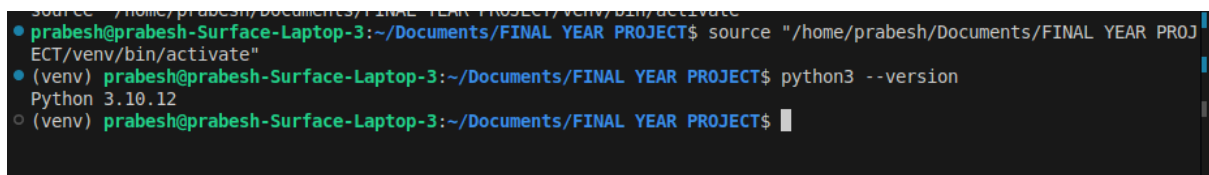


Figure 52 Use Case Diagram in a scenario of Security Administrator

8.2. Appendix 12: Development Progress

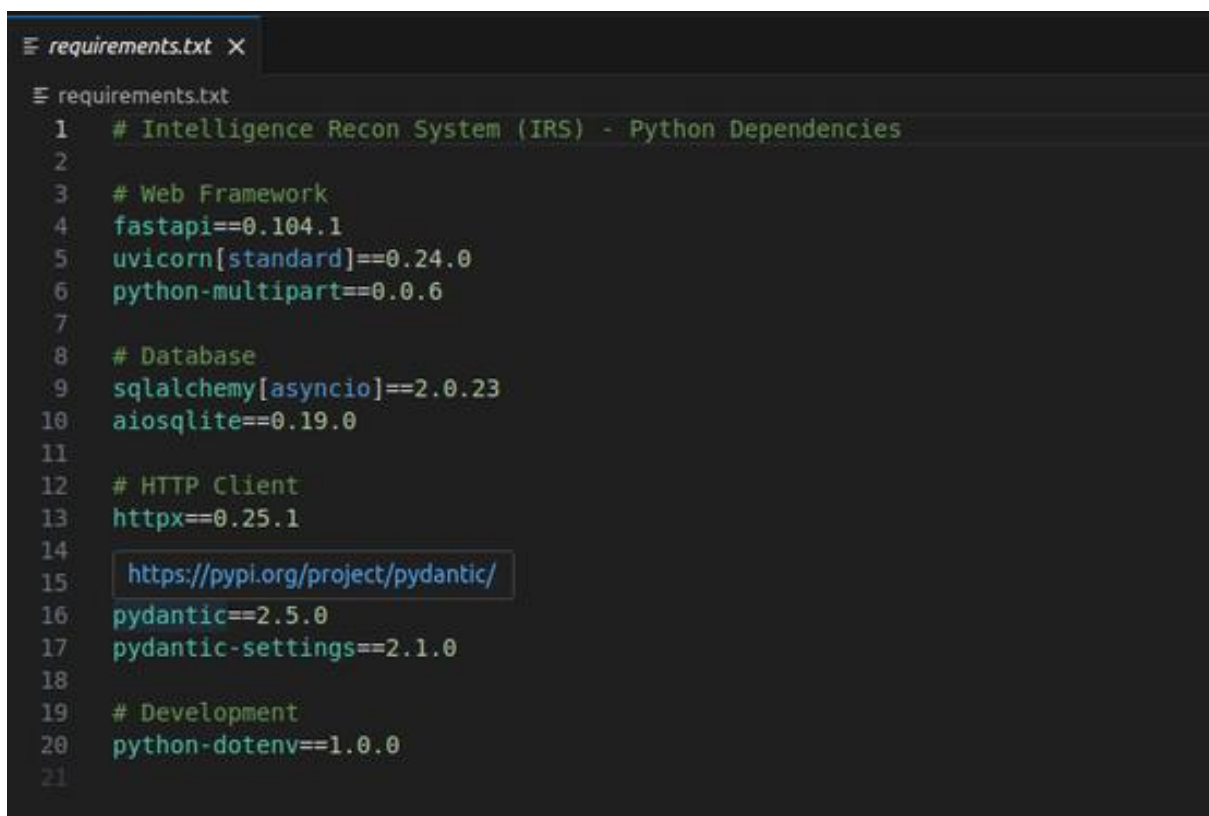
1. Sprint 1: Initial Setup and Tools Installation

Sprint 1 aimed at establishing the base development environment of the Intelligence Recon System, Python, and Fast API, as well as project setup. It was also the installation and verification of all necessary reconnaissance and scanning tools on the target machine to guarantee that they execute appropriately via the command line. At the close of this sprint, the project had a working skeleton of the backend and verified external tools, and it was ready to be further integrated in subsequent sprints.



```
source "/home/prabesh/Documents/FINAL YEAR PROJECT/venv/bin/activate"
prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ python3 --version
Python 3.10.12
prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$
```

Figure 53 Activating the project environment and checking the Python version



```
requirements.txt
# Intelligence Recon System (IRS) - Python Dependencies
# Web Framework
fastapi==0.104.1
uvicorn[standard]==0.24.0
python-multipart==0.0.6
# Database
sqlalchemy[asyncio]==2.0.23
aiosqlite==0.19.0
# HTTP Client
httpx==0.25.1
https://pypi.org/project/pydantic/
pydantic==2.5.0
pydantic-settings==2.1.0
# Development
python-dotenv==1.0.0
```

Figure 54 IRS Python Dependencies

```

(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ pip install -r requirements.txt
Defaulting to user installation because normal site-packages is not writeable
Collecting fastapi==0.104.1 (from -r requirements.txt (line 4))
  Using cached fastapi-0.104.1-py3-none-any.whl.metadata (24 kB)
Collecting uvicorn==0.24.0 (from uvicorn[standard]==0.24.0->-r requirements.txt (line 5))
  Using cached uvicorn-0.24.0-py3-none-any.whl.metadata (6.4 kB)
Collecting python-multipart==0.0.6 (from -r requirements.txt (line 6))
  Using cached python-multipart-0.0.6-py3-none-any.whl.metadata (2.5 kB)
Collecting sqlalchemy==2.0.23 (from sqlalchemy[asyncio]==2.0.23->-r requirements.txt (line 9))
  Using cached SQLAlchemy-2.0.23-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (9.6 kB)
Collecting aiosqlite==0.19.0 (from -r requirements.txt (line 10))
  Using cached aiosqlite-0.19.0-py3-none-any.whl.metadata (4.3 kB)
Collecting httpx==0.25.1 (from -r requirements.txt (line 13))
  Using cached httpx-0.25.1-py3-none-any.whl.metadata (7.1 kB)
Collecting pydantic==2.5.0 (from -r requirements.txt (line 16))
  Using cached pydantic-2.5.0-py3-none-any.whl.metadata (174 kB)
Collecting pydantic-settings==2.1.0 (from -r requirements.txt (line 17))
  Using cached pydantic-settings-2.1.0-py3-none-any.whl.metadata (2.9 kB)
Collecting python-dotenv==1.0.0 (from -r requirements.txt (line 20))
  Using cached python-dotenv-1.0.0-py3-none-any.whl.metadata (21 kB)
Collecting anyio==4.0.0 (from fastapi==0.104.1->-r requirements.txt (line 4))
  Using cached anyio-4.0.0-py3-none-any.whl.metadata (4.7 kB)
Collecting starlette==0.28.0 (from fastapi==0.104.1->-r requirements.txt (line 4))
  Using cached starlette-0.27.0-py3-none-any.whl.metadata (5.8 kB)
Requirement already satisfied: typing-extensions==4.8.0 in /home/prabesh/.local/lib/python3.10/site-packages (from fastapi==0.104.1->-r requirements.txt (line 4)) (4.8.0)
Requirement already satisfied: annotated-types==0.4.0 in /home/prabesh/.local/lib/python3.10/site-packages (from pydantic==2.5.0->-r requirements.txt (line 16)) (0.7.0)
Collecting pydantic-core==2.14.1 (from pydantic==2.5.0->-r requirements.txt (line 16))
  Using cached pydantic_core-2.14.1-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (6.5 kB)
Requirement already satisfied: click==7.0 in /usr/local/lib/python3.10/dist-packages (from uvicorn==0.24.0->uvicorn[standard]==0.24.0->-r requirements.txt (line 5)) (8.2.1)
Requirement already satisfied: h11==0.8 in /home/prabesh/.local/lib/python3.10/site-packages (from uvicorn==0.24.0->uvicorn[standard]==0.24.0->-r requirements.txt (line 5)) (0.16.0)
Requirement already satisfied: greenlet==0.4.17 in /home/prabesh/.local/lib/python3.10/site-packages (from sqlalchemy==2.0.23->sqlalchemy[asyncio]==2.0.23->-r requirements.txt (line 9)) (3.3.0)
Requirement already satisfied: certifi in /usr/lib/python3/dist-packages (from httpx==0.25.1->-r requirements.txt (line 13)) (2020.6.20)
Requirement already satisfied: httpcore in /home/prabesh/.local/lib/python3.10/site-packages (from httpx==0.25.1->-r requirements.txt (line 13)) (1.0.9)
Requirement already satisfied: idna in /usr/lib/python3/dist-packages (from httpx==0.25.1->-r requirements.txt (line 13)) (3.3)
Requirement already satisfied: sniffio in /home/prabesh/.local/lib/python3.10/site-packages (from httpx==0.25.1->-r requirements.txt (line 13)) (1.3.1)
Requirement already satisfied: httptools==0.5.0 in /home/prabesh/.local/lib/python3.10/site-packages (from uvicorn[standard]==0.24.0->-r requirements.txt (line 5)) (0.7.1)
Requirement already satisfied: pyyaml==5.1 in /home/prabesh/.local/lib/python3.10/site-packages (from uvicorn[standard]==0.24.0->-r requirements.txt (line 5)) (6.0.3)
Requirement already satisfied: uvloop==0.15.0, <0.15.1, >=0.14.0 in /home/prabesh/.local/lib/python3.10/site-packages (from uvicorn[standard]==0.24.0->-r requirements.txt (line 5)) (0.22.1)
Requirement already satisfied: watchfiles==0.13 in /home/prabesh/.local/lib/python3.10/site-packages (from uvicorn[standard]==0.24.0->-r requirements.txt (line 5)) (1.1.1)
Requirement already satisfied: websockets==10.4 in /home/prabesh/.local/lib/python3.10/site-packages (from uvicorn[standard]==0.24.0->-r requirements.txt (line 5)) (15.8.1)
Requirement already satisfied: exceptiongroup in /home/prabesh/.local/lib/python3.10/site-packages (from anyio==4.0.0->fastapi==0.104.1->-r requirements.txt (line 4)) (1.3.1)
Downloading fastapi-0.104.1-py3-none-any.whl (92 kB)
Downloading pydantic-2.5.0-py3-none-any.whl (407 kB)
Downloading uvicorn-0.24.0-py3-none-any.whl (59 kB)
Downloading python-multipart-0.0.6-py3-none-any.whl (45 kB)
Downloading SQLAlchemy-2.0.23-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (3.0 MB)
----- 3.0/3.0 MB 150.8 MB/s 0:00:20
Downloading aiosqlite-0.19.0-py3-none-any.whl (15 kB)

```

Figure 55 Installing Python dependencies

```

(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$
(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$
(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$
(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ uvicorn app.main:app --reload
INFO: Will watch for changes in these directories: ['/home/prabesh/Documents/FINAL YEAR PROJECT']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [31123] using WatchFiles
INFO: Started server process [31126]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:30468 - "GET / HTTP/1.1" 200 OK

```

Figure 56 Starting the IRS FAST API with Uvicorn

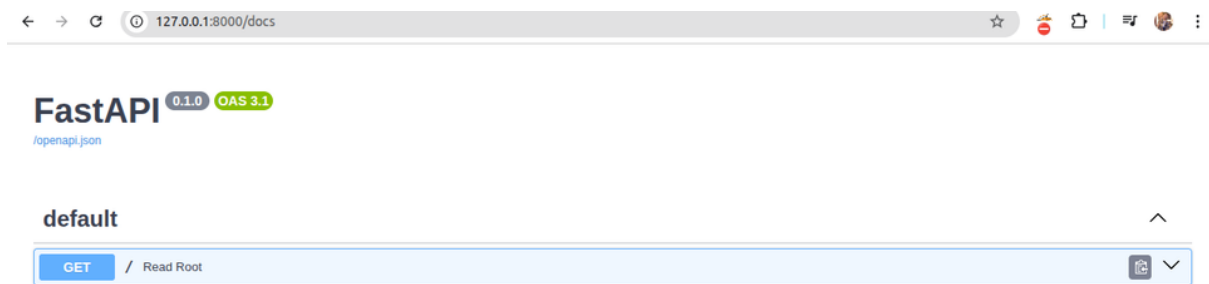


Figure 57 Fast API Swagger page showing the root endpoint

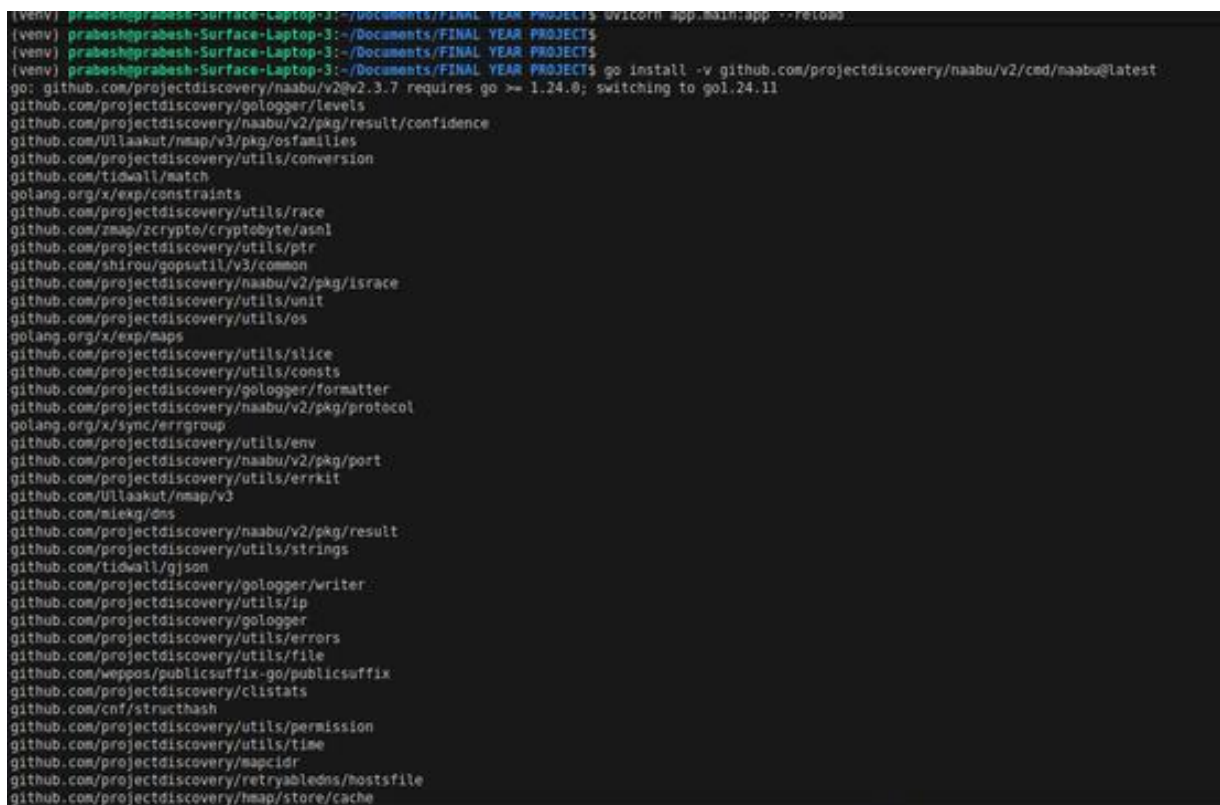


Figure 58 Installing Nabbu tool with its go modules


```
(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECTS$ go install -v github.com/projectdiscovery/naabu/v2/cmd/naabu@latest
github.com/projectdiscovery/naabu/v2/internal/pdcp
(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECTS$ naabu -h
Naabu is a port scanning tool written in Go that allows you to enumerate open ports for hosts in a fast and reliable manner.

Usage:
  naabu [flags]

Flags:
INPUT:
  -host string[]      hosts to scan ports for (comma-separated)
  -list, -l string    list of hosts to scan ports (file)
  -exclude-hosts, -eh string  hosts to exclude from the scan (comma-separated)
  -exclude-file, -ef string  list of hosts to exclude from scan (file)

PORT:
  -port, -p string      ports to scan (80,443, 100-200)
  -top-ports, -tp string  top ports to scan (default 100) [full,100,1000]
  -exclude-ports, -ep string[]  ports to exclude from scan (file or comma-separated)
  -ports-file, -pf string[]  list of ports to scan (file or comma-separated)
  -port-threshold, -pts int  port threshold to skip port scan for the host
  -exclude-cdn, -ec        skip full port scans for CDN/WAF (only scan for port 80,443)
  -display-cdn, -cdn       display cdn in use

RATE-LIMIT:
  -c int    general internal worker threads (default 25)
  -rate int  packets to send per second (default 1000)

UPDATE:
  -up, -update        update naabu to latest version
  -duc, -disable-update-check  disable automatic naabu update check

OUTPUT:
  -o, -output string      file to write output to (optional)
  -lor, -list-output-fields  list of fields to output (comma separated)
  -eot, -exclude-output-fields string[]  exclude output fields output based on a condition
  -j, -json              write output in JSON lines format
  -csv                  write output in csv format

CONFIGURATION:
  -config string      path to the naabu configuration file (default $HOME/.config/naabu/config.yaml)
  -scan-all-ips, -sa  scan all the IP's associated with DNS record
  -ip-version, -iv string[]  ip version to scan of hostname (4,6) - (default 4) (default ['4'])
  -scan-type, -s string      type of port scan (SYN/CONNECT) (default 'c')
  -source-ip string          source ip and port to use for scan (default '0.0.0.0:8080')
```

Figure 59 Verify Successful Installation

```
(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECTS$ go install -v github.com/projectdiscovery/nuclei/v3/cmd/nuclei@latest
go: downloading github.com/projectdiscovery/nuclei/v3 v3.6.1
go: github.com/projectdiscovery/nuclei/v3@v3.6.1 requires go >= 1.24.2; switching to go1.24.11
go: downloading github.com/projectdiscovery/gologger v1.1.64
go: downloading github.com/projectdiscovery/retryablehttp-go v1.1.1
go: downloading github.com/projectdiscovery/dsl v0.8.8
go: downloading github.com/projectdiscovery/fastdialer v0.4.20
go: downloading github.com/projectdiscovery/wappalizergo v0.2.59
go: downloading github.com/projectdiscovery/networkpolicy v0.1.32
go: downloading github.com/projectdiscovery/cdncheck v1.2.14
go: downloading github.com/projectdiscovery/retryabledns v1.0.111
go: downloading github.com/klauspost/compress v1.18.2
go: downloading github.com/minio/miniz v1.0.1
go: downloading github.com/pierrec/lz4/v4 v4.1.22
go: downloading github.com/spf13/afero v1.15.0
go: downloading golang.org/x/mod v0.29.0
go: downloading github.com/ericlagergren/decimal v0.0.0-20240411145413-00de7ca16731
go: downloading github.com/felixge/httpsnoop v1.0.4
go: downloading go.opentelemetry.io/otel/metric v1.38.0
go: downloading github.com/go-pg/zerochecker v0.2.0
go: downloading github.com/vmihailenco/msgpack/v5 v5.3.4
go: downloading github.com/vmihailenco/tagparser v0.1.2
go: downloading github.com/tmthrgd/go-hex v0.0.0-20190904060850-447a3041c3bc
go: downloading github.com/vmihailenco/bufpool v0.1.11
go: downloading github.com/gogo/protobuf v1.3.2
go: downloading github.com/go-logr/logr v1.4.3
go: downloading github.com/vmihailenco/tagparser/v2 v2.0.0
go: downloading github.com/go-logr/stdr v1.2.2
go: downloading go.opentelemetry.io/auto/sdk v1.1.0
go: downloading google.golang.org/protobuf v1.36.6
go: downloading github.com/projectdiscovery/gologger/levels
go: downloading github.com/klauspost/compress/internal/le
go: downloading github.com/pierrec/lz4/v4/internal/lz4errors
go: downloading github.com/klauspost/compress/internal/cpuinfo
go: downloading github.com/klauspost/compress/internal/race
go: downloading github.com/minio/miniz/internal/race
go: downloading github.com/klauspost/compress
go: downloading github.com/charmbracelet/x/ansi/parser
go: downloading github.com/andybalholm/brotli/matchfinder
go: downloading github.com/STARRY-S/zip
go: downloading github.com/bodgit/seamless/internal/otf
```

Figure 60 Installation of nuclei scanner and go

```
(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ nuclei -h
-tid, -team-id string      upload scan results to given team id (optional) (default "none")
-cup, -cloud-upload        upload scan results to pdcp dashboard [DEPRECATED use -dashboard]
-sid, -scan-id string      upload scan results to existing scan id (optional)
-sname, -scan-name string  scan name to set (optional)
-pd, -dashboard            upload / view nuclei results in projectdiscovery cloud (pdc) UI dashboard
-pdu, -dashboard-upload string upload / view nuclei results file (jsonl) in projectdiscovery cloud (pdc) UI dashboard

AUTHENTICATION:
-sf, -secret-file string[] path to config file containing secrets for nuclei authenticated scan
-ps, -prefetch-secrets    prefetch secrets from the secrets file

EXAMPLES:
Run nuclei on single host:
$ nuclei -target example.com

Run nuclei with specific template directories:
$ nuclei -target example.com -t http/cves/ -t ssl

Run nuclei against a list of hosts:
$ nuclei -list hosts.txt

Run nuclei with a JSON output:
$ nuclei -target example.com -json-export output.json

Run nuclei with sorted Markdown outputs (with environment variables):
$ MARKDOWN_EXPORT_SORT_MODE=template nuclei -target example.com -markdown-export nuclei_report/

Additional documentation is available at: https://docs.nuclei.sh/getting-started/running
(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$
```

Figure 61 Verified installation of nuclei

```
(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ go install -v github.com/projectdiscovery/httpx/cmd/httpx@latest
go: github.com/projectdiscovery/httpx@v1.7.4 requires go >= 1.24.1; switching to go1.24.11
(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ httpx --help
HTTPX 🐼

A next generation HTTP client.

Usage: httpx <URL> [OPTIONS]

-m, --method METHOD      Request method, such as GET, POST, PUT, PATCH, DELETE, OPTIONS, HEAD.
                        [Default: GET, or POST if a request body is included]

-p, --params <NAME VALUE> ... Query parameters to include in the request URL.

-c, --content TEXT      Byte content to include in the request body.

-d, --data <NAME VALUE> ... Form data to include in the request body.

-f, --files <NAME FILENAME> ... Form files to include in the request body.

-j, --json TEXT          JSON data to include in the request body.

-h, --headers <NAME VALUE> ... Include additional HTTP headers in the request.

--cookies <NAME VALUE> ... Cookies to include in the request.

--auth <USER PASS>      Username and password to include in the request. Specify '-' for the password to use a password prompt.
                        Note that using --verbose/-v will expose the Authorization header, including the password encoding in a
                        trivially reversible format.

--proxies URL            Send the request via a proxy. Should be the URL giving the proxy address.

--timeout FLOAT          Timeout value to use for network operations, such as establishing the connection, reading some data,
                        etc... [Default: 5.0]

--follow-redirects       Automatically follow redirects.

--no-verify              Disable SSL verification.

--http2                 Send the request using HTTP/2, if the remote server supports it.

--download FILE          Save the response content as a file, rather than displaying it.
```

Figure 62 Install httpx and verified


```

* (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL_YEAR_PROJECT$ go install -v github.com/projectdiscovery/subfinder/v2/cmd/subfinder@latest
go: downloading github.com/projectdiscovery/subfinder/v2 v2.11.0
go: github.com/projectdiscovery/subfinder/v2@v2.11.0 requires go >= 1.24.0; switching to go1.24.11
github.com/projectdiscovery/gologger/levels
github.com/refraction-networking/utls/internal/fips140tls
github.com/refraction-networking/utls/internal/byteorder
github.com/projectdiscovery/utls/conversion
github.com/refraction-networking/utls/dicttls
github.com/gaissmai/bart/internal/art
github.com/zmap/zcrypto/cryptobyte/asn1
github.com/projectdiscovery/utls/ptr
github.com/projectdiscovery/utls/race
github.com/projectdiscovery/gologger/formatter
github.com/projectdiscovery/utls/errors
github.com/hako/durafmt
github.com/projectdiscovery/utls/os
github.com/refraction-networking/utls/internal/boring
github.com/refraction-networking/utls/internal/helper
github.com/refraction-networking/utls/internal/hkdf
github.com/projectdiscovery/gologger/writer
github.com/refraction-networking/utls/internal/quickvarint/protocol
github.com/refraction-networking/utls/internal/hpke
github.com/refraction-networking/utls/internal/tls12
github.com/refraction-networking/utls/internal/tls13
github.com/refraction-networking/utls/internal/quickvarint
github.com/projectdiscovery/ratelimit
github.com/projectdiscovery/hmap/store/cache
github.com/projectdiscovery/utls/slice
github.com/projectdiscovery/hmap/store/disk
github.com/projectdiscovery/utls/strings
github.com/projectdiscovery/utls/consts
github.com/refraction-networking/utls
github.com/projectdiscovery/utls/env
github.com/projectdiscovery/retryabledns/doh
github.com/gaissmai/bart/internal/bitset
github.com/projectdiscovery/gologger
github.com/projectdiscovery/utls/maps
github.com/projectdiscovery/utls/errkit
github.com/gaissmai/bart/internal/allot
github.com/projectdiscovery/utls/file
github.com/projectdiscovery/utls/ip
github.com/gaissmai/bart/internal/lpm
github.com/gaissmai/bart/internal/sparse

```

Figure 63 Install Subfinder

```

(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL_YEAR_PROJECT$ go install -v github.com/projectdiscovery/subfinder/v2/cmd/subfinder@latest
* (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL_YEAR_PROJECT$ subfinder -h
Subfinder is a subdomain discovery tool that discovers subdomains for websites by using passive online sources.

Usage:
  subfinder [flags]

Flags:
INPUT:
  -d, -domain string[]  domains to find subdomains for
  -dL, -list string      file containing list of domains for subdomain discovery

SOURCE:
  -s, -sources string[]  specific sources to use for discovery (-s crtsh,github). Use -ls to display all available sources.
  -recursive             use only sources that can handle subdomains recursively rather than both recursive and non-recursive sources
  -all                  use all sources for enumeration (slow)
  -es, -exclude-sources string[]  sources to exclude from enumeration (-es alienvault,zoomeyeapi)

FILTER:
  -m, -match string[]    subdomain or list of subdomain to match (file or comma separated)
  -f, -filter string[]   subdomain or list of subdomain to filter (file or comma separated)

RATE-LIMIT:
  -rl, -rate-limit int    maximum number of http requests to send per second (global)
  -rls, -rate-limits value  maximum number of http requests to send per second for providers in key-value format (-rls hackertarget=10/m) (default ["g
=60/m", "pugrecon=10/s", "robtex=18446744073709551615/ms", "securitytrails=1/s", "shodan=1/s", "virustotal=4/m", "hackertarget=2/s", "waybackarchive=15
", "securitytrails=2/s", "sitedossier=8/m", "netlas=1/s", "github=83/m", "hudsonrock=3/s"])
  -t int                 number of concurrent goroutines for resolving (-active only) (default 10)

UPDATE:
  -up, -update           update subfinder to latest version
  -duc, -disable-update-check  disable automatic subfinder update check

OUTPUT:
  -o, -output string      file to write output to
  -oj, -json              write output in JSONL(lines) format
  -od, -output-dir string  directory to write output (-dL only)
  -cs, -collect-sources   include all sources in the output (-json only)
  -oi, -ip               include host IP in output (-active only)

CONFIGURATION:
  -config string          flag config file (default "/home/prabesh/.config/subfinder/config.yaml")
  -pc, -provider-config string  provider config file (default "/home/prabesh/.config/subfinder/provider-config.yaml")
  -r string[]             comma separated list of resolvers to use

```

Figure 64 Verified installation of subfinder

```

• (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECTS go install -v github.com/tomonomon/assetfinder@latest
internal/coverage/rtcov
internal/codebugs
internal/unsafeheader
internal/goarch
internal/goexperiment
internal/byteorder
internal/goos
internal/profile
runtime/internal/math
internal/abi
internal/cpu
internal/runtime/atomic
internal/runtime/syscall
runtime/internal/sys
internal/race
internal/chacha8rand
sync/atomic
unicode
cap
unicode/utf8
encoding
math/bits
internal/bytealg
internal/itoa
internal/asan
internal/runtime/exithook
internal/msan
unicode/utf16
container/list
crypto/internal/alias
crypto/subtle
math
crypto/internal/boring/sig
vendor/golang.org/x/crypto/cryptobyte/asn1
vendor/golang.org/x/crypto/internal/alias
internal/nettrace
log/internal
internal/stringlite
runtime
iter
internal/weak
internal/reflectlite

```

Figure 65 Install Assetfinder with go and fetching dependencies

```

mime/multipart
crypto/tls
net/http/httptrace
net/http
• (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ assetfinder -h
Usage of assetfinder:
  -subs-only
            Only include subdomains of search domain
• (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$

```

Figure 66 Verified installation of assetfinder

[illegible]

Figure 67 Install Amass

```
(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ git clone https://github.com/aboul3la/Sublist3r.git
remote: Total 383 (delta 0), reused 0 (delta 0), pack-reused 383 (from 1)
Receiving objects: 100% (383/383), 1.12 MiB | 560.00 KiB/s, done.
Resolving deltas: 100% (213/213), done.
* (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ python3 sublist3r.py -h
python3: can't open file '/home/prabesh/Documents/FINAL YEAR PROJECT/sublist3r.py': [Errno 2] No such file or directory
* (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ git clone https://github.com/aboul3la/Sublist3r.git
fatal: destination path 'Sublist3r' already exists and is not an empty directory.
* (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ cd ~/Sublist3r/
* (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT/Sublist3r$ pip install -r requirements.txt
Defaulting to user installation because normal site-packages is not writeable
Collecting argparse (from -r requirements.txt (line 1))
  Downloading argparse-1.4.0-py2.py3-none-any.whl.metadata (2.8 kB)
Requirement already satisfied: dnspython in /usr/lib/python3/dist-packages (from -r requirements.txt (line 2)) (2.1.0)
Requirement already satisfied: requests in /home/prabesh/.local/lib/python3.10/site-packages (from -r requirements.txt (line 3)) (2.32.5)
Requirement already satisfied: charset-normalizer<4,>=2 in /home/prabesh/.local/lib/python3.10/site-packages (from requests->-r requirements.txt (line 3)) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/lib/python3/dist-packages (from requests->-r requirements.txt (line 3)) (3.3)
Requirement already satisfied: urllib3<3,>=1.21.1 in /home/prabesh/.local/lib/python3.10/site-packages (from requests->-r requirements.txt (line 3)) (2.5.0)
Requirement already satisfied: certifi<2017.4.17 in /usr/lib/python3/dist-packages (from requests->-r requirements.txt (line 3)) (2020.6.20)
Downloading argparse-1.4.0-py2.py3-none-any.whl (23 kB)
Installing collected packages: argparse
Successfully installed argparse-1.4.0
* (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT/Sublist3r$ python3 sublist3r.py -h
usage: sublist3r.py [-h] -d DOMAIN [-b [BRUTEFORCE]] [-p PORTS] [-v [VERBOSE]] [-t THREADS] [-e ENGINES] [-o OUTPUT] [-n]

OPTIONS:
  -h, --help            show this help message and exit
  -d DOMAIN, --domain DOMAIN
                        Domain name to enumerate it's subdomains
  -b [BRUTEFORCE], --bruteforce [BRUTEFORCE]
                        Enable the subbrute bruteforce module
  -p PORTS, --ports PORTS
                        Scan the found subdomains against specified tcp ports
  -v [VERBOSE], --verbose [VERBOSE]
                        Enable Verbosity and display results in realtime
  -t THREADS, --threads THREADS
                        Number of threads to use for subbrute bruteforce
  -e ENGINES, --engines ENGINES
                        Specify a comma-separated list of search engines
  -o OUTPUT, --output OUTPUT
                        Save the results to text file
  -n, --no-color         Output without color

Example: python3 sublist3r.py -d example.com -b -p 80,443 -v -t 10 -e google,bing -o output.txt -n
* (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT/Sublist3r$
```

Figure 68 Cloning Sublist3r, installing requirements and verified installation

```
(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT/Sublist3r$
* (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT/Sublist3r$
* (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT/Sublist3r$ pip install ctl
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: ctl in /home/prabesh/.local/lib/python3.10/site-packages (1.2.0)
Requirement already satisfied: GitPython>=3.1.36 in /home/prabesh/.local/lib/python3.10/site-packages (from ctl) (3.1.45)
Requirement already satisfied: confu==1.4 in /home/prabesh/.local/lib/python3.10/site-packages (from ctl) (1.9.0)
Requirement already satisfied: grainy==1.4 in /home/prabesh/.local/lib/python3.10/site-packages (from ctl) (1.8.1)
Requirement already satisfied: munge==1 in /home/prabesh/.local/lib/python3.10/site-packages (from ctl) (1.4.0)
Requirement already satisfied: natsort==8 in /usr/lib/python3/dist-packages (from ctl) (8.0.2)
Requirement already satisfied: ogra==0.46.0 in /home/prabesh/.local/lib/python3.10/site-packages (from ctl) (0.57.0)
Requirement already satisfied: pluginmgr==1 in /home/prabesh/.local/lib/python3.10/site-packages (from ctl) (1.2.1)
Requirement already satisfied: pydantic==2.3.0 in /home/prabesh/.local/lib/python3.10/site-packages (from ctl) (2.5.0)
Requirement already satisfied: semver==3 in /home/prabesh/.local/lib/python3.10/site-packages (from ctl) (3.0.0)
Requirement already satisfied: charset-normalizer==3.3.0 in /home/prabesh/.local/lib/python3.10/site-packages (from munge==1->ctl) (3.4.4)
Requirement already satisfied: click==8.0.1 in /usr/lib/python3/dist-packages (from munge==1->ctl) (8.2.1)
Requirement already satisfied: pyyaml==6.0.1 in /home/prabesh/.local/lib/python3.10/site-packages (from munge==1->ctl) (6.0.3)
Requirement already satisfied: requests==2.32.0 in /home/prabesh/.local/lib/python3.10/site-packages (from munge==1->ctl) (2.32.5)
Requirement already satisfied: toml==0.10.2 in /usr/lib/python3/dist-packages (from munge==1->ctl) (0.10.2)
Requirement already satisfied: tomlkit==0.11.6 in /home/prabesh/.local/lib/python3.10/site-packages (from munge==1->ctl) (0.13.3)
Requirement already satisfied: urllib3==2.0.0 in /home/prabesh/.local/lib/python3.10/site-packages (from munge==1->ctl) (2.5.0)
Requirement already satisfied: gitdb<3,>=4.0.1 in /home/prabesh/.local/lib/python3.10/site-packages (from GitPython>=3.1.36->ctl) (4.0.12)
Requirement already satisfied: smmap<8,>=3.0.1 in /home/prabesh/.local/lib/python3.10/site-packages (from gitdb<3,>=4.0.1->GitPython>=3.1.36->ctl) (5.0.2)
Requirement already satisfied: cryptography in /usr/lib/python3/dist-packages (from ogra==0.46.0->ctl) (44.0.3)
Requirement already satisfied: deprecated in /home/prabesh/.local/lib/python3.10/site-packages (from ogra==0.46.0->ctl) (1.3.1)
Requirement already satisfied: https in /home/prabesh/.local/lib/python3.10/site-packages (from ogra==0.46.0->ctl) (0.25.1)
Requirement already satisfied: pyforgejo==2.0.0 in /home/prabesh/.local/lib/python3.10/site-packages (from ogra==0.46.0->ctl) (2.0.5)
Requirement already satisfied: pygithub in /home/prabesh/.local/lib/python3.10/site-packages (from ogra==0.46.0->ctl) (2.8.1)
Requirement already satisfied: python-gitlab in /home/prabesh/.local/lib/python3.10/site-packages (from ogra==0.46.0->ctl) (7.0.0)
Requirement already satisfied: importlib-metadata==1 in /usr/lib/python3/dist-packages (from pluginmgr==1->ctl) (4.6.4)
Requirement already satisfied: annotated-types==0.4.0 in /home/prabesh/.local/lib/python3.10/site-packages (from pydantic==2.3.0->ctl) (0.7.0)
Requirement already satisfied: pydantic-core==2.14.1 in /home/prabesh/.local/lib/python3.10/site-packages (from pydantic==2.3.0->ctl) (2.14.1)
Requirement already satisfied: typing-extensions==4.6.1 in /home/prabesh/.local/lib/python3.10/site-packages (from pydantic==2.3.0->ctl) (4.15.0)
Collecting httpx (from ogra==0.46.0->ctl)
  Using cached httpx-0.28.1-py3-none-any.whl.metadata (7.1 kB)
Collecting pydantic==2.3.0 (from ctl)
  Using cached pydantic-2.12.5-py3-none-any.whl.metadata (90 kB)
Collecting python-dotenv==1.1.0 (from pyforgejo==2.0.0->ogra==0.46.0->ctl)
  Using cached python-dotenv-1.2.1-py3-none-any.whl.metadata (25 kB)
Requirement already satisfied: anyio in /home/prabesh/.local/lib/python3.10/site-packages (from httpx->ogra==0.46.0->ctl) (3.7.1)
Requirement already satisfied: certifi in /usr/lib/python3/dist-packages (from httpx->ogra==0.46.0->ctl) (2020.6.20)
Requirement already satisfied: httpcore==1.* in /home/prabesh/.local/lib/python3.10/site-packages (from httpx->ogra==0.46.0->ctl) (1.0.9)
Requirement already satisfied: idna in /usr/lib/python3/dist-packages (from httpx->ogra==0.46.0->ctl) (3.3)
Requirement already satisfied: h11==0.16 in /home/prabesh/.local/lib/python3.10/site-packages (from httpcore==1.*->httpx->ogra==0.46.0->ctl) (0.16.0)
Collecting pydantic-core==2.41.5 (from pydantic==2.3.0->ctl)
```

Figure 69 Install CTL

```

(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ ctl -h

usage: ctl [--debug] [--home HOME] [--verbose] [--quiet] [--version] {} ...

control

options:
  --debug      enable extra debug output
  --home HOME  specify the home directory, by default will check in order: $CTL_HOME, ./Ctl, /home/prabesh/.ctl
  --verbose    enable more verbose output
  --quiet      no output at all
  --version    show program's version number and exit

[ALL] Configured Operations:
{}

Plugins
  command      runs a command
  config        ctl config tool
(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$

```

Figure 70 Verified installation of ctl

```

(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ go install -v github.com/lc/gau/v2/cmd/gau@latest
github.com/valyala/bytebufferpool
hash/adler32
github.com/modern-go/reflect2
html
golang.org/x/text/transform
github.com/deckarep/golang-set/v2
runtime/debug
golang.org/x/sys/unix
github.com/klauspost/compress/flate
github.com/valyala/fasthttp/fasthttputil
github.com/valyala/fasthttp/stackless
github.com/andybalholm/brotli
golang.org/x/text/unicode/bidi
github.com/modern-go/concurrent
encoding/csv
golang.org/x/text/unicode/norm
github.com/lynxsecurity/viper/internal/encoding
github.com/hashicorp/hcl/hcl/strconv
github.com/lynxsecurity/pflag
github.com/hashicorp/hcl/hcl/token
github.com/hashicorp/hcl/hcl/ast
golang.org/x/text/secure/bidirule
regexp/syntax
github.com/hashicorp/hcl/json/token
github.com/hashicorp/hcl/json/scanner
github.com/hashicorp/hcl/json/parser
golang.org/x/net/idna
text/tabwriter
github.com/klauspost/compress/gzip
github.com/klauspost/compress/zlib
github.com/lynxsecurity/viper/internal/encoding/json
github.com/pelletier/go-toml
github.com/mitchellh/mapstructure
github.com/json-iterator/go
github.com/bobesa/go-domain-util/domainutil
regexp

```

Figure 71 Install GAU


```

github.com/TC/gau/v2/runner
• (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECTS$ gau -h
Usage of gau:
--blacklist strings    list of extensions to skip
--config string        location of config file (default $HOME/.gau.toml or %USERPROFILE%\gau.toml)
--fc strings           list of status codes to filter
--fp                  remove different parameters of the same endpoint
--from string          fetch urls from date (format: YYYYMM)
--ft strings           list of mime-types to filter
--json                output as json
--mc strings           list of status codes to match
--mt strings           list of mime-types to match
--o string             filename to write results to
--providers strings    list of providers to use (wayback,commoncrawl,otx,urlscan)
--proxy string         http proxy to use
--retries uint         retries for HTTP client
--subs               include subdomains of target domain
--threads uint        number of workers to spawn (default 1)
--timeout uint        timeout (in seconds) for HTTP client (default 45)
--to string           fetch urls to date (format: YYYYMM)
--verbose             show verbose output
--version             show gau version
○ (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECTS$

```

Figure 72 Verified installation of GAU

```

--version             show gau version
○ (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECTS$ go install -v github.com/projectdiscovery/katana/cmd/katana@latest
go: github.com/projectdiscovery/katana@v1.3.0 requires go >= 1.24.1; switching to go1.24.11
github.com/projectdiscovery/utls/conversion
github.com/projectdiscovery/utls/race
github.com/projectdiscovery/gologger/levels
github.com/projectdiscovery/katana/pkg/engine
github.com/go-rod/rod/lib/assets
github.com/go-rod/rod/lib/js
github.com/ysmood/leakless/pkg/shared
github.com/projectdiscovery/utls/ptr
github.com/projectdiscovery/utls/maps
github.com/projectdiscovery/utls/env
github.com/projectdiscovery/utls/os
github.com/projectdiscovery/utls/slice
github.com/projectdiscovery/utls/strings
github.com/projectdiscovery/utls/permission
github.com/projectdiscovery/utls/time
github.com/projectdiscovery/utls/errkit
github.com/projectdiscovery/gologger/formatter
github.com/projectdiscovery/gologger/writer
github.com/andybalholm/cascadia
github.com/go-rod/rod/lib/launcher/flags
github.com/go-rod/rod/lib/utls
github.com/ysmood/leakless/pkg/utls
github.com/ysmood/got/lib/lcs
github.com/go-rod/rod/lib/proto
github.com/projectdiscovery/fastdialer/fastdialer/ja3/impersonate
github.com/ysmood/leakless
github.com/projectdiscovery/utls/file
github.com/projectdiscovery/utls/maps
github.com/projectdiscovery/gologger
github.com/go-rod/rod/lib/defaults
github.com/projectdiscovery/utls/consts
github.com/projectdiscovery/utls/ip
github.com/go-rod/rod/lib/cdp
github.com/projectdiscovery/utls/context
github.com/projectdiscovery/utls/url
github.com/projectdiscovery/fastdialer/fastdialer/utls
github.com/zmap/zcrypto/x509
github.com/PuerkitoBio/goquery
github.com/smacker/go-tree-sitter
github.com/projectdiscovery/utls/slices

```

Figure 73 Install Katana

```

github.com/projectdiscovery/katana/internal/runner
• (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ katana -h
Katana is a fast crawler focused on execution in automation
pipelines offering both headless and non-headless crawling.

Usage:
  katana [flags]

Flags:
INPUT:
  -u, -list string[]  target url / list to crawl
  -resume string      resume scan using resume.cfg
  -e, -exclude string[] exclude host matching specified filter ('cdn', 'private-ips', 'cidr', 'ip', 'regex')

CONFIGURATION:
  -r, -resolvers string[]  list of custom resolver (file or comma separated)
  -d, -depth int           maximum depth to crawl (default 3)
  -jc, -js-crawl          enable endpoint parsing / crawling in javascript file
  -jsl, -jsluice          enable jsluice parsing in javascript file (memory intensive)
  -ct, -crawl-duration value maximum duration to crawl the target for (s, m, h, d) (default s)
  -kf, -known-files value  enable crawling of known files (all,robotstxt,sitemapxml), a minimum depth of 3 is required to ensu
  -mrs, -max-response-size int maximum response size to read (default 4194304)
  -timeout int            time to wait for request in seconds (default 10)
  -time-stable int        time to wait until the page is stable in seconds (default 1)
  -aff, -automatic-form-fill enable automatic form filling (experimental)
  -fx, -form-extraction   extract form, input, textarea & select elements in jsonl output
  -retry int              number of times to retry the request (default 1)
  -proxy string           http/socks5 proxy to use
  -td, -tech-detect       enable technology detection
  -H, -headers string[]   custom header/cookie to include in all http request in header:value format (file)
  -config string          path to the katana configuration file
  -fc, -form-config string path to custom form configuration file
  -flc, -field-config string path to custom field configuration file
  -s, -strategy string    Visit strategy (depth-first, breadth-first) (default "depth-first")
  -iqp, -ignore-query-params Ignore crawling same path with different query-param values
  -tlsi, -tls-impersonate enable experimental client hello (ja3) tls randomization
  -dr, -disable-redirects disable following redirects (default false)
  -pc, -path-climb        enable path climb (auto crawl parent paths)

DEBUG:
  -health-check, -hc      run diagnostic check up
  -elog, -error-log string file to write sent requests error log
  -pprof-server            enable pprof server
? master 0 0 0 AWS

```

Figure 74 Verified installation of katana

```

• (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ go install -v github.com/jaeles-project/gospider@latest
github.com/golang/groupcache/lru
golang.org/x/net/html/atom
golang.org/x/text/internal/utf8internal
golang.org/x/text/encoding/internal/identifier
github.com/gobwas/glob/util/runes
google.golang.org/protobuf/internal/flags
google.golang.org/protobuf/internal/set
golang.org/x/sys/internal/unsafeheader
internal/saferio
hash/fnv
golang.org/x/text/transform
golang.org/x/text/internal/tag
github.com/gobwas/glob/util/strings
github.com/saintfish/chardet
google.golang.org/protobuf/internal/pragma
encoding/gob
encoding/xml
golang.org/x/net/html
golang.org/x/text/internal/language
golang.org/x/text/encoding
golang.org/x/text/runes
github.com/antchfx/xpath
golang.org/x/text/encoding/internal
github.com/gobwas/glob/match
golang.org/x/text/encoding/charmap
golang.org/x/text/encoding/japanese
golang.org/x/text/encoding/korean
github.com/andybalholm/cascadia
golang.org/x/text/encoding/simplifiedchinese
golang.org/x/text/encoding/traditionalchinese
golang.org/x/text/encoding/unicode
golang.org/x/text/internal/language/compact
github.com/gobwas/glob/syntax/lexer
github.com/PuerkitoBio/goquery
github.com/gocolly/colly/v2/debug
github.com/gobwas/glob/syntax/ast
golang.org/x/text/language
github.com/gobwas/glob/compiler
github.com/gobwas/glob/syntax
net/http/cookiejar
? master 0 0 0 AWS

```

Figure 75 Install Gospider

```

github.com/jaetes-project/gospider/core
• (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ gospider -h
Fast web spider written in Go - v1.1.6 by @thebl4ckturtle & @j3ssiej3j

Usage:
  gospider [flags]

Flags:
  -s, --site string           Site to crawl
  -S, --sites string         Site list to crawl
  -p, --proxy string          Proxy (Ex: http://127.0.0.1:8080)
  -o, --output string         Output folder
  -u, --user-agent string     User Agent to use
                              web: random web user-agent
                              mobi: random mobile user-agent
                              or you can set your special user-agent (default "web")
  --cookie string             Cookie to use (testA=a; testB=b)
  -H, --header stringArray    Header to use (Use multiple flag to set multiple header)
  --burp string               Load headers and cookie from burp raw http request
  --blacklist string           Blacklist URL Regex
  --whitelist string           Whitelist URL Regex
  --whitelist-domain string    Whitelist Domain
  -L, --filter-length string   Turn on length filter
  -t, --threads int            Number of threads (Run sites in parallel) (default 1)
  -c, --concurrent int         The number of the maximum allowed concurrent requests of the matching domains (default 5)
  -d, --depth int              MaxDepth limits the recursion depth of visited URLs. (Set it to 0 for infinite recursion) (default 0)
  -k, --delay int              Delay is the duration to wait before creating a new request to the matching domains (second)
  -K, --random-delay int       RandomDelay is the extra randomized duration to wait added to Delay before creating a new request
  -m, --timeout int            Request timeout (second) (default 10)
  -B, --base                   Disable all and only use HTML content
                              Enable linkfinder in javascript file (default true)
  --js                         Try to crawl sitemap.xml
  --sitemap                    Try to crawl robots.txt (default true)
  -a, --other-source           Find URLs from 3rd party (Archive.org, CommonCrawl.org, VirusTotal.com, AlienVault.com)
  -w, --include-sub            Include subdomains crawled from 3rd party. Default is main domain
  -r, --include-other-source   Also include other-source's urls (still crawl and request)
  --subs                       Include subdomains
  --debug                      Turn on debug mode
  --json                       Enable JSON output
  -v, --verbose                Turn on verbose
  -q, --quiet                  Suppress all the output and only show URL
  --no-redirect                Disable redirect
  --version                     Check version

```

Figure 76 Verified installation of gospider

```

• (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ go install -v github.com/ffuf/ffuf/v2@latest
container/ring
golang.org/x/net/html/atom
os/signal
github.com/adrg/xdg/internal/pathutil
github.com/ffuf/pencode/pkg/pencode
github.com/pelletier/go-toml
golang.org/x/net/html
github.com/adrg/xdg
net/http/httputil
github.com/andybalholm/brotli
github.com/andybalholm/cascadia
github.com/ffuf/ffuf/v2/pkg/ffuf
github.com/PuerkitoBio/goquery
github.com/ffuf/ffuf/v2/pkg/filter
github.com/ffuf/ffuf/v2/pkg/input
github.com/ffuf/ffuf/v2/pkg/interactive
github.com/ffuf/ffuf/v2/pkg/output
github.com/ffuf/ffuf/v2/pkg/scraper
github.com/ffuf/ffuf/v2/pkg/runner
• (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ ffuf -h
Fuzz Faster U Fool - v1.1.0

HTTP OPTIONS:
  -H           Header "Name: Value", separated by colon. Multiple -H flags are accepted.
  -X           HTTP method to use (default: GET)
  -b           Cookie data "NAME1=VALUE1; NAME2=VALUE2" for copy as curl functionality.
  -d           POST data
  -ignore-body Do not fetch the response content. (default: false)
  -r           Follow redirects (default: false)
  -recursion   Scan recursively. Only FUZZ keyword is supported, and URL (-u) has to end in it. (default: false)
  -recursion-depth Maximum recursion depth. (default: 0)
  -replay-proxy Replay matched requests using this proxy.
  -timeout     HTTP request timeout in seconds. (default: 10)
  -u           Target URL
  -x           HTTP Proxy URL

GENERAL OPTIONS:
  -V           Show version information. (default: false)
  -ac          Automatically calibrate filtering options (default: false)
  -acc         Custom auto-calibration string. Can be used multiple times. Implies -ac
  -c           Colorize output. (default: false)
  -maxtime     Maximum running time in seconds for entire process. (default: 0)
  -maxtime-job Maximum running time in seconds per job. (default: 0)

```

Figure 77 Install and Verified ffuf tool

```

• (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ sudo apt install -y cewl
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
cewl is already the newest version (5.5.2-1).
The following packages were automatically installed and are no longer required:
  golang-1.18 golang-1.18-doc golang-1.18-go golang-1.18-src golang-src libxdo3
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 22 not upgraded.
• (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ cewl --help
CeWL 5.5.2 (Grouping) Robin Wood (robin@diginiinja) (https://diginiinja/)
Usage: cewl [OPTIONS] ... <url>

OPTIONS:
  -h, --help: Show help.
  -k, --keep: Keep the downloaded file.
  -d <x>, --depth <x>: Depth to spider to, default 2.
  -m, --min_word_length: Minimum word length, default 3.
  -o, --offsite: Let the spider visit other sites.
  --exclude: A file containing a list of paths to exclude
  --allowed: A regex pattern that path must match to be followed
  -w, --write: Write the output to the file.
  -u, --ua <agent>: User agent to send.
  -n, --no-words: Don't output the wordlist.
  -g <x>, --groups <x>: Return groups of words as well
  --lowercase: Lowercase all parsed words
  --with-numbers: Accept words with numbers in as well as just letters
  --convert-umlauts: Convert common ISO-8859-1 (Latin-1) umlauts (ä-ae, ö-oe, ü-ue, ß-ss)
  -a, --meta: include meta data.
  --meta_file file: Output file for meta data.
  -e, --email: Include email addresses.
  --email_file <file>: Output file for email addresses.
  --meta-temp-dir <dir>: The temporary directory used by exiftool when parsing files, default /tmp.
  -c, --count: Show the count for each word found.
  -v, --verbose: Verbose.
  --debug: Extra debug information.

Authentication
  --auth_type: Digest or basic.
  --auth_user: Authentication username.
  --auth_pass: Authentication password.

Proxy Support

```

Figure 78 Install and verified cewl

```

(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ go install -v github.com/projectdiscovery/dnsx/cmd/dnsx@latest
github.com/projectdiscovery/utls/file
github.com/projectdiscovery/utls/maps
github.com/projectdiscovery/utls/reader
github.com/projectdiscovery/utls/errors
github.com/projectdiscovery/gologger
github.com/projectdiscovery/ratelimit
github.com/projectdiscovery/utls/url
github.com/projectdiscovery/utls/folder
github.com/projectdiscovery/utls/process
github.com/projectdiscovery/retryabledns
github.com/projectdiscovery/hmap/store/hybrid
github.com/projectdiscovery/utls/update
github.com/projectdiscovery/goflags
github.com/projectdiscovery/networkpolicy
github.com/projectdiscovery/fastdialer/fastdialer/metafiles
github.com/projectdiscovery/cdncheck
github.com/projectdiscovery/fastdialer/fastdialer
github.com/projectdiscovery/dnsx/libs/dnsx
github.com/projectdiscovery/retryablehttp-go
github.com/projectdiscovery/utls/auth/pdcp
github.com/projectdiscovery/asnmap/libs
github.com/projectdiscovery/mapcidr/asn
github.com/projectdiscovery/dnsx/internal/runner
github.com/projectdiscovery/dnsx/cmd/dnsx
• (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ dnsx -h
dnsx is a fast and multi-purpose DNS toolkit allow to run multiple probes using retryabledns library.

Usage:
  dnsx [flags]

Flags:
INPUT:
  -l, --list string      list of sub(domains)/hosts to resolve (file or stdin)
  -d, --domain string    list of domain to bruteforce (file or comma separated or stdin)
  -w, --wordlist string   list of words to bruteforce (file or comma separated or stdin)

QUERY:
  -a                  query A record (default)
  -aaaa              query AAAA record
  -cname              query CNAME record
  -ns                 query NS record
  -txt                query TXT record
  -srv                query SRV record
  -ptr                query PTR record

```

Figure 79 Install and Verified dnsx


```

• (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ go install -v github.com/projectdiscovery/shuffledns/cmd/shuffledns@latest
go: downloading github.com/projectdiscovery/shuffledns v1.2.1
go: github.com/projectdiscovery/shuffledns@v1.2.1 requires go >= 1.24.0; switching to go1.24.11
github.com/projectdiscovery/shuffledns/cmd/shuffledns
github.com/projectdiscovery/shuffledns/pkg/parser
github.com/projectdiscovery/shuffledns/pkg/batcher
github.com/projectdiscovery/shuffledns/pkg/store
github.com/projectdiscovery/shuffledns/pkg/dnsx
github.com/projectdiscovery/shuffledns/pkg/update
github.com/projectdiscovery/shuffledns/pkg/wildcards
github.com/projectdiscovery/shuffledns/pkg/massdns
github.com/projectdiscovery/shuffledns/pkg/runner
github.com/projectdiscovery/shuffledns/cmd/shuffledns
• (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ shuffledns -h
shuffledns is a wrapper around massdns written in go that allows you to enumerate valid subdomains using active bruteforce as well as resolve subdomains with
ng and easy input-output support.

Usage:
  shuffledns [flags]

Flags:
INPUT:
  -d, -domain string[]      Domain to find or resolve subdomains for
  -ad, -auto-domain         Automatically extract root domains
  -l, -list string          File containing list of subdomains to resolve
  -w, -wordlist string       File containing words to bruteforce for domain
  -r, -resolver string      File containing list of resolvers for enumeration
  -tr, -trusted-resolver string File containing list of trusted resolvers
  -ri, -raw-input string    Validate raw full massdns output
  -mode string              Execution mode (bruteforce, resolve, filter)

RATE-LIMIT:
  -t int Number of concurrent massdns resolves (default 10000)

UPDATE:
  -up, -update           update shuffledns to latest version
  -duc, -disable-update-check disable automatic shuffledns update check

OUTPUT:
  -o, -output string      File to write output to (optional)
  -j, -json              Make output format as ndjson
  -wo, -wildcard-output string Dump wildcard ips to output file

```

Figure 80 Install and Verified Shuffledns

```

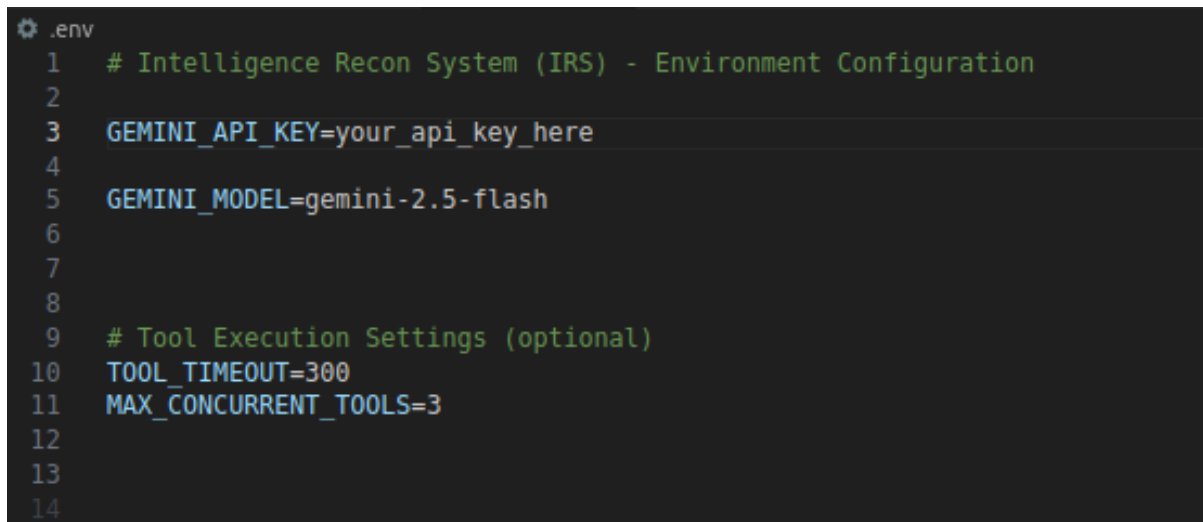
# BUILD COMMAND
command = self.build_command(target, output_file, **kwargs)
self.logger.info(f"Executing: {' '.join(command)}")

# Execute command
process = await asyncio.create_subprocess_exec(
    *command,
    stdout=asyncio.subprocess.PIPE,
    stderr=asyncio.subprocess.PIPE
)

try:
    stdout, stderr = await asyncio.wait_for(
        process.communicate(),
        timeout=timeout
    )

```

Figure 81 Asynchronous subprocess execution in the backend



```
.env
1  # Intelligence Recon System (IRS) - Environment Configuration
2
3  GEMINI_API_KEY=your_api_key_here
4
5  GEMINI_MODEL=gemini-2.5-flash
6
7
8
9  # Tool Execution Settings (optional)
10 TOOL_TIMEOUT=300
11 MAX_CONCURRENT_TOOLS=3
12
13
14
```

Figure 82 IRS environment configuration (.env) for Gemini and tool settings

2. Sprint 2 and Sprint 3

Sprint 2 involved development of the backend core, endpoints, database structures and services to store and normalize scan results of multiple reconnaissance tools. It also brought about asynchronous orchestration in case scans could be performed in the background and status, detail and reporting endpoints to be available later to integrate with the frontend.

Sprint 3 was aimed at creating the intelligent scan workflow, beginning with Naabu and Httpx to gather early evidence such as open ports and HTTP services to each target. All these clues are then transferred to an AI decision layer that is expected to select the most applicable reconnaissance tools in each target and OWASP category. The decision logic has been incorporated in the backend orchestration in a way that its output is able to self-control what tools to operate and what to store as the outcome. The integration of Gemini is not fully completed, and API settings and timely tuning are required to make the full utilization of AI-guided tool selection possible.

```

# Create FastAPI app
app = FastAPI(
    title="Intelligence Recon System (IRS)",
    description="AI-assisted OWASP Top 10 reconnaissance platform",
    version="1.0.0",
    lifespan=lifespan
)

# Add CORS middleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # In production, specify allowed origins
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

```

Figure 83 Fast API and CORS setup

```

router = APIRouter(prefix="/scans", tags=["scans"])

@router.post("", response_model=ScanResponse)
async def create_scan(
    scan_data: ScanCreate,
    db: AsyncSession = Depends(get_db)
):
    # Validate selected tools
    invalid_tools = [
        tool for tool in scan_data.selected_tools
        if tool not in settings.AVAILABLE_TOOLS
    ]

    if invalid_tools:
        raise HTTPException(
            status_code=400,
            detail=f"Invalid tools: {' '.join(invalid_tools)}"
        )

    # Require at least one tool to be selected by the user
    if not scan_data.selected_tools:
        raise HTTPException(
            status_code=400,
            detail="You must select at least one tool before starting a scan. "
            "AI will refine this list based on clues, but cannot choose tools from an empty selection."
        )

    # Create scan
    scan = await ScanService.create_scan(
        db=db,
        target=scan_data.target,
        owasp_category=(parameter) scan_data: ScanCreate,
        selected_tools=scan_data.selected_tools
    )

    return ScanResponse(
        scan_id=scan.id,
        status=scan.status.value,
        message="Scan created successfully. AI decision and tool execution in progress."
    )

```

Figure 84 Scan creation endpoint with tool validation in the IRS Fast Api backend

```

5         )
6
7         return ScanListResponse(**result)
8
9     @router.delete("/{scan_id}")
10    async def delete_scan(
11        scan_id: int,
12        db: AsyncSession = Depends(get_db)
13    ):
14        """Delete a scan and its associated stored data."""
15        deleted = await ScanService.delete_scan(db, scan_id)
16        if not deleted:
17            raise HTTPException(status_code=404, detail="Scan not found")
18        return JSONResponse(status_code=200, content={"detail": "Scan deleted"})
19
20    @router.post("/{scan_id}/retry", response_model=ScanResponse)
21    async def retry_scan(
22        scan_id: int,
23        db: AsyncSession = Depends(get_db)
24    ):
25
26        scan = await ScanService.retry_scan(db, scan_id)
27
28        return ScanResponse(
29            scan_id=scan.id,
30            status=scan.status.value,
31            message="Scan retry initiated. AI decision and tool execution in progress."
32        )
33
34

```

Figure 85 Scan delete and retry endpoint for managing scans in IRS API

```

@router.get("/{scan_id}/status", response_model=ScanStatus)
async def get_scan_status(
    scan_id: int,
    db: AsyncSession = Depends(get_db)
):
    status = await ScanService.get_scan_status(db, scan_id)

    if not status:
        raise HTTPException(status_code=404, detail="Scan not found")

    return ScanStatus(**status)

@router.get("/{scan_id}", response_model=ScanDetailResponse)
async def get_scan_detail(
    scan_id: int,
    db: AsyncSession = Depends(get_db)
):
    detail = await ScanService.get_scan_detail(db, scan_id)

    if not detail:
        raise HTTPException(status_code=404, detail="Scan not found")

    return ScanDetailResponse(**detail)

@router.get("", response_model=ScanListResponse)
async def list_scans(
    target: Optional[str] = Query(None, description="Filter by target"),
    status: Optional[str] = Query(None, description="Filter by status"),
    page: int = Query(1, ge=1, description="Page number"),
    page_size: int = Query(20, ge=1, le=100, description="Items per page"),
    db: AsyncSession = Depends(get_db)
):
    result = await ScanService.list_scans(
        db=db,
        target=target,
        status=status,
        page=page,
        page_size=page_size
    )

```

Figure 86 Scan status, detail, and list endpoints querying IRS scan results

```

class ScanStatus(str, enum.Enum):
    """Scan status enumeration."""
    CREATING = "creating"
    PREPARING = "preparing"
    RUNNING = "running"
    COMPLETED = "completed"
    COMPLETED_WITH_ERRORS = "completed_with_errors"
    FAILED = "failed"

class Scan(BaseModel):
    """Scan model representing a complete recon run for a target."""

    __tablename__ = "scans"

    # Target information
    target = Column(String(255), nullable=False, index=True)
    owasp_category = Column(String(20), nullable=False, index=True)

    # Status tracking
    status = Column(SQLEnum(ScanStatus), default=ScanStatus.CREATING, nullable=False, index=True)
    completed_at = Column(DateTime, nullable=True)

    # Tool selection
    user_selected_tools = Column(Text, nullable=True) # JSON list of tool names
    ai_tools_to_run = Column(Text, nullable=True) # JSON list of tool names AI recommended
    ai_tools_skipped = Column(Text, nullable=True) # JSON list of {tool, reason}
    ai_raw_response = Column(Text, nullable=True) # Raw AI response for debugging

    # Error tracking
    error_summary = Column(Text, nullable=True)

    # Relationships
    tool_runs = relationship("ToolRun", back_populates="scan", cascade="all, delete-orphan")
    findings = relationship("Finding", back_populates="scan", cascade="all, delete-orphan")

```

Figure 87 Scan status enum and scan model

```

class ToolRunStatus(str, enum.Enum):
    """Tool run status enumeration."""
    QUEUED = "queued"
    RUNNING = "running"
    COMPLETED = "completed"
    FAILED = "failed"
    TIMEOUT = "timeout"

class ToolRun(BaseModel):
    """ToolRun model representing execution of a single tool within a scan."""

    __tablename__ = "tool_runs"

    # Scan relationship
    scan_id = Column(Integer, ForeignKey("scans.id", ondelete="CASCADE"), nullable=False, index=True)
    scan = relationship("Scan", back_populates="tool_runs")

    # Tool information
    tool_name = Column(String(100), nullable=False, index=True)
    status = Column(SQLEnum(ToolRunStatus), default=ToolRunStatus.QUEUED, nullable=False)

    # Execution timing
    started_at = Column(DateTime, nullable=True)
    finished_at = Column(DateTime, nullable=True)

    # Output and results
    raw_output_path = Column(String(500), nullable=True) # Path to raw output file
    summary = Column(Text, nullable=True) # Short summary of results
    error_message = Column(Text, nullable=True) # Error message if failed

```

Figure 88 Tool run status enum and Tool Run model


```

class FindingType(str, enum.Enum):
    """Finding type enumeration."""
    ASSET = "asset"
    ENDPOINT = "endpoint"
    PORT = "port"
    VULNERABILITY = "vulnerability"
    MISCONFIGURATION = "misconfiguration"
    INFORMATION = "information"

class FindingSeverity(str, enum.Enum):
    """Finding severity enumeration."""
    INFO = "info"
    LOW = "low"
    MEDIUM = "medium"
    HIGH = "high"
    CRITICAL = "critical"

class Finding(BaseModel):
    """Finding model representing a normalized piece of reconnaissance data."""
    __tablename__ = "findings"

    # Scan relationship
    scan_id = Column(Integer, ForeignKey("scans.id", ondelete="CASCADE"), nullable=False, index=True)
    scan = relationship("Scan", back_populates="findings")

    # Source information
    tool_name = Column(String(100), nullable=False, index=True)

    # Classification
    type = Column(SQLEnum(FindingType), nullable=False, index=True)
    severity = Column(SQLEnum(FindingSeverity), default=findingseverity.INFO, nullable=False, index=True)
    owasp_category = Column(String(20), nullable=True, index=True)

    # Finding details
    location = Column(String(500), nullable=False) # URL, host:port, or resource identifier
    description = Column(Text, nullable=False)
    evidence = Column(Text, nullable=True) # Supporting data (headers, payloads, etc.)

```

Figure 89 Finding enums and Finding model

```

class ScanService:
    """Service for managing scan lifecycle and orchestration."""

    @staticmethod
    async def create_scan(
        db: AsyncSession,
        target: str,
        owasp_category: str,
        selected_tools: List[str]
    ) -> Scan:
        # Create scan record
        scan = Scan(
            target=target,
            owasp_category=owasp_category,
            status=ScanStatus.CREATING,
            user_selected_tools=json.dumps(selected_tools)
        )

        db.add(scan)
        await db.commit()
        await db.refresh(scan)

        logger.info(f"Created scan {scan.id} for target={target}, owasp={owasp_category}")

        # Start background workflow
        asyncio.create_task(
            ScanService._execute_scan_workflow(scan.id, target, owasp_category, selected_tools)
        )

        return scan

```

Figure 90 Scan Service create_scan

```

@staticmethod
async def _gather_initial_clues(scan_id: int, target: str) -> Dict[str, Any]:
    """Run initial clues tools and gather reconnaissance data.

    Args:
        scan_id: Scan ID
        target: Target domain

    Returns:
        Dictionary of clues
    """
    clues = {
        "open_ports": [],
        "http_services": [],
        "server_headers": [],
        "page_titles": [],
        "status_codes": [],
        "technologies": []
    }

    # Run Naabu (port scan)
    try:
        naabu_result = await execute_tool("Naabu", target, scan_id, timeout=120)
        if naabu_result.success:
            ports = [f["location"].split(":")[-1] for f in naabu_result.findings]
            clues["open_ports"] = ports[:20] # Limit
    except Exception as e:
        logger.error(f"Naabu clues failed: {e}")

    # Run Httpx (HTTP probe)
    try:
        httpx_result = await execute_tool("Httpx", target, scan_id, timeout=120)
        if httpx_result.success:
            for finding in httpx_result.findings:
                try:
                    evidence = json.loads(finding.get("evidence", "{}"))
                    clues["http_services"].append(evidence.get("url", ""))
                    if evidence.get("title"):
                        clues["page_titles"].append(evidence["title"])
                    if evidence.get("status_code"):
                        clues["status_codes"].append(str(evidence["status_code"]))
                    if evidence.get("technologies"):
                        clues["technologies"].extend(evidence["technologies"])
                except json.JSONDecodeError:
                    pass
    except Exception as e:
        logger.error(f"Httpx clues failed: {e}")

    logger.info(f"Clues gathered: {clues}")
    return clues

```

Figure 91 Gather initial clues with Naabu and Httpx

```

6 @staticmethod
7 async def _execute_tools(
8     db: AsyncSession,
9     scan_id: int,
10    target: str,
11    tool_names: List[str]
12 ) -> None:
13
14    async def execute_one_tool(tool_name: str):
15
16        # Get tool run record
17        result = await db.execute(
18            select(ToolRun).where(
19                ToolRun.scan_id == scan_id,
20                ToolRun.tool_name == tool_name
21            )
22        )
23        tool_run = result.scalar_one_or_none()
24
25        if not tool_run:
26            logger.error(f"ToolRun not found for {tool_name}")
27            return
28
29        tool_run.status = ToolRunStatus.RUNNING
30        tool_run.started_at = datetime.utcnow()
31        await db.commit()
32
33        tool_result = await execute_tool(tool_name, target, scan_id)
34
35        tool_run.finished_at = datetime.utcnow()
36
37        if tool_result.success:
38            tool_run.status = ToolRunStatus.COMPLETED
39            tool_run.summary = tool_result.summary
40            tool_run.raw_output_path = str(
41                settings.SCANS_DIR / str(scan_id) / f"{tool_name.lower()}.out"
42            )
43
44            for finding_data in tool_result.findings:
45                finding = Finding(
46                    scan_id=scan_id,
47                    tool_name=tool_name,
48                    type=finding_data.get("type", "information"),
49                    severity=finding_data.get("severity", "info"),
50                    owasp_category=finding_data.get("owasp_category"),
51                    location=finding_data.get("location", ""),
52                    description=finding_data.get("description", ""),
53                    evidence=finding_data.get("evidence")
54                )
55                db.add(finding)
56            else:
57                if tool_result.error_message and "timeout" in tool_result.error_message.lower():
58                    tool_run.status = ToolRunStatus.TIMEOUT
59                else:
60                    tool_run.status = ToolRunStatus.FAILED
61                    tool_run.error_message = tool_result.error_message
62
63            await db.commit()
64            logger.info(f"Completed {tool_name}: {tool_run.status}")
65        for tool_name in tool_names:
66            await execute_one_tool(tool_name)

```

Figure 92 Execute recon tools and save findings


```

14 @dataclass
15 class AIDecision:
16     """AI decision result."""
17     tools_to_run: List[str]
18     tools_skipped: List[Dict[str, str]]
19     execution_batches: Optional[List[List[str]]] = None
20     reasoning: Optional[str] = None
21     raw_response: Optional[str] = None
22     success: bool = True
23     error: Optional[str] = None
24
25     async def decide_tools(
26         target: str,
27         owasp_category: str,
28         owasp_name: str,
29         selected_tools: List[str],
30         clues: Dict[str, Any]
31     ) -> AIDecision:
32
33         logger.info(f"Requesting AI decision for target={target}, owasp={owasp_category}")
34
35         if not settings.GEMINI_API_KEY:
36             logger.error("GEMINI_API_KEY not configured")
37             return _fallback_decision(selected_tools, "Gemini API key not configured")
38
39         try:
40             # Build prompt
41             prompt = build_decision_prompt(target, owasp_category, owasp_name, selected_tools, clues)
42
43             # Call Gemini API
44             url = f"https://generativelanguage.googleapis.com/v1beta/models/{settings.GEMINI_MODEL}:generateContent?key={settings.GEMINI_API_KEY}"
45
46             payload = {
47                 "contents": [{
48                     "parts": [{
49                         "text": prompt
50                     }]
51                 }],
52                 "generationConfig": {
53                     "temperature": 0.3,
54                     "maxOutputTokens": 2048,
55                 }
56             }
57
58             async with httpx.AsyncClient(timeout=30.0) as client:
59                 response = await client.post(url, json=payload)
60                 response.raise_for_status()
61
62             result = response.json()
63
64             # Extract response text
65             try:
66                 response_text = result["candidates"][0][0]["content"]["parts"][0][0]["text"]
67             except (KeyError, IndexError) as e:
68                 logger.error(f"Unexpected Gemini response structure: {e}")
69                 return _fallback_decision(selected_tools, "Invalid API response structure")
70
71             logger.info(f"Raw AI response: {response_text}")

```

Figure 93 AI decision class and Gemini API call

```

12
13 async def execute_tool(
14     tool_name: str,
15     target: str,
16     scan_id: int,
17     timeout: Optional[int] = None
18 ) -> ToolResult:
19
20     # Get tool class
21     tool_class = TOOL_REGISTRY.get(tool_name)
22
23     if not tool_class:
24         logger.error(f"Unknown tool: {tool_name}")
25         return ToolResult(
26             tool_name=tool_name,
27             success=False,
28             summary="Unknown tool",
29             error_message=f"Tool '{tool_name}' not found in registry"
30         )
31
32     # Create output file path
33     output_dir = settings.SCANS_DIR / str(scan_id)
34     output_dir.mkdir(parents=True, exist_ok=True)
35     output_file = output_dir / f"{tool_name.lower()}.out"
36
37     # Execute tool
38     tool: BaseTool = tool_class()
39     timeout_value = timeout or settings.TOOL_TIMEOUT
40
41     logger.info(f"Executing {tool_name} for scan {scan_id}, target={target}, timeout={timeout_value}s")
42
43     result = await tool.execute(
44         target=target,
45         output_file=output_file,
46         timeout=timeout_value
47     )
48

```

Figure 94 Execute a single tool with timeout and output file

[Click here](#) to go back

8.3. Appendix Project risks, threats, and contingency plans

Threats and risks can be encountered during the project development process since every single system can be exposed to risks. The following risks and threats are mitigated along with contingency plans to minimize systemic defects.

- **AI Triage Dependency Risk**

Risk: AI Triage (Gemini API) provides faulty filtering, which reduces the effectiveness of scans.

Threats: Gemini API update software failure, AI misunderstands target or proposes a tool that is not relevant.

Contingency plan

- Introduce Manual Override and Fallback to operate 5 most important tools without AI.
- Have a strict version locked prompt template.

- **Creation of tool Risk and Data Inconsistency risk.**

Risk: Security tools malfunction, crash system or invalid output of raw results.

Threats: Critical tool will not run with update, change of format output or tool requires unconfigured environmental variables.

Contingency plan

- Install Version Locking and Isolation on all the 15 security tools.
- Crash of subprocesses is isolated using the try/except in Python.
- In case of parsing failures, do away with inconsistent data and record the error.

- **Performance and Scalability risk.**

Risk: Asynchronous Execution Core is inefficient in managing concurrent tools, thus leading to system lag.

Threats: Subprocess module will be a bottleneck, server will run out of resources or database will write slowly.

Contingency plan

- A Fixed Parallel Limit will be implemented to control simultaneous tools.
- Make use of the inbuilt asyncio of Python to have a responsive UI.

8.4. Expected Outcomes and Deliverables

The purpose of the IRS project is to replace manually executed, time-intensive command lines with a singular, user-friendly system driven by AI-based intelligence. Enhancing vulnerability assessment process is done through automating the process of selecting and executing security tools according to the real-time context of the situation and the intent of the user, this reduces the amount of human effort and maximizes efficiency.

8.5. Deliverables

1. IRS

The complete system can execute hyper-focused, AI-optimized security assessments via one web screen.

2. Unified Interface and Monitoring Dashboard

This allows analysts to manage all security scans, track real-time progress, and review aggregated findings from a single central hub.

3. Structured Output and Persistent Results

It transforms raw data so that the user can easily analyze the standardized results and securely store all the historical data for long-term review.

4. Security Tool Execution Layer

This layer runs the 15 in-built tools, enabling the system to execute a complete and powerful suite of security checks instructed by the AI.

5. Downloadable Standardized Report

Users can generate professional, ready-to-share HTML reports for audits, documentation, or incident review.

8.6. Resource Requirements

8.6.1. Hardware Requirement

Personal devices (Laptop) will be used as development machines.

8.6.2. Software Requirement

Category	Software Requirements	Specifications
OS	Ubuntu	Open-source and stable Linux distribution is the development environment.
Development Tools	VS Code	I used IDE to code, debug, and version control.
	HTML, CSS, JavaScript	Technologies of User Interface developments.
	Python/Fast API	Python backend language: fast Api, high performance, asynchronous API.
Core AI Model	Gemini2.5 Flash	Intelligent triage and data validation of the AI Decision Node
Data Storage	PostgreSQL or Firestore	Storage of all scan results and history record keeping.
Gantt Chart	Click up	Project timeline and progress tracking visual tool.
Documentation	MS Word	Keep records of project.
Diagram	Eraser, Draw.io, Visual Paradigm	Shows ideas and process.
Version control	Git	Tracks the changes to the source file
Project Management Tool	Trello	Used to keep track of milestones

Table 4 Table of Software Requirements